

大模型技术创新赛

1.1 竞赛规则

一、 赛题背景

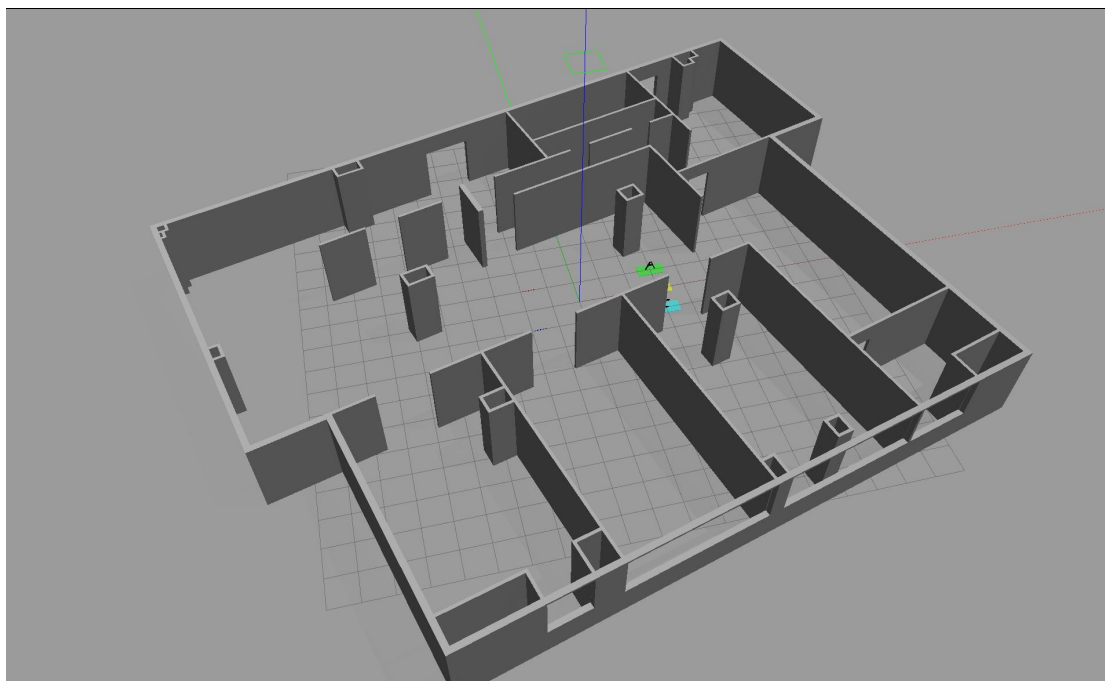
大模型技术创新赛是以人工智能前沿技术为核心驱动的综合性竞赛，聚焦大模型技术的突破性应用与生态构建，旨在激发高校团队探索大模型在自然语言处理、多模态交互、知识推理等领域的创新潜力。赛事通过开放真实场景数据集与算力资源，鼓励参赛者攻克模型轻量化、推理效率优化、垂直领域适配等关键技术难题，倡导构建可持续的 AI 创新生态。比赛不仅为技术人才提供展示与交流平台，更致力于打通产学研用链条，加速大模型技术从实验室研究向产业赋能的转化进程，助力数字经济时代智能化升级。

为推动大模型技术在智能物流领域的创新应用，本赛题以仓储基地分拣及搬运为主题，参赛选手需基于指定设备设计一套智能作业系统，结合大模型、计算机视觉、导航、机械臂抓取技术，解决货物识别与导航、精准抓取与放置等核心问题。本赛项旨在激发参赛选手的创新能力，推动智能物流技术与可持续发展的融合。

二、 比赛内容

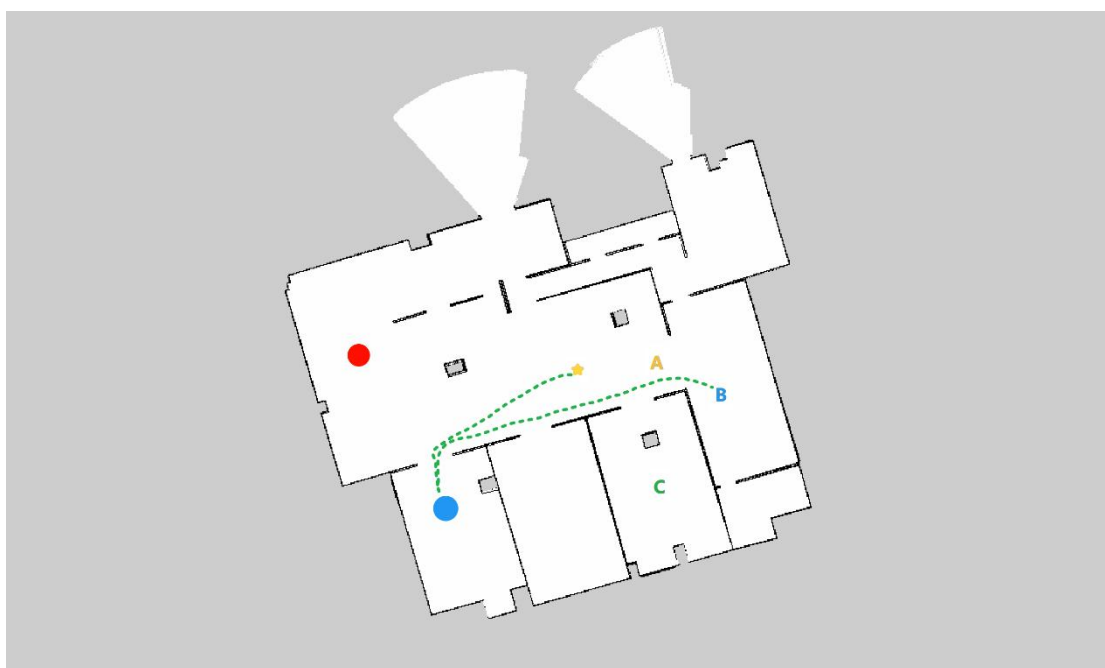
比赛使用仿真平台，结合大模型、ROS2 机器人操作系统、视觉技术等主流技术，开发一套目标识别到作业执行的智能系统。

比赛开始利用移动机器人规划最优路径到达目标区域，避开障碍物；基于大模型和计算机视觉技术，识别仓储中的目标货物（如红色或蓝色包裹），并生成位置信息；执行货物抓取、搬运与放置动作，将目标物拾取后放置于指定区域；通过语言大模型用自然语言发布命令，设计可视化界面显示作业内容。参赛队伍需根据提供的仿真环境（Gazebo + ROS2），设计算法方案并实现功能。



仿真场景地图示意图

在仿真场景中，根据收到的命令，如“需要搬运 2 个蓝色包裹到 B 区”，利用大语言模型解析、分解为任务指令；完成控制机器人实现导航到货物位置，识别指定颜色的包裹，控制机械臂抓取指定货物，控制机器人导航到放置区域等任务。



流程示意图

三、 竞赛规则

比赛仅 1 轮，限时 5 分钟。

(1) 裁判发布搬运物体颜色与放置区域任务(如:搬运 2 个红色包裹到 A 区, 3 个蓝色包裹至 C 区)。

(2) 参赛队伍与裁判申请比赛开始;

(3) 开始计时, 同时启动机器人作业;

(4) 比赛过程中, 机器人通过 SLAM、大模型与视觉系统等技术, 发布任务、识别目标并规划路径, 完成抓取、搬运和放置任务;

(5) 机器人完成抓取、搬运和放置任务, 同时需要将机器人工作状态实时显示到 FoxGlove 中。

3. 比赛结束

(1) 比赛过程中, 参赛队员举手示意结束, 则比赛结束; 若时间截止, 比赛自动结束。

(2) 比赛结束, 选手确认成绩后离场。

评审打分表

序号	评分项	得分	备注
1	系统启动, 机器人与场景完全成功加载并显示在仿真环境中 (10 分)		机器人与场景完全加载得分, 未加载完全不得分。
2	指令解析, 成功发布大模型指令并将指令解析到 FoxGlove 中显示 (10 分)		正确将指令解析显示到 FoxGlove 中即得分, 解析错误或未成功解析不得分。
3	货物抓取, 机械臂成功抓取到货物 (20 分)		每抓取到一个货物得 4 分, 总分为 20 分。
4	货物搬运放置, 成功抓取目标并放置于指定区域 (20 分)		每成功到一个货物得 4 分, 总分为 20 分。
5	机器人避障, 机器人成功完成场景中对可移动物体的避障 (10 分)		每增加 1 个移动物体并成功实现避障得 5 分, 总分为 10 分。
6	状态反馈, 成功在 FoxGlove 中显示路径规划轨迹、机械臂抓取状态、每一轮任务完成情况 (10 分)		每次完成状态有效反馈即得分, 否则不得分。
7	场景设计分, 构建复杂地形并对机器人完成任务增加复杂度 (10 分)		根据场景中复杂度评分。
8	技术文档 (20 分)		考察参赛选手算法的创新性、逻辑性。
总分			

1.2 竞赛分析

为了让大家更深入的了解该赛事的竞赛核心, 该小节从 Linux 系统简介, 虚拟机和 U 盘双系统使用, ROS2 机器人操作系统, Gazebo 仿真环境, Nav2 导航系统, MoveIt2 机械臂控制, 计算机视觉技术, 语言大模型的应用, 可视化工具使用多个角度进行竞赛分析。

一、Linux 系统简介

Ubuntu 是 ROS2 的最佳开发环境, 其开源特性和稳定性能为机器人开发提

供有力支持。需要学习 Linux 的基本操作，ROS 发行版与 Ubuntu 系统的紧密关联，尽管 ROS2 对 Windows 等平台有一定支持，但 Ubuntu 仍是最优选择。同时，需掌握 Linux 命令行操作，如 `cd`、`pwd`、`mkdir`、`ls`、`gedit` 等常用命令，这些命令在搭建开发环境、编译代码、运行程序等过程中必不可少。

二、 虚拟机和 U 盘双系统使用

虚拟机为在 Windows 系统上运行 Ubuntu 系统提供了便捷方式，需学习虚拟机的基本使用，包括 Linux 在虚拟机中的搭建与使用。重点掌握虚拟机网络配置，了解 NAT、桥接、仅主机三种网卡类型的特点，根据需求选择合适的模式，如桥接模式可实现虚拟机与外部网络的通信，确保 ROS 程序能正常与外部交互。

U 盘双系统则通过启动盘在物理机上独立运行 Ubuntu，直接使用物理机硬件，避免虚拟机性能损耗，适合对算力要求较高的场景。需掌握启动盘的使用、BIOS 启动项设置，双系统切换。

三、 ROS2 机器人操作系统

ROS2 是整个系统的核心通信框架，是开展后续所有操作（包括 Gazebo 仿真环境的使用、各模块间的交互等）的基础。需熟练掌握其话题（Topics）、服务（Services）和动作（Actions）的使用。比如，通过话题控制机器人速度，利用动作控制机械臂运动，调用服务实现夹爪对物体的抓取与释放。此外，还需了解 ROS2 的节点管理、消息类型定义等知识，确保各个模块之间能够高效、稳定地通信。

四、 Gazebo 仿真环境

比赛基于 Gazebo 仿真平台，而 Gazebo 的使用依赖于 ROS2 的支持，所有的视觉识别、导航规划、机械臂控制等操作都需要在仿真环境中进行验证。通过 `ros2` 命令启动包含机械臂、底盘和传感器的仿真环境，能够在仿真环境中添加障碍物、设置货物位置等。此外，还需了解仿真环境中传感器（雷达、相机等），例如雷达的扫描范围和精度、相机等参数设置，以便更好地利用这些数据进行决策。

五、 Nav2 导航系统

Nav2 是移动机器人实现自主导航的核心框架，其整合了感知、路径规划、运动控制与定位等核心功能，能够支撑机器人在存在动态障碍、完成从起点到目

标点的自主移动。

在应用层面，需掌握如何向 Nav2 系统发布目标点以触发导航任务，以及如何通过设置初始位姿估计为机器人提供全局定位基准。同时，要学会通过订阅导航过程中生成的路径规划数据，实时监控导航状态，确保机器人能够高效避开障碍并精准抵达目标区域。

六、机械臂控制和 MoveIt2

在 Gazebo 仿真环境中，机械臂控制虽提供了基础的关节控制接口（如直接发送关节角度指令）和关节状态反馈功能，但直接通过关节角度进行精细化作业（如精准抓取特定位置的货物）存在显著难度。

MoveIt2 用于机械臂运动规划与控制。其核心价值在于通过封装运动学算法与路径优化逻辑，将复杂的机械臂控制转化为对末端执行器目标位姿的直接设定。

通过 MoveIt2，可实现从“目标货物抓取点”到“机械臂末端运动轨迹”的自动转换，无需手动计算关节角度，即可精准控制机械臂完成抓取，大幅降低机械臂操作的复杂度。

七、计算机视觉技术

计算机视觉技术用于目标识别与定位，需学习图像处理基础知识，如使用 OpenCV 进行颜色阈值分割、轮廓检测，识别货物颜色和形状等。

八、语言大模型的应用

语言大模型（LLM）在人机交互中的应用场景广泛且具有变革性，**智能客服与虚拟助手、车载与物联网交互**等。

在技术层面通常的方案有 **NLP 解析、MCP 架构、Function Calling** 等。从自然语言提取关键信息再映射为类机器人指令，控制机器人完成任务。

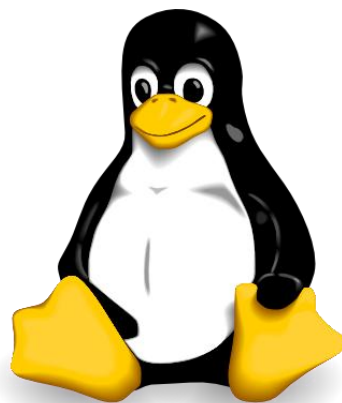
九、可视化工具使用

可视化工具（rqt、rviz、foxglove）用于监控机器人状态，需学习各工具的功能和使用方法。如 rqt 监控节点和话题，rviz 展示机器人模型和传感器数据，foxglove 通过 Rosbridge WebSocket protocol 实现数据可视化，支持自定义布局整合暂时的信息。

1.3 竞赛实战指导

一、Linux 系统简介

时间回到 1991 年，一位热爱计算机的芬兰大学生林纳斯，在熟悉了操作系统原理和 unix 系统后，决定自己动手做一个，实践是检验真理的唯一标准，说干就干，他参考已有的一些通用标准，重新设计了一套操作系统内核，不仅可以实现多用户、多任务的管理，还可以兼容 unix 原有的应用程序。最重要的是，他把这套尚不成熟的操作系统分享到了互联网上并用自己的名字命名了这套系统，也就是 **Linux**。



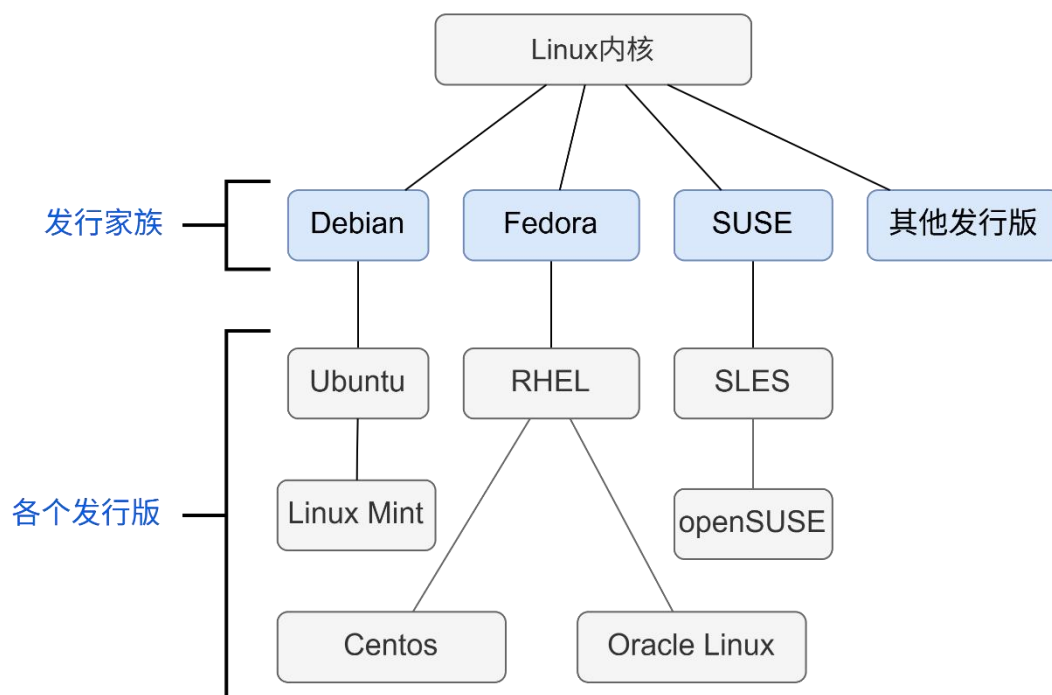
原本出于个人爱好的 Linux 系统通过互联网快速传播，更多爱好者看到 Linux 之后，也激发了乐于分享的热情，就把使用过程中的问题和修复方法做了反馈。

一石激起千层浪，越来越多人加入到了 Linux 维护的行列当中，一个原本功能有限、bug 很多的操作系统，快速强大起来，伴随其中发扬光大的是开源精神。

与 Windows 系统收费，或 MacOS 硬件绑定的模式不同，**Linux 是一套免费并且开放源代码的操作系统**，任何人都可以使用或者提交反馈，这就吸引了大量的开发者、爱好者，甚至很多企业，现在，每年对 Linux 系统提交的代码量已经成为衡量一个大公司技术实力的重要指标之一。

Linux 发展迅猛，已经成为了性能稳定的多用户操作系统，也是 ROS2 依赖的重要底层系统。虽然 ROS2 目前也支持 Windows、MacOS，但对 Linux 系统的支持最好。

所以在使用 ROS2 之前，我们需要先安装 Linux，此时会出现另外一个概念——**发行版**。



什么叫发行版呢？准确来讲，我们提到的 **Linux** 应该叫做**操作系统内核**，并没有可视化界面，发行版就是给这个内核加上华丽的外衣，把操作界面和各种应用软件放到一起，打包成我们安装系统的镜像。



所以一般情况下，我们常用到的 Linux 系统，都是各种各样的发行版，比如 Ubuntu、Fedora、RedHat 等等，每一个发行版都有其适用的场景，比如 RedHat 适合商业应用、CentOS 适合服务器、Ubuntu、Fedora 适合个人使用等，每一个版本的界面不太一样，但核心都是 Linux，操作方法基本相同。

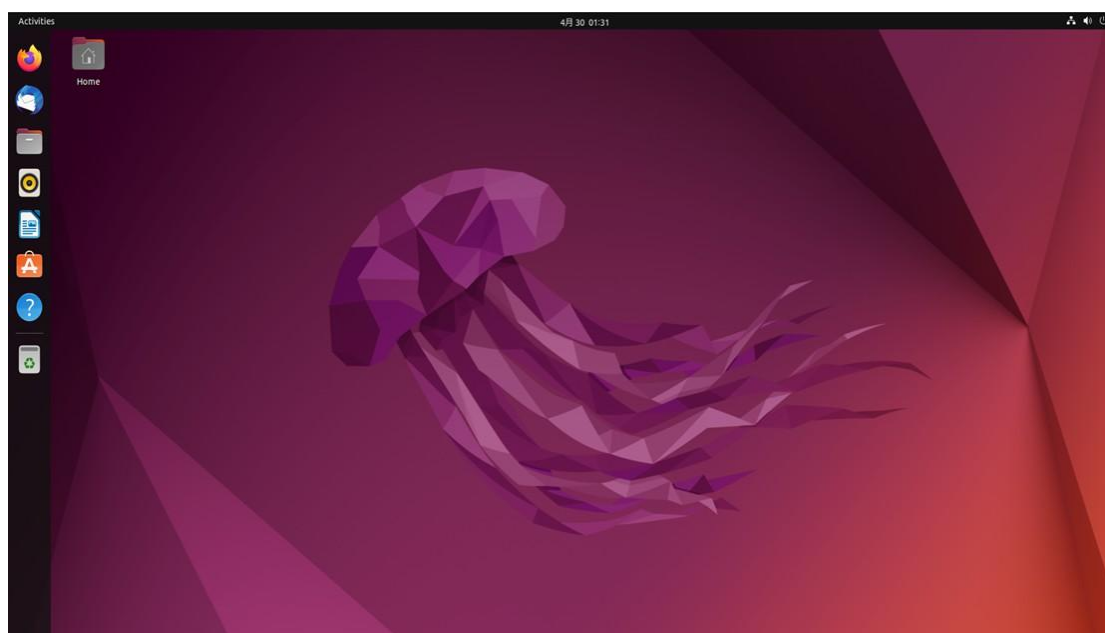
Ubuntu 系统简介

Ubuntu 诞生于 2004 年 10 月份，每六个月发布一个新版本，用户可以一直免费升级使用。在软件开发领域，无论是互联网开发，还是人工智能开发，还是机器人开发，Ubuntu 都占据绝对重要的位置。

Ubuntu 的版本变动比较快，版本不同会影响上层应用的移植效果。版本的

编号如 Ubuntu22.04。22 代表 2022 年，04 表示 2022 年的 4 月份发布，除了 04 还可能会出现的是 10，就是 10 月份发布，所以从数字编号上就可以看出各个版本发布的顺序。

为了让更多开发者有一个稳定的系统环境，Ubuntu 每隔两年的 4 月份会发布一个长期支持版，后缀会加 LTS，保证 5 年之内持续维护更新，比如 Ubuntu20.04 LTS，Ubuntu22.04 LTS，除此之外的版本都是普通版，只维护 18 个月。



目前最新的长期支持版是 Ubuntu24.04 LTS，但是当前不论是 ROS2 还是 Gazebo，在 Ubuntu22.04 LTS 都会比 Ubuntu24.04 LTS 稳定一些。

二、 虚拟机和 U 盘双系统使用

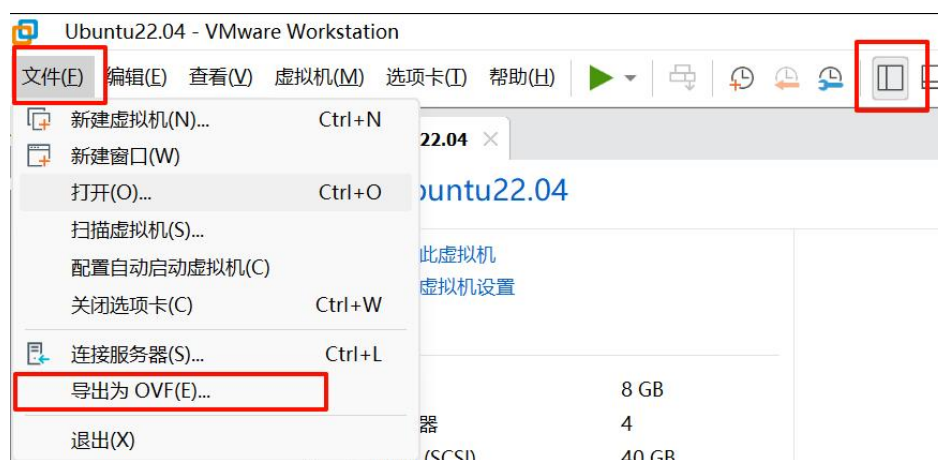
VMware 虚拟机使用

虚拟机是一个软件，可以在已有系统之上，构建另外一个虚拟的系统，让多个操作环境同时运行。我们大多数电脑的环境都是 windows，如果用双系统或者硬盘安装 linux 的维护成本比较大。虚拟机会更方便，但是性能不能很好的释放。

	虚拟机安装	硬盘安装
安装难易	简单	复杂
硬件支持	一般	好
运行速度	慢	快
安全备份	简单	复杂
适合人群	初次接触或偶尔使用者	有一定经验的开发者

Ubuntu 虚拟机安装可参考：[Ubuntu 虚拟机安装](#)。也可以导入已经配置好的虚拟机，直接使用。

安装好之后通常会导出配置好的虚拟机，方便之后快速创建虚拟机和恢复数据。可以导出为 ova/ovf 格式——一种开放标准的虚拟机打包格式，用于描述和分发虚拟机，可以迁移至其他平台，但是文件体积会比较大。导入时直接双击即可。



导出 OVF

如果没有迁移平台的需求，也可以直接备份必要的虚拟机文件。压缩后文件体积会小一些。双击 .vmx 文件导入。

名称	压缩后大小	原始大小	类型
Ubuntu22.04.nvram	1,694	8,684	VMware 虚拟机非易变 RAM
Ubuntu22.04.vmdk	4,492,852,957	12,601,655,296	VMware 虚拟磁盘文件
Ubuntu22.04.vmsd	0	0	VMware 快照元数据
Ubuntu22.04.vmx	1,118	3,070	VMware 虚拟机配置
Ubuntu22.04.vmxs	199	266	VMware 组成员

VMware 必要的虚拟机文件

虚拟机网络设置

虚拟机网卡类型有三种，NAT、桥接、仅主机；默认一般是 NAT,如果需要对虚拟机内部的 ros 程序可以和外部通信需要设置网卡为桥接模式。

可以在编辑>虚拟网络编辑器>更改设置，选择桥接模式的虚拟网卡（一般是 VMnet0），选择对应的网卡，比如有线网卡。（默认自动可能桥接失败，尤其是设备有多个网卡时）

名称	类型	外部连接	主机连接	DHCP	子网地址
VMnet0	桥接模式	Realtek Gaming 2.5GbE Famil...	-	-	-
VMnet1	仅主机...	-	已连接	已启用	192.168.80.0
VMnet2	桥接模式	Intel(R) Wi-Fi 6E AX210 160...	-	-	-
VMnet8	NAT	NAT	已连接	已启用	192.168.190.0

添加网络(E)... 移除网络(O) 重命名网络(W)...

VMnet 信息

☒ 桥接模式(将虚拟机直接连接到外部网络)(B)

已桥接至(G): Intel(R) Wi-Fi 6E AX210 160MHz 自动设置(U)...

☐ NAT 模式(与虚拟机共享主机的 IP 地址)(N) NAT 设置(S)...

☐ 仅主机模式(在专用网络内连接虚拟机)(H)

☐ 将主机虚拟适配器连接到此网络(V)

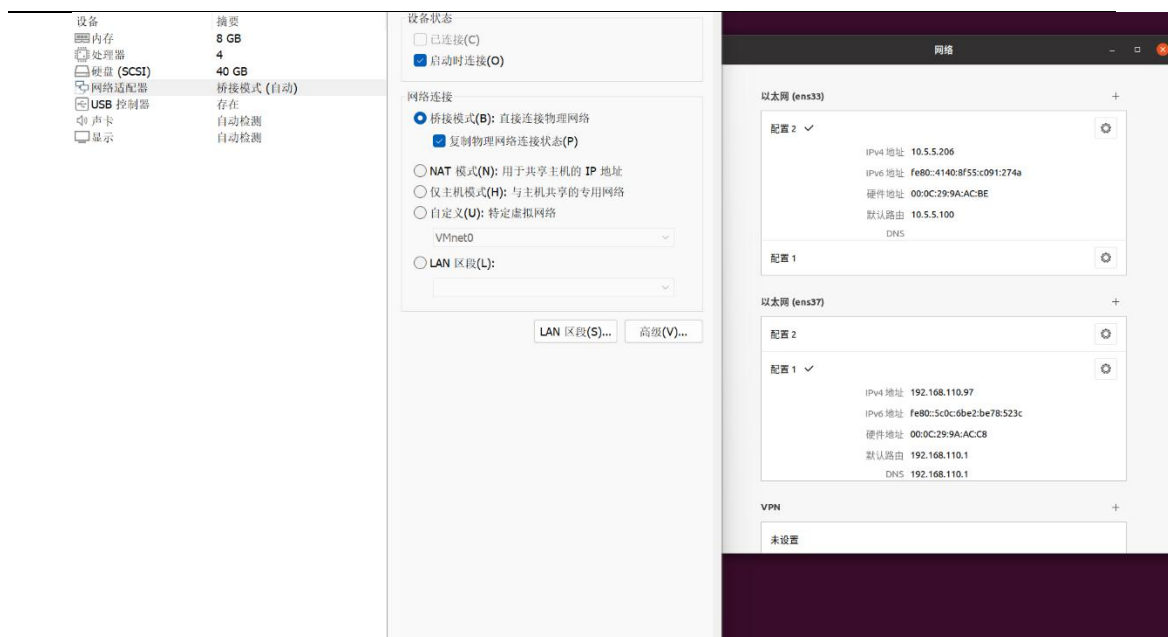
主机虚拟适配器名称: VMware 网络适配器 VMnet2

☐ 使用本地 DHCP 服务将 IP 地址分配给虚拟机(D) DHCP 设置(P)...

子网 IP (I): . . . 子网掩码(M): . . .

还原默认设置(R) 导入(T)... 导出(X)... 确定 取消 应用(A) 帮助

之后在虚拟机设置>网络适配器>连接方式选择桥接模式。在虚拟机内部设置静态 IP，和电脑在同一网段即可。



三种模式的区别

桥接模式 在这种模式下，VMWare 虚拟出来的操作系统就像是局域网中的一台独立的主机，它可以访问网内任何一台机器。在桥接模式下，你需要手工为虚拟系统配置 IP 地址、子网掩码，而且还要和宿主机处于同一网段，这样虚拟系统才能和宿主机进行通信。同时，由于这个虚拟系统是局域网中的一个独立的主机系统，那么就可以手工配置它的 TCP/IP 配置信息，以实现通过局域网的网关或路由器访问互联网。

主机模式 在 host-only 模式中，所有的虚拟系统是可以相互通信的，但虚拟系统和真实的网络是被隔离开的。

NAT 使用 NAT 模式，就是让虚拟系统借助 NAT(网络地址转换)功能，通过宿主机所在的网络来访问公网。也就是说，使用 NAT 模式可以实现在虚拟系统里访问互联网。NAT 模式下的虚拟系统的 TCP/IP 配置信息是由 VMnet8(NAT)虚拟网络的 DHCP 服务器提供的，无法进行手工修改，因此虚拟系统也就无法和本局域网中的其他真实主机进行通讯。采用 NAT 模式最大的优势是虚拟系统接入互联网非常简单，你不需要进行任何其他的配置，只需要宿主机能访问互联网即可。

U 盘双系统使用

U 盘双系统则通过启动盘在物理机上独立运行 Ubuntu，直接使用物理机硬件，避免虚拟机性能损耗，适合对算力要求较高的场景。

进入 U 盘系统

1. 进入 BIOS

插入系统 U 盘，然后重启电脑。在开机过程中，根据屏幕提示按下相应的按键（通常是 Del、F2、Esc 或 F10 等）进入 BIOS 设置界面。不同品牌和型号的主板进入 BIOS 的按键可能不同，常见的按键包括：

Del：大多数主板使用此键。

F2：部分华硕（ASUS）和技嘉（Gigabyte）主板使用此键。

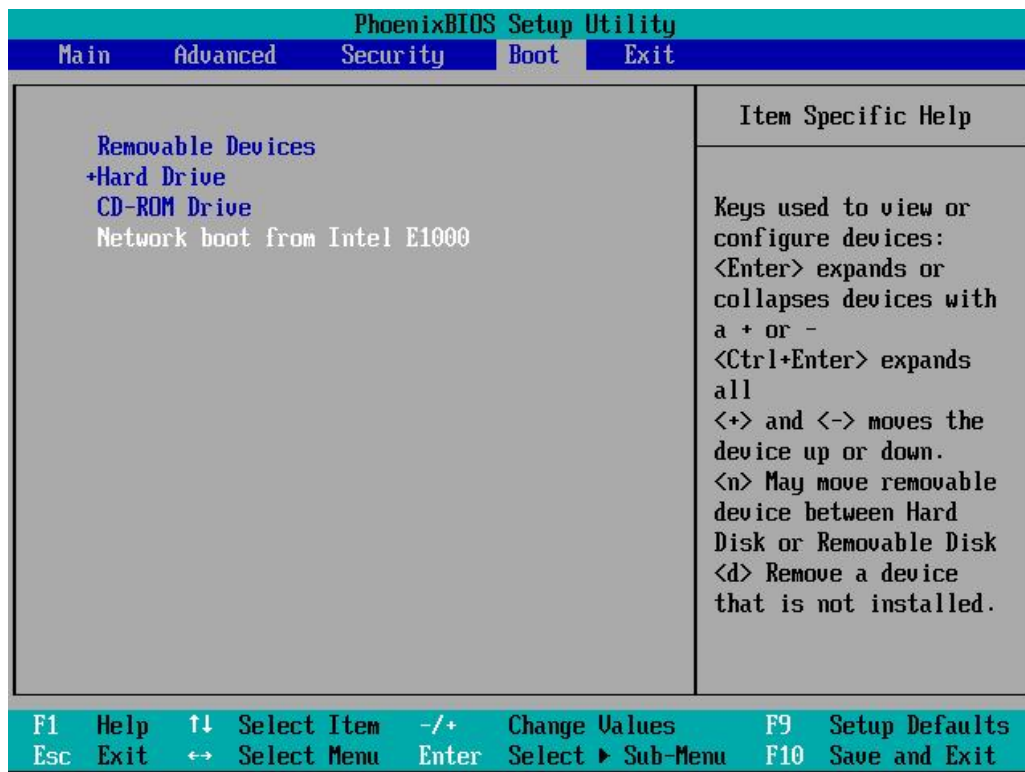
Esc：部分惠普（HP）和联想（Lenovo）笔记本使用此键。

F10：部分戴尔（Dell）和惠普（HP）笔记本使用此键。

2. 设置启动项

找到启动设置：进入 BIOS 后，找到“启动顺序”（Boot Order）或“启动设备”（Boot Device）选项。设置 U 盘为第一启动项：

在启动顺序列表中，找到 U 盘设备（通常显示为“USB Drive”或类似名称）。将 U 盘设置为第一启动项。通常可以通过上下箭头键选择设备，然后按“+”或“-”键调整顺序，或者直接选择“设置为第一启动项”选项。



3. 启用 USB 启动功能

检查 USB 启动功能：在 BIOS 设置中，找到“安全启动”（Secure Boot）或

“启动选项” (Boot Options)。

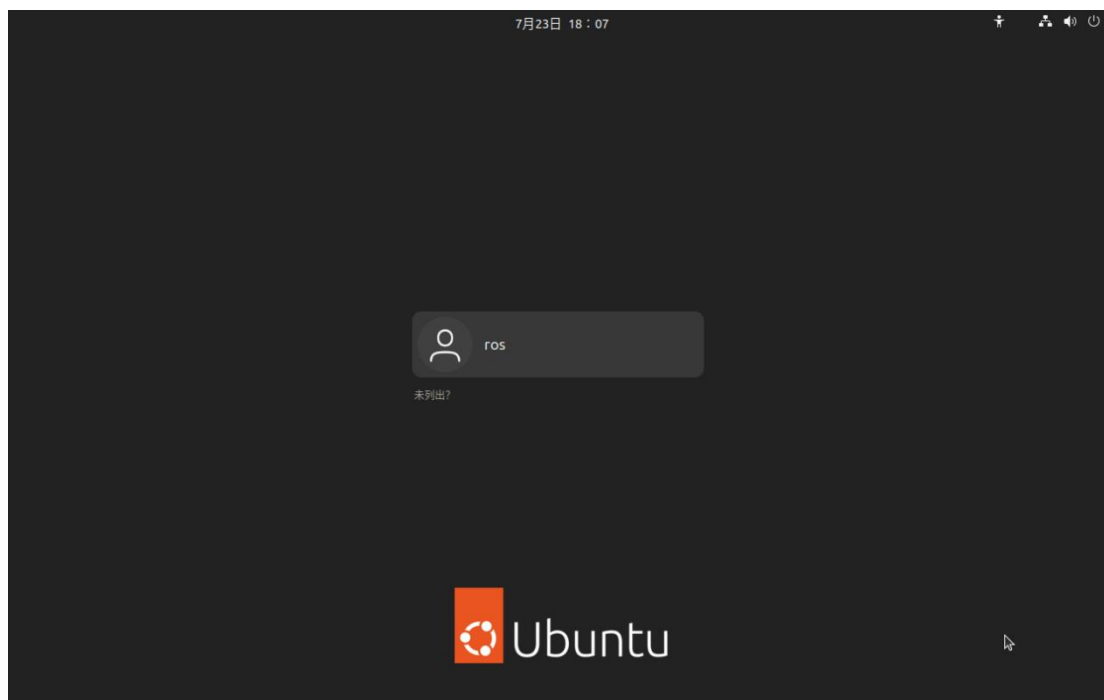
如果“安全启动”已启用，建议先禁用它，因为某些 U 盘启动盘可能不支持安全启动。

确保“USB 启动”或“从 USB 设备启动”选项已启用。如果未启用，即使 U 盘插入，也可能无法从 U 盘启动。

4. 保存设置并重启

保存设置：完成上述设置后，按“F10”键（或 BIOS 界面中提示的保存键）保存设置并退出 BIOS。

重启电脑：电脑将自动重启，并尝试从 U 盘启动。如果 U 盘启动盘制作正确，系统将从 U 盘加载启动界面。因 U 盘读写速度限制，加载过程大概黑屏 2~3 分钟。



三、 ROS2-机器人操作系统

简介

ROS——一句话概括是用于构建机器人应用程序的软件开发工具包。ROS 是开源的，它由一组软件库和若干工具组成。

整个 ROS 生态可以包括通信(Plumbing)、工具(Tools)、功能(Capabilities)与社区 (Community) 四大部分组成。



通信→ 通信是整个 ROS 系统的核心实现,是 ROS 内置的一个消息传递系统。在构建新的应用或使用与硬件交互的软件时,如何实现通信是首先要解决的问题,为此 ROS 提供了专门的消息传递系统,它可以管理不同节点之间的通信细节,提高开发者的工作效率。这套消息传递系统使用了“面向接口”的编程思想,可以通过清晰规范的接口,将不同模块分离,从而也将故障隔离。使得 ROS 系统更易于维护、扩展性更强且保证了程序的高复用性。接口规范可以由开发者自行定义。同时为了方便使用,在 ROS 中也提供了许多标准的接口,这些标准接口有着广泛应用,比如:将雷达或摄像头数据传输到可视化的用户界面或是传输到定位算法模块,都会使用到标准接口。也提供了多种通信模型,比如发布订阅、请求响应、请求反馈,这些后面也会带大家详细了解。

工具→ 构建机器人应用是比较有挑战性的。开发者除了会遇到一些传统的软件开发问题之外,还需要通过传感器或执行器与物理世界进行异步交互。显而易见的,良好的开发工具可以提高应用程序的开发效率,在 ROS 中就内置了节点启动工具--launch、终端调试工具、可视化程序--RQT,Rviz、录制回放--rosviz等一系列工具。这些工具不光可以提高开发效率,还可以在发布产品时直接包含在产品之中。

功能集→ ROS 生态系统是机器人软件的聚宝盆。一个开源的应用商店,无论开发者需要用于 GPS 的设备驱动程序、用于机器人控制器,还是用于移动机器人的地图系统,ROS 都能满足你的需求。从驱动程序到算法,再到用户界面,ROS 都提供了相关实现,开发者只需专注于自身应用程序即可。

ROS 的目标是提供一站式的技术支持,降低构建机器人应用程序的门槛。ROS 希望任何开发者都可以将自己的“创意”变为现实,而无需了解底层软件和硬件的所有内容。

还有一个就是 ROS 社区

ROS 社区规模庞大、多样且全球化,从学生和业余爱好者到跨国公司和政府机构,各行各业的人和组织都在推动着 ROS 项目的发展。

共同开发和维护大部分 ROS 核心软件，有丰富使用文档和社区环境。

更多资料可以查看：

ROS 官网：<https://docs.ros.org/>

ROS2 Documentation: <https://docs.ros.org/en/humble/>

ROS 社区：

<https://robotics.stackexchange.com/>

<https://discourse.openrobotics.org/>

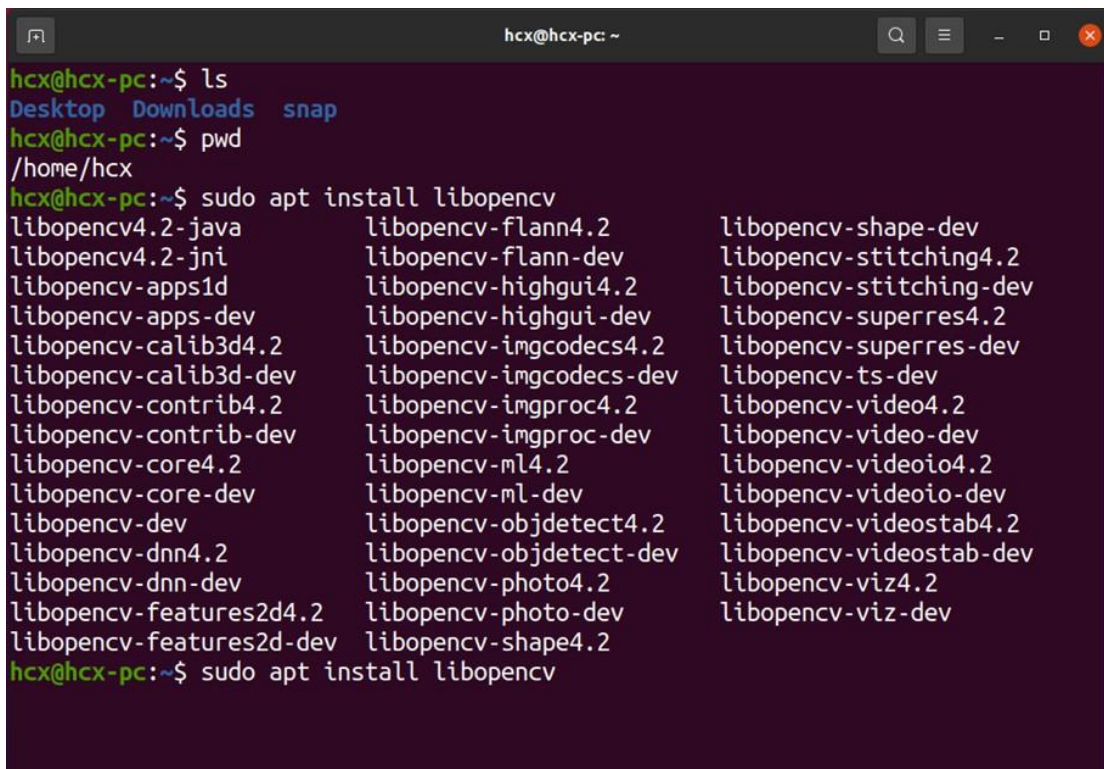
<https://www.guyuehome.com/>

ROS2 基础使用

Linux 中的命令行

命令都是通过字符的方式输入的，怎么输入呢，并不是使用写字本，而是使用专门的软件，叫做 **Terminal**，**终端**。启动终端的方式有很多种：

- 在应用列表中打开
- 快捷键 Ctrl+Alt+T 打开
- 鼠标右键选择打开终端



```
hcx@hcx-pc: ~  
hcx@hcx-pc:~$ ls  
Desktop Downloads snap  
hcx@hcx-pc:~$ pwd  
/home/hcx  
hcx@hcx-pc:~$ sudo apt install libopencv  
libopencv4.2-java      libopencv-flann4.2      libopencv-shape-dev  
libopencv4.2-jni      libopencv-flann-dev     libopencv-stitching4.2  
libopencv-apps1d      libopencv-highgui4.2    libopencv-stitching-dev  
libopencv-apps-dev     libopencv-highgui-dev   libopencv-superres4.2  
libopencv-calib3d4.2   libopencv-imgcodecs4.2  libopencv-superres-dev  
libopencv-calib3d-dev  libopencv-imgcodecs-dev libopencv-ts-dev  
libopencv-contrib4.2  libopencv-imgproc4.2    libopencv-video4.2  
libopencv-contrib-dev  libopencv-imgproc-dev   libopencv-video-dev  
libopencv-core4.2     libopencv-ml4.2         libopencv-videoio4.2  
libopencv-core-dev     libopencv-ml-dev        libopencv-videoio-dev  
libopencv-dev          libopencv-objdetect4.2  libopencv-videostab4.2  
libopencv-dnn4.2      libopencv-objdetect-dev libopencv-videostab-dev  
libopencv-dnn-dev      libopencv-photo4.2      libopencv-viz4.2  
libopencv-features2d4.2 libopencv-photo-dev     libopencv-viz-dev  
libopencv-features2d-dev libopencv-shape4.2  
hcx@hcx-pc:~$ sudo apt install libopencv
```

打开后这个深色背景的窗口就是终端，因为都是命令的输入，所以我们很少

会用到鼠标，这也是为什么科幻电影中看到的黑客，笔记本电脑随身带，但是从来看不到鼠标的原因。

初次上手，大家一定会觉得命令行既枯燥，又难以记忆，这很正常，随着我们对这项工具的熟悉，大家一定可以慢慢体会到命令行操作的魅力所在。

至于命令行指令及功能参数的数量，确实多到令人发指，不过不用死机硬背，我们常用的命令也就一二十个，其他命令在需要用的时候搜索一下即可。

常用命令操作

我们先来体验一下 Linux 的常用命令，找找感觉。

cd

- 语法：cd <目录路径>
- 功能：改变工作目录。若没有指定“目录路径”，则回到用户的主目录

pwd

- 语法：pwd
- 功能：此命令显示出当前工作目录的绝对路径

A terminal window titled 'gyh@ubuntu: ~' with standard window controls. The terminal shows a sequence of commands and their outputs: 'pwd' returns '/home/gyh'; 'cd ..' changes the directory to '/home'; 'pwd' returns '/home'; 'cd /home/gyh/' changes the directory back to the user's home; 'pwd' returns '/home/gyh'; 'cd ..' changes the directory back to '/home'; 'pwd' returns '/home'; 'cd ~' changes the directory back to the user's home; the final prompt 'gyh@ubuntu:~\$' is shown with a cursor.

```
gyh@ubuntu:~$ pwd
/home/gyh
gyh@ubuntu:~$ cd ..
gyh@ubuntu:/home$ pwd
/home
gyh@ubuntu:/home$ cd /home/gyh/
gyh@ubuntu:~$ pwd
/home/gyh
gyh@ubuntu:~$ cd ..
gyh@ubuntu:/home$ pwd
/home
gyh@ubuntu:/home$ cd ~
gyh@ubuntu:~$
```

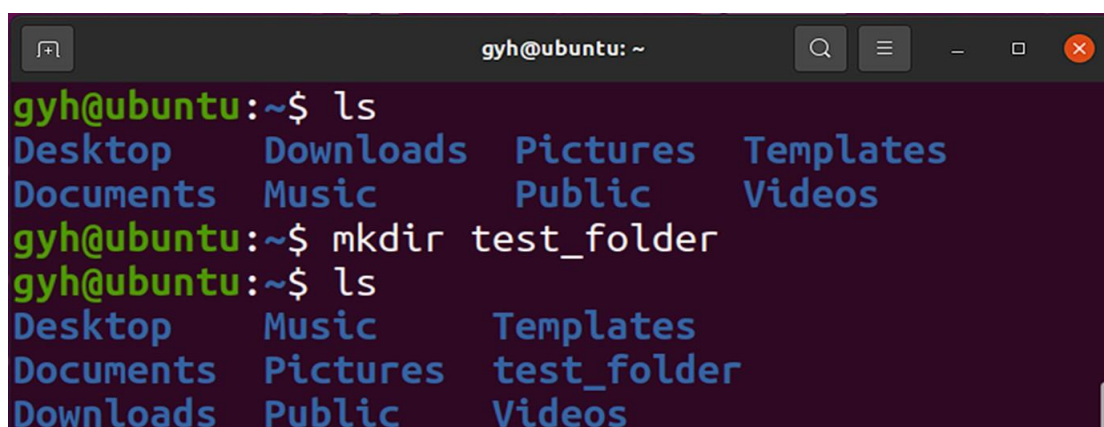
mkdir

- 语法：mkdir [选项] <目录名称>
- 功能：创建一个目录/文件夹

ls

- 语法：ls [选项] [目录名称...]

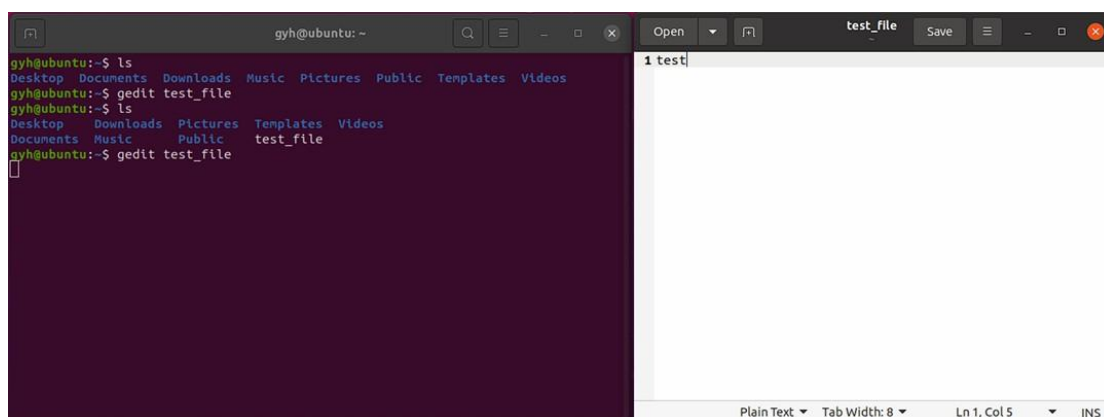
- 功能：列出目录/文件夹中的文件列表



```
gyh@ubuntu: ~  
gyh@ubuntu:~$ ls  
Desktop    Downloads  Pictures   Templates  
Documents  Music      Public     Videos  
gyh@ubuntu:~$ mkdir test_folder  
gyh@ubuntu:~$ ls  
Desktop    Music      Templates  
Documents  Pictures   test_folder  
Downloads  Public     Videos
```

gedit

- 语法：gedit <文件名称>
- 功能：打开 gedit 编辑器编辑文件，若没有此文件则会新建



```
gyh@ubuntu:~$ ls  
Desktop Documents Downloads Music Pictures Public Templates Videos  
gyh@ubuntu:~$ gedit test_file  
gyh@ubuntu:~$ ls  
Desktop Downloads Pictures Templates Videos  
Documents Music Public test_file  
gyh@ubuntu:~$ gedit test_file
```

The gedit editor window shows the file 'test_file' with the content '1 test'.

mv

- 语法：mv [选项] <源文件或目录> <目的地文件或目录>
- 功能：为文件或目录改名或将文件由一个目录移入另一个目录中

cp

- 语法：cp [选项] <源文件名称或目录名称> <目的文件名称或目录名称>
- 功能：把一个文件或目录拷贝到另一文件或目录中，或者把多个源文件复制到目标目录中

```
gyh@ubuntu: ~  
gyh@ubuntu:~$ ls  
Desktop  Downloads  Pictures  Templates  test_folder  
Documents Music      Public    test_file  Videos  
gyh@ubuntu:~$ mv test_file test_folder/  
gyh@ubuntu:~$ ls  
Desktop  Downloads  Pictures  Templates  Videos  
Documents Music      Public    test_folder  
gyh@ubuntu:~$ cd test_folder/  
gyh@ubuntu:~/test_folder$ ls  
test_file  
gyh@ubuntu:~/test_folder$ cp test_file /home/gyh/  
gyh@ubuntu:~/test_folder$ cd ..  
gyh@ubuntu:~$ ls  
Desktop  Downloads  Pictures  Templates  test_folder  
Documents Music      Public    test_file  Videos
```

rm

- 语法: rm [选项] <文件名称或目录名称...>
- 功能: 该命令的功能为删除一个目录中的一个或多个文件或目录, 它也可以将某个目录及其下的所有文件及子目录均删除。对于链接文件, 只是删除了链接, 原有文件均保持不变

```
gyh@ubuntu: ~  
gyh@ubuntu:~$ ls  
Desktop  Downloads  Pictures  Templates  test_folder  
Documents Music      Public    test_file  Videos  
gyh@ubuntu:~$ rm test_file  
gyh@ubuntu:~$ ls  
Desktop  Downloads  Pictures  Templates  Videos  
Documents Music      Public    test_folder  
gyh@ubuntu:~$ rm test_folder/  
rm: cannot remove 'test_folder/': Is a directory  
gyh@ubuntu:~$ rm -r test_folder/  
gyh@ubuntu:~$ ls  
Desktop Documents Downloads Music Pictures Public Templates Videos
```

sudo

- 语法: sudo [选项] [指令]
- 功能: 以系统管理员权限来执行指令

```
gyh@ubuntu: ~  
gyh@ubuntu:~$ sudo apt update  
[sudo] password for gyh:  
Hit:1 http://security.ubuntu.com/ubuntu focal-security InRelease  
Hit:2 http://us.archive.ubuntu.com/ubuntu focal InRelease  
Hit:3 http://us.archive.ubuntu.com/ubuntu focal-updates InRelease  
Hit:4 http://us.archive.ubuntu.com/ubuntu focal-backports InRelease  
Reading package lists... Done  
Building dependency tree  
Reading state information... Done  
312 packages can be upgraded. Run 'apt list --upgradable' to see them.  
gyh@ubuntu:~$ sudo apt install tree  
Reading package lists... Done  
Building dependency tree  
Reading state information... Done  
The following NEW packages will be installed:  
  tree  
0 upgraded, 1 newly installed, 0 to remove and 312 not upgraded.  
Need to get 43.0 kB of archives.  
After this operation, 115 kB of additional disk space will be used.  
Get:1 http://us.archive.ubuntu.com/ubuntu focal/universe amd64 tree amd64 1.8.0-1 [43.0 kB]  
Fetched 43.0 kB in 1s (41.1 kB/s)  
Selecting previously unselected package tree.  
(Reading database ... 183335 files and directories currently installed.)  
Preparing to unpack ../tree_1.8.0-1_amd64.deb ...  
Unpacking tree (1.8.0-1) ...  
Setting up tree (1.8.0-1) ...  
Processing triggers for man-db (2.9.1-1) ...
```

这些命令大家不需要死记硬背，未来一边用一边查，用的多了，就会熟悉。

Linux 命令导航：

<https://www.runoob.com/linux/linux-command-manual.html>

使用 turtlesim 、 ros2 和 rqt

Turtlesim 是一个轻量级的用于学习 ROS 2 的模拟器。它以最基本的方式展示了 ROS 2 的功能。

ros2 命令行工具是用户管理、检视和与 ROS 系统交互的方式。它支持多个命令，针对系统及其操作的不同方面。用户可以使用它来启动一个节点、设置参数、监听话题等等。

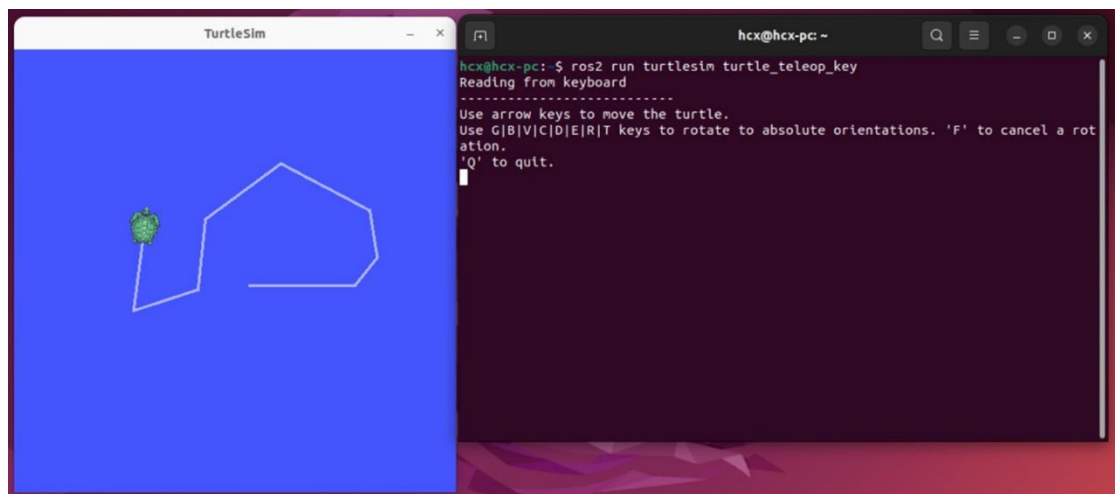
rqt 是一个用于 ROS 2 的图形用户界面（GUI）工具。rqt 中的所有操作都可以通过命令行完成，但是 rqt 提供了一种更加用户友好的方式来操作 ROS 2 元素。可以使用 `sudo apt install ros-humble-rqt*` 安装工具和插件。

运行节点程序

想要运行 ROS2 中某个节点，可以使用 `ros2 run` 命令进行操作，运行海龟仿真节点和键盘控制节点：

```
ros2 run turtlesim turtlesim_node
```

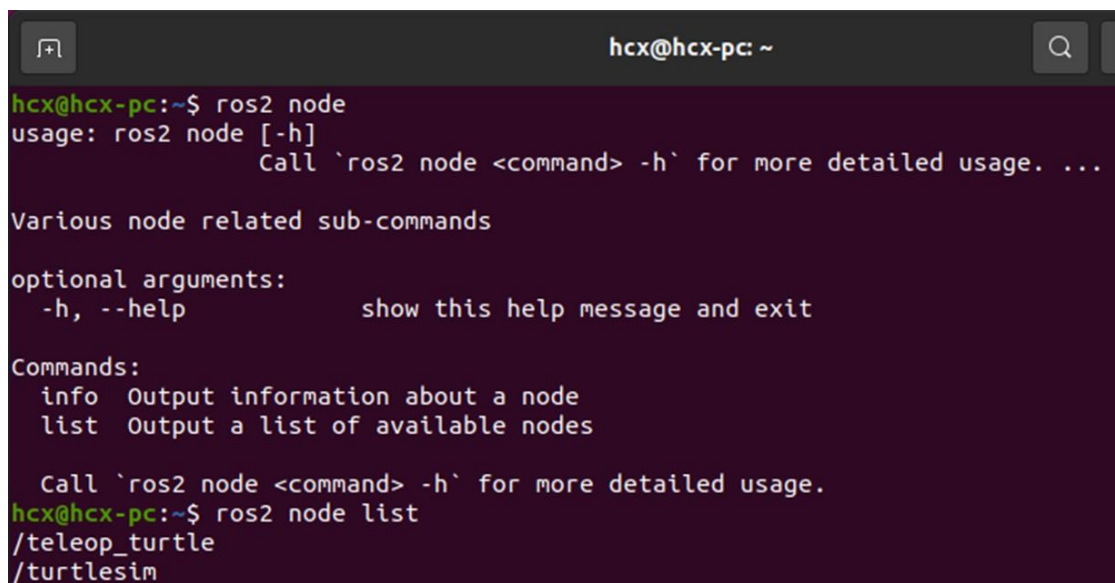
`ros2 run turtlesim turtle_teleop_key`



查看节点信息

当前运行的 ROS 系统中都有哪些节点呢？可以这样来查看：

`ros2 node list`



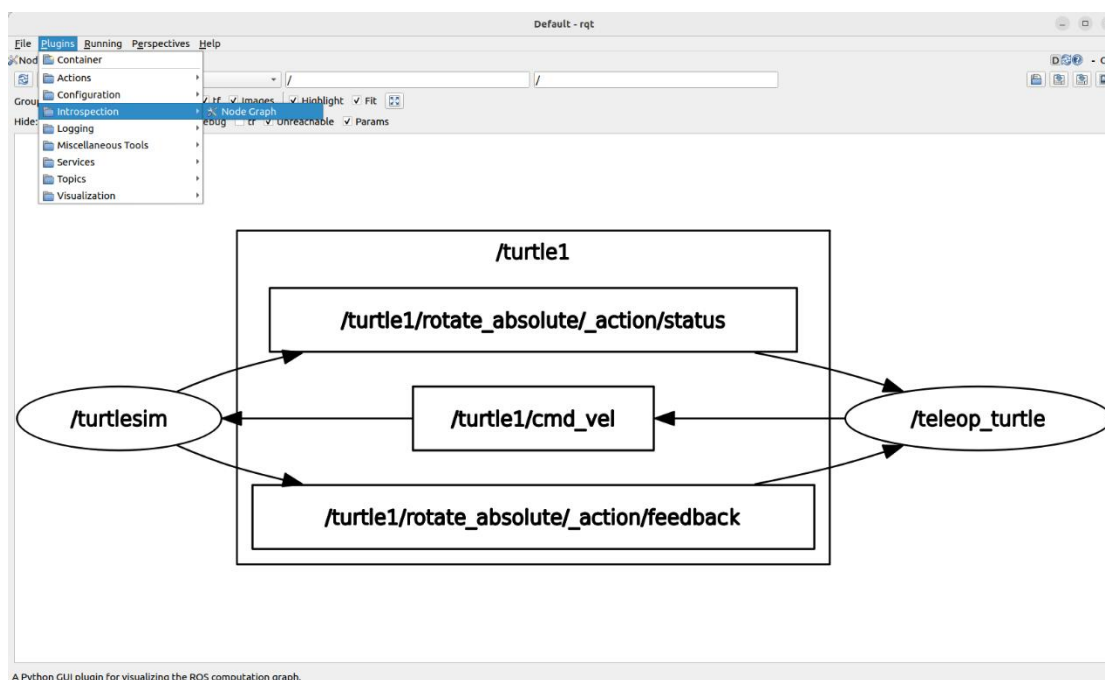
如果对某一个节点感兴趣，加上一个 info 子命令，就可以知道它的详细信息啦：

`ros2 node info /turtlesim`

```
hcx@hcx-pc ~
hcx@hcx-pc:~$ ros2 node info /turtlesim
/turtlesim
Subscribers:
  /parameter_events: rcl_interfaces/msg/ParameterEvent
  /turtle1/cmd_vel: geometry_msgs/msg/Twist
Publishers:
  /parameter_events: rcl_interfaces/msg/ParameterEvent
  /rosout: rcl_interfaces/msg/Log
  /turtle1/color_sensor: turtlesim/msg/Color
  /turtle1/pose: turtlesim/msg/Pose
Service Servers:
  /clear: std_srvs/srv/Empty
  /kill: turtlesim/srv/Kill
  /reset: std_srvs/srv/Empty
  /spawn: turtlesim/srv/Spawn
  /turtle1/set_pen: turtlesim/srv/SetPen
  /turtle1/teleport_absolute: turtlesim/srv/TeleportAbsolute
  /turtle1/teleport_relative: turtlesim/srv/TeleportRelative
  /turtlesim/describe_parameters: rcl_interfaces/srv/DescribeParameters
  /turtlesim/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
  /turtlesim/get_parameters: rcl_interfaces/srv/GetParameters
  /turtlesim/list_parameters: rcl_interfaces/srv/ListParameters
  /turtlesim/set_parameters: rcl_interfaces/srv/SetParameters
  /turtlesim/set_parameters_atomically: rcl_interfaces/srv/SetParametersAtomically
Service Clients:

Action Servers:
  /turtle1/rotate_absolute: turtlesim/action/RotateAbsolute
Action Clients:
```

终端输入 `rqt` 打开 graph 插件，可以看到已经启动的节点和存在通信的话题



该图形描述了 `/turtlesim` 节点和 `/teleop_turtle` 节点之间如何通过一个主题进行通信。`/teleop_turtle` 节点将数据（您输入的按键操作来移动乌龟）发布到 `/turtle1/cmd_vel` 主题上，而 `/turtlesim` 节点订阅该主题以接收数据。

查看话题信息

当前系统中都有话题呢，使用如下命令即可查看：

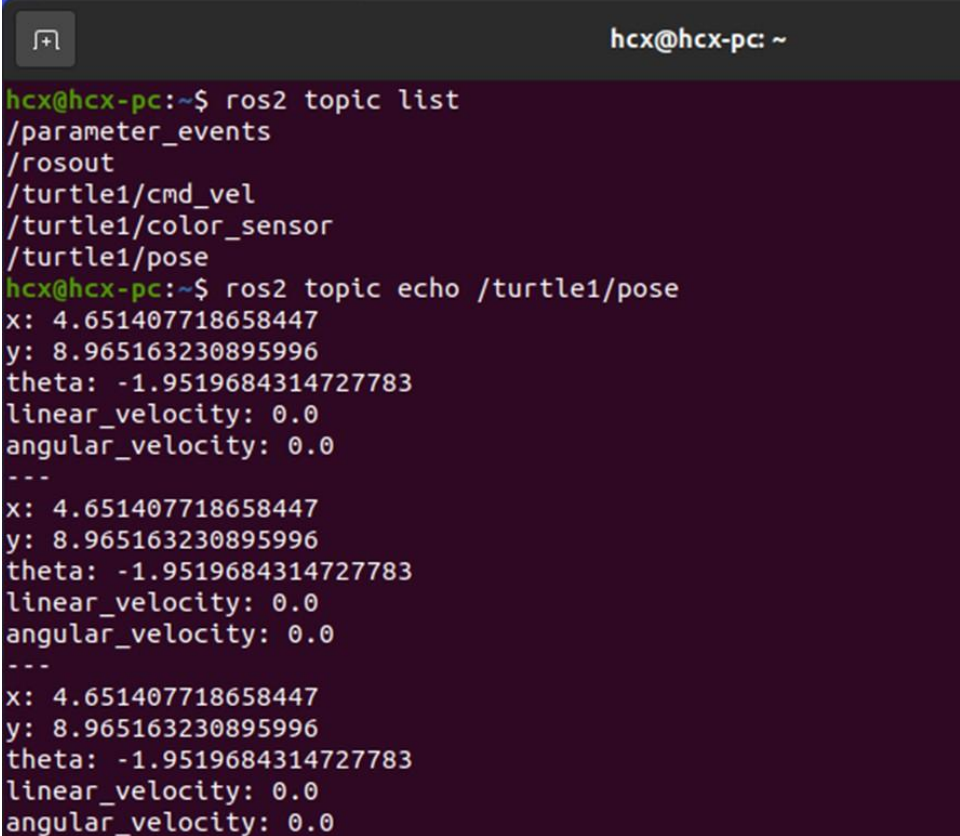
```
ros2 topic list
```


还想看到某一个话题中的消息数据，加上 echo 子命令试一试：

```
ros2 topic echo /turtle1/pose
```

Info 可以查看话题信息再加上-v 可以查看更多信息

```
ros2 topic info /turtle1/pose -v
```



```
hcx@hcx-pc: ~  
hcx@hcx-pc:~$ ros2 topic list  
/parameter_events  
/rosout  
/turtle1/cmd_vel  
/turtle1/color_sensor  
/turtle1/pose  
hcx@hcx-pc:~$ ros2 topic echo /turtle1/pose  
x: 4.651407718658447  
y: 8.965163230895996  
theta: -1.9519684314727783  
linear_velocity: 0.0  
angular_velocity: 0.0  
---  
x: 4.651407718658447  
y: 8.965163230895996  
theta: -1.9519684314727783  
linear_velocity: 0.0  
angular_velocity: 0.0  
---  
x: 4.651407718658447  
y: 8.965163230895996  
theta: -1.9519684314727783  
linear_velocity: 0.0  
angular_velocity: 0.0
```



```

hcx@hcx-pc: ~
^Chcx@hcx-pc:~$ ros2 topic info /turtle1/pose
Type: turtlesim/msg/Pose
Publisher count: 1
Subscription count: 0
hcx@hcx-pc:~$
hcx@hcx-pc:~$
hcx@hcx-pc:~$ ros2 topic hz /turtle1/pose
average rate: 62.430
      min: 0.014s max: 0.018s std dev: 0.00085s window: 64
average rate: 62.494
      min: 0.012s max: 0.019s std dev: 0.00095s window: 127
^Chcx@hcx-pc:~$ ros2 topic bw /turtle1/pose
Subscribed to [/turtle1/pose]
1.51 KB/s from 62 messages
      Message size mean: 0.02 KB min: 0.02 KB max: 0.02 KB
1.51 KB/s from 100 messages
      Message size mean: 0.02 KB min: 0.02 KB max: 0.02 KB
^Chcx@hcx-pc:~$ ros2 topic type /turtle1/pose
turtlesim/msg/Pose
  
```

发布话题消息

在运行 `ros2 topic list -t` 后，我们可以看到之前提到的主题类型，它们告诉我们每个主题使用的消息类型。`cmd_vel` 主题的类型是：

`geometry_msgs/msg/Twist`

这意味着在 `geometry_msgs` 包中有一个名为 `Twist` 的 `msg`。

现在我们可以在此类型上运行 `ros2 interface show <msg type>` 了解其详细信息。

```
ros2 interface show geometry_msgs/msg/Twist
```

会看到

This expresses velocity in free space broken into its linear and angular parts.

```

Vector3 linear
  float64 x
  float64 y
  float64 z
Vector3 angular
  float64 x
  float64 y
  float64 z
  
```

这告诉你 `/turtlesim` 节点期望接收一个带有两个向量 `linear` 和 `angular` 的消息，每个向量都有三个元素。我们使用 `echo` 命令看到

``/teleop\_turtle``传递给``/turtlesim``的数据，它具有相同的结构：

```
linear:
  x: 2.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0
---
```

现在可以直接从命令行发布数据到一个话题上，使用以下命令：

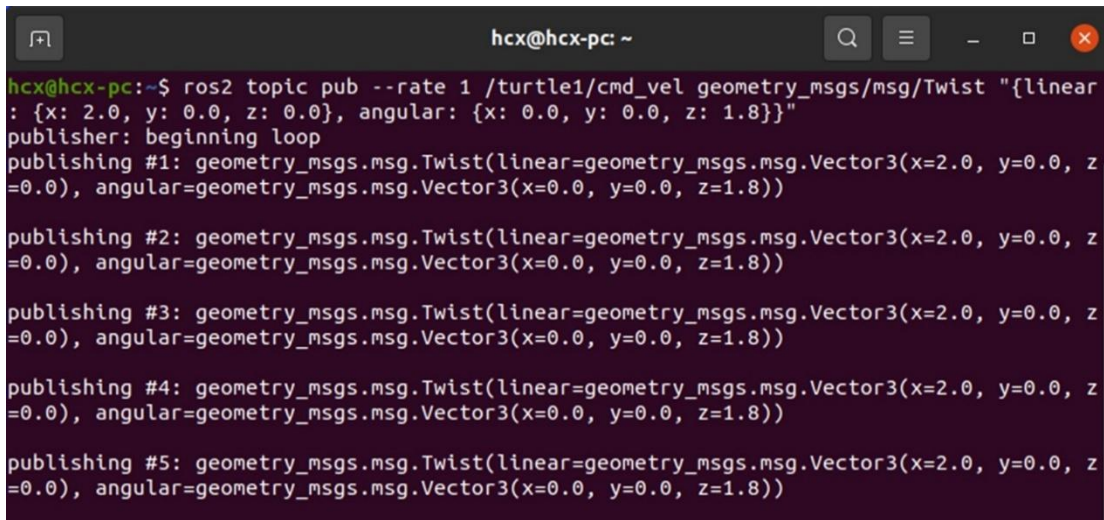
```
ros2 topic pub <topic_name> <msg_type> '<args>'
```

'<args>' 参数是您将传递给话题的实际数据，其结构正如您在前一节中了解到的。

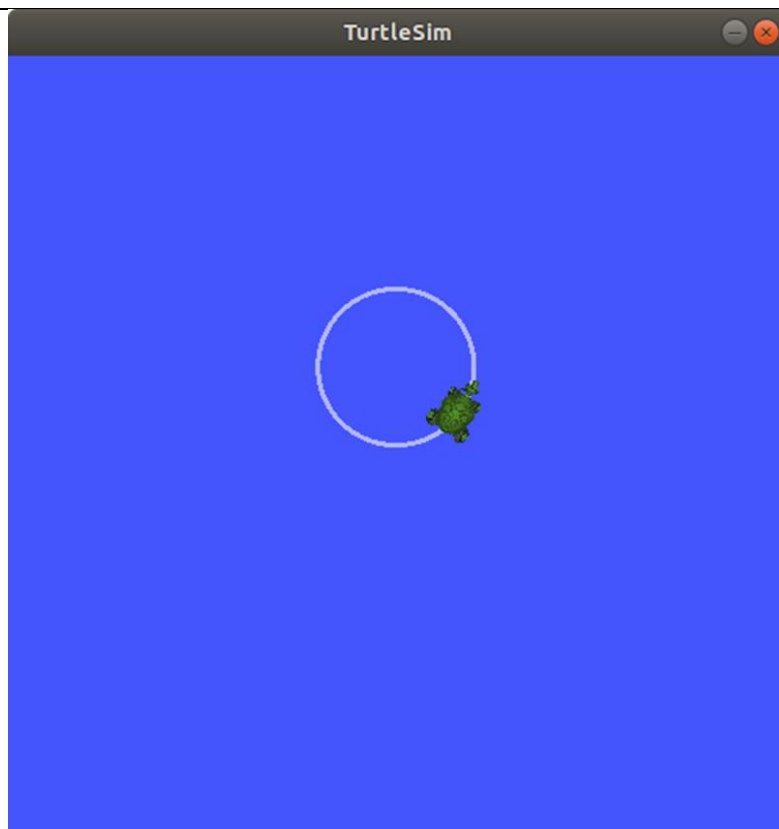
请注意，这个参数需要以 YAML 语法输入。想要控制海龟动起来完整的命令，如下所示：

```
ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}"
```

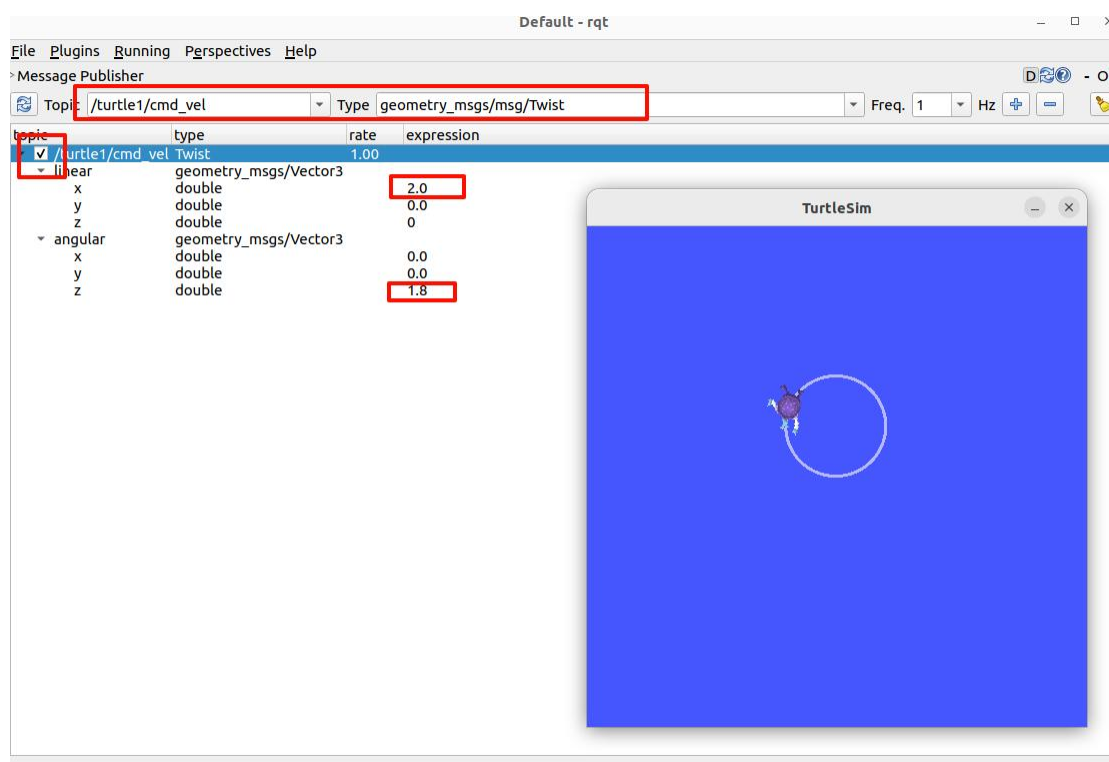
--once 是一个可选参数，表示“发布一条消息后退出”。



```
hcx@hcx-pc: ~
hcx@hcx-pc:~$ ros2 topic pub --rate 1 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}"
publisher: beginning loop
publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=2.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=1.8))
publishing #2: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=2.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=1.8))
publishing #3: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=2.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=1.8))
publishing #4: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=2.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=1.8))
publishing #5: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=2.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=1.8))
```



在 rqt 中打开 Message Publisher 插件发布话题消息

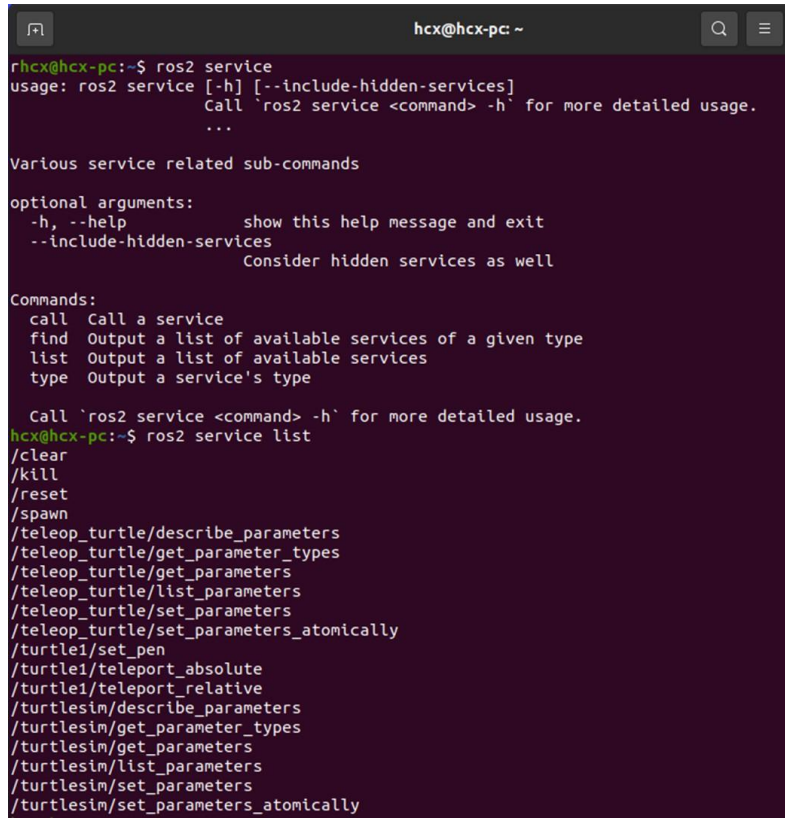


发送服务请求

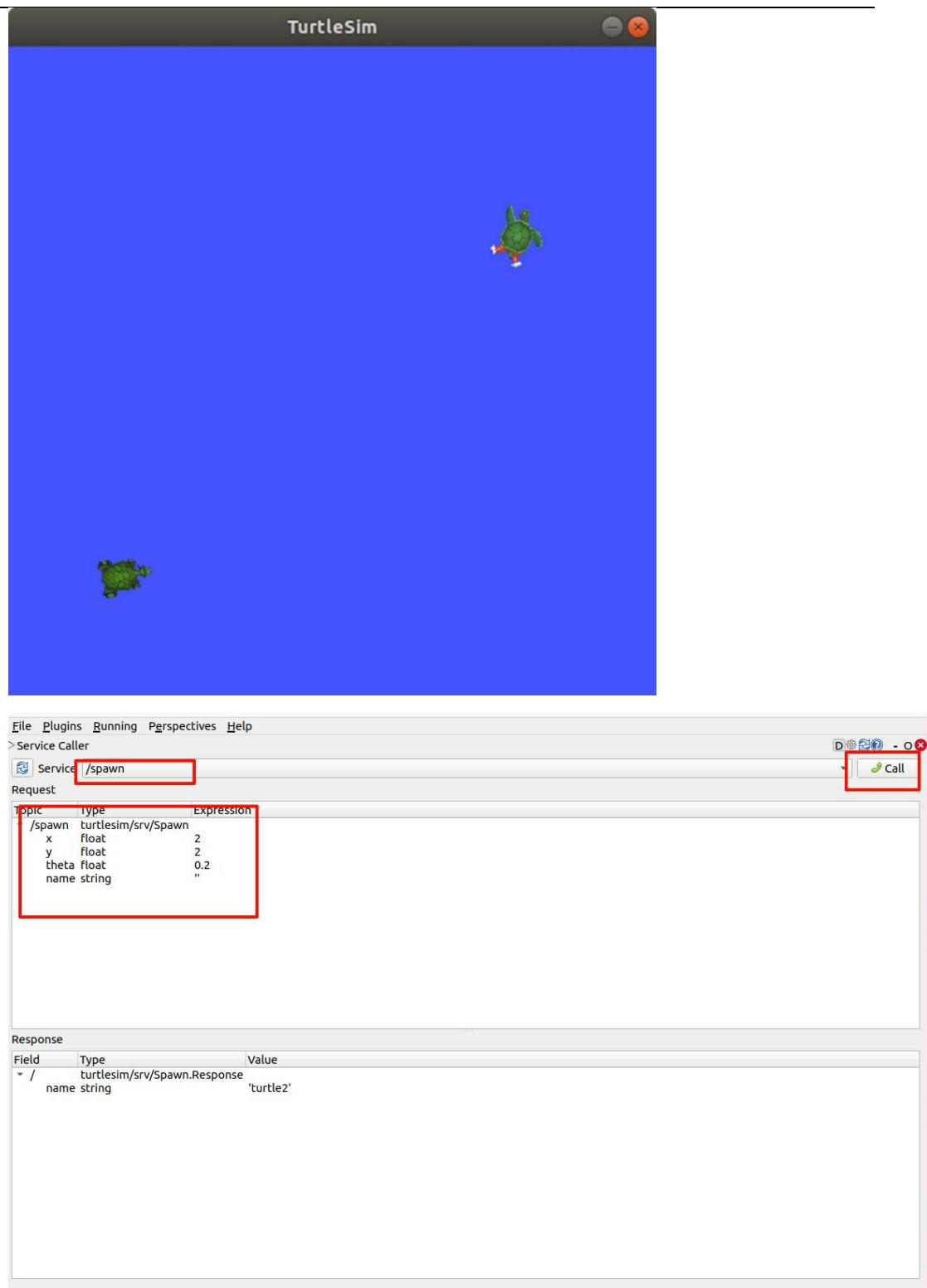
一只海龟太孤单，仿真器还提供改了一个服务——产生海龟，我们试一试服

务调用，再来一只海龟：

```
ros2 service call /spawn turtlesim/srv/Spawn "{x: 2, y: 2, theta: 0.2,  
name: ''}"
```



```
hcx@hcx-pc: ~  
rhc@hcx-pc:~$ ros2 service  
usage: ros2 service [-h] [--include-hidden-services]  
                Call 'ros2 service <command> -h' for more detailed usage.  
...  
  
Various service related sub-commands  
  
optional arguments:  
  -h, --help            show this help message and exit  
  --include-hidden-services  
                        Consider hidden services as well  
  
Commands:  
  call    Call a service  
  find    Output a list of available services of a given type  
  list    Output a list of available services  
  type    Output a service's type  
  
  Call 'ros2 service <command> -h' for more detailed usage.  
hcx@hcx-pc:~$ ros2 service list  
/clear  
/kill  
/reset  
/spawn  
/teleop_turtle/describe_parameters  
/teleop_turtle/get_parameter_types  
/teleop_turtle/get_parameters  
/teleop_turtle/list_parameters  
/teleop_turtle/set_parameters  
/teleop_turtle/set_parameters_atomically  
/turtle1/set_pen  
/turtle1/teleport_absolute  
/turtle1/teleport_relative  
/turtlesim/describe_parameters  
/turtlesim/get_parameter_types  
/turtlesim/get_parameters  
/turtlesim/list_parameters  
/turtlesim/set_parameters  
/turtlesim/set_parameters_atomically
```

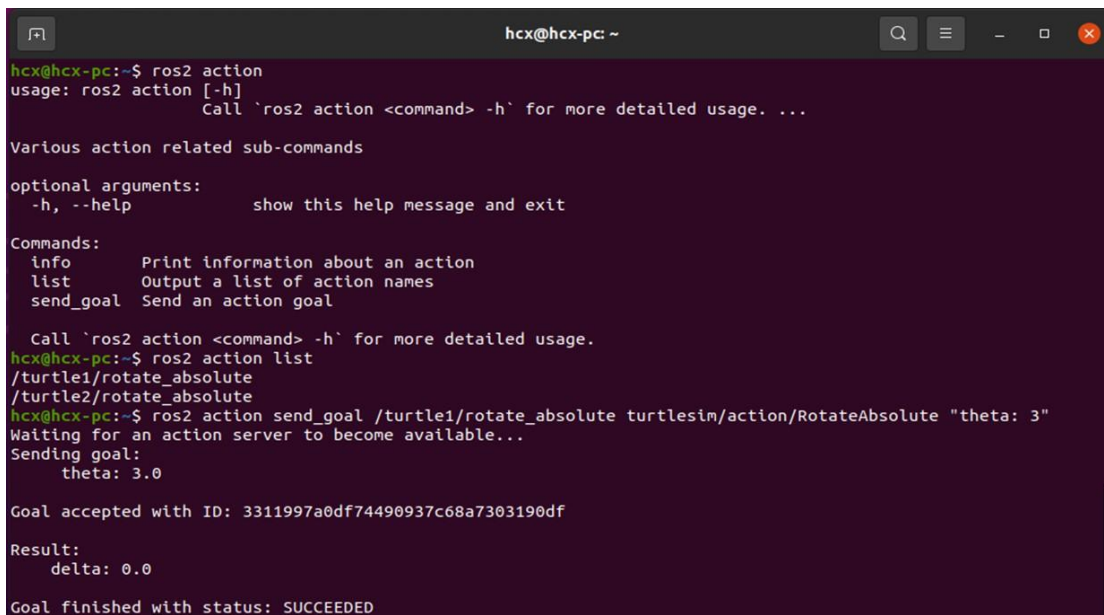


发送动作目标

想要让海龟完成一个具体动作，比如转到指定角度，仿真器中提供的这个 action 可以帮上忙，通过命令行这样发送动作目标：

```
ros2 action send_goal /turtle1/rotate_absolute
```

```
turtlesim/action/RotateAbsolute "theta: 3"
```



```
hcx@hcx-pc: ~  
hcx@hcx-pc:~$ ros2 action  
usage: ros2 action [-h]  
          Call 'ros2 action <command> -h' for more detailed usage. ...  
  
Various action related sub-commands  
  
optional arguments:  
  -h, --help            show this help message and exit  
  
Commands:  
  info                Print information about an action  
  list                Output a list of action names  
  send_goal           Send an action goal  
  
  Call 'ros2 action <command> -h' for more detailed usage.  
hcx@hcx-pc:~$ ros2 action list  
/turtle1/rotate_absolute  
/turtle2/rotate_absolute  
hcx@hcx-pc:~$ ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute "theta: 3"  
Waiting for an action server to become available...  
Sending goal:  
  theta: 3.0  
  
Goal accepted with ID: 3311997a0df74490937c68a7303190df  
  
Result:  
  delta: 0.0  
  
Goal finished with status: SUCCEEDED
```

以上就是 ROS2 中我们常用的命令啦，每一个命令的子命令还有很多，大家可以自己尝试看看。

初学者：CLI 工具：

<https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools.html>

编写一个 ROS2 程序

colcon 是 ROS 构建工具，catkin_make、catkin_tools 和 ament_tools 的迭代。有关 colcon 设计的更多信息，请参阅[此文档](#)。

安装 colcon 可以使用这行命令

```
sudo apt install python3-colcon-common-extensions
```

但是一般会安装 [ros-dev-tools](#)，一次性安装 colcon 和其他构建 ROS 包的工具

```
sudo apt install ros-dev-tools
```

这会安装 colcon、git、cmake 等 ROS 开发工具

- **工作空间** (Workspace) ，开发过程的大本营
- **功能包** (Package) ：功能源码的聚集地
- **节点** (Node) ：机器人的工作细胞
- **话题** (Topic) ：节点间传递数据的桥梁
- **服务** (Service) ：节点间的你问我答
- **通信接口** (Interface) ：数据传输的标准结构
- **参数** (Parameter) ：机器人系统的全局字典
- **动作** (Action) ：完整行为的流程管理
- **分布式通信** (Distributed Communication) ：多计算平台的任务分配
- **DDS** (Data Distribution Service) ：机器人的神经网络



ROS2 包 (Package) 的基本结构

功能包是 ROS2 应用程序的核心，但是功能包不能直接构建，必须依赖于工作空间，一个 ROS2 工作空间的目录结构如下：

```
Workspace --- 自定义的工作空间。
|--- build: 存储中间文件的目录，该目录下会为每一个功能包创建一个单独子目录。
|--- install: 安装目录，该目录下会为每一个功能包创建一个单独子目录。
|--- log: 日志目录，用于存储日志文件。
|--- src: 用于存储功能包源码的目录。
    |-- C++功能包
        |-- package.xml: 包信息，比如:包名、版本、作者、依赖项。
        |-- CMakeLists.txt: 配置编译规则，比如源文件、依赖项、目标文件。
        |-- src: C++源文件目录。
        |-- include: 头文件目录。
        |-- msg: 消息接口文件目录。
        |-- srv: 服务接口文件目录。
        |-- action: 动作接口文件目录。
    |-- Python 功能包
        |-- package.xml: 包信息，比如:包名、版本、作者、依赖项。
        |-- setup.py: 与 C++功能包的 CMakeLists.txt 类似。
        |-- setup.cfg: 功能包基本配置文件。
        |-- resource: 资源目录。
        |-- test: 存储测试相关文件。
        |-- 功能包同名目录: Python 源文件目录。
```

另外，无论是 Python 功能包还是 C++功能包，都可以自定义一些配置文件相关的目录。

|-- C++或 Python 功能包

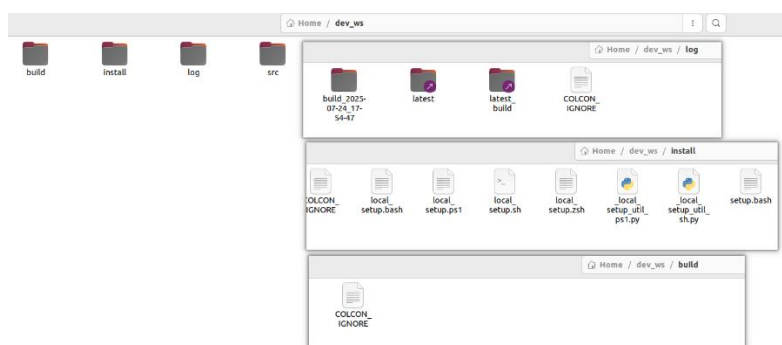
|-- launch: 存储 launch 文件。
 |-- rviz: 存储 rviz2 配置相关文件。
 |-- urdf: 存储机器人建模文件。
 |-- params: 存储参数文件。
 |-- world: 存储仿真环境相关文件。
 |-- map: 存储导航所需地图文件。
 |--

创建工作空间

了解了工作空间的概念和结果，接下来我们可以使用如下命令创建一个工作空间

```
# 一个空的工作空间必须存在 src 目录才能被编译
mkdir -p ~/demo_ws/src
cd ~/demo_ws/
# 使用 colcon 编译
colcon build
```

编译成功后，就可以在工作空间中看到自动生产的 build、log、install 文件夹了。



创建功能包

如何在 ROS2 中创建一个功能包呢？我们可以使用这个指令：

```
ros2 pkg create --build-type <build-type> <package_name> --node-name <node_name>
```

ros2 命令中：

- **pkg**: 表示功能包相关的功能；
- **create**: 表示创建功能包；
- **build-type**: 表示新创建的功能包是 C++ 还是 Python 的，如果使用 C++ 或者 C，那这里就跟 ament_cmake，如果使用 Python，就跟 ament_python；
- **package_name**: 新建功能包的名字，功能包名称再工作空间中必须是唯一

的。

- **node-name**: 可选参数, 在包中创建一个简单的 Hello World 类型的可执行文件。

比如在终端中分别创建 C++ 和 Python 版本的功能包:

```
cd ~/demo_ws/src
ros2 pkg create --build-type ament_cmake pkg_c --node-name hello_ros # C++
ros2 pkg create --build-type ament_python pkg_py --node-name hello_ros # Python
```

编译功能包

在创建好的功能包中, 我们可以继续完成代码的编写, 之后需要编译和配置环境变量, 才能正常运行:

```
cd ~/demo_ws
colcon build # 编译工作空间所有功能包
# 也可以编译指定功能包
colcon build --packages-select pkg_c pkg_py
```

功能包的结构

功能包并不是普通的文件夹, 那如何判断一个文件夹是否是功能包呢? 我们分析下刚才新创建两个功能包的结构。

C++功能包

首先看下 C++ 类型的功能包, 其中必然存在两个文件: **package.xml** 和 **CMakeLists.txt**。



include



src



CMakeLists
.txt



package.
xml

package.xml 文件的主要内容如下, 包含功能包的版权描述, 和各种依赖的声明。

```
1 <?xml version="1.0"?>
2 <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema"?>
3 <package format="3">
4   <name>pkg_c</name>
5   <version>0.0.0</version>
6   <description>TODO: Package description</description>
7   <maintainer email="email@email.com">ros</maintainer>
8   <license>TODO: License declaration</license>
9
10  <buildtool_depend>ament_cmake</buildtool_depend>
11
12  <test_depend>ament_lint_auto</test_depend>
13  <test_depend>ament_lint_common</test_depend>
14
15  <export>
16    <build_type>ament_cmake</build_type>
17  </export>
18 </package>
```

CMakeLists.txt 文件是编译规则, C++ 代码需要编译才能运行, 所以必须要

在该文件中设置如何编译，使用 CMake 语法。

```
1 cmake_minimum_required(VERSION 3.8)
2 project(pkg_c)
3
4 if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
5   add_compile_options(-Wall -Wextra -Wpedantic)
6 endif()
7
8 # find dependencies
9 find_package(ament_cmake REQUIRED)
10 # uncomment the following section in order to fill in
11 # further dependencies manually.
12 # find_package(<dependency> REQUIRED)
13
14 add_executable(hello_ros src/hello_ros.cpp)
15 target_include_directories(hello_ros PUBLIC
16   ${CMAKE_CURRENT_SOURCE_DIR}/include>
17   ${CMAKE_CURRENT_SOURCE_DIR}/include>)
18 target_compile_features(hello_ros PUBLIC c_std_99 cxx_std_17) # Require C99 and C++17
19
20 install(TARGETS hello_ros
21   DESTINATION lib/${PROJECT_NAME})
22
23 if(BUILD_TESTING)
24   find_package(ament_lint_auto REQUIRED)
25   # the following line skips the linter which checks for copyrights
26   # comment the line when a copyright and license is added to all source files
27   set(ament_cmake_copyright_FOUND TRUE)
28   # the following line skips cpplint (only works in a git repo)
29   # comment the line when this package is in a git repo and when
30   # a copyright and license is added to all source files
31   set(ament_cmake_cpplint_FOUND TRUE)
32   ament_lint_auto_find_test_dependencies()
33 endif()
34
35 ament_package()
```

Python 功能包

C++功能包需要将源码编译成可执行文件，但是 Python 语言是解析型的，不需要编译，所以会有一些不同，但也会有这两个文件：**package.xml** 和 **setup.py**。



package.xml 文件的主要内容和 C++版本功能包一样，包含功能包的版权描述，和各种依赖的声明。

```
1 <?xml version="1.0"?>
2 <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema"?>
3 <package format="3">
4   <name>pkg_py</name>
5   <version>0.0.0</version>
6   <description>TODO: Package description</description>
7   <maintainer email="email@email.com">ros</maintainer>
8   <license>TODO: License declaration</license>
9
10  <test_depend>ament_copyright</test_depend>
11  <test_depend>ament_flake8</test_depend>
12  <test_depend>ament_pep257</test_depend>
13  <test_depend>python3-pytest</test_depend>
14
15  <export>
16    <build_type>ament_python</build_type>
17  </export>
18 </package>
```

setup.py 文件里边也包含一些版权信息，除此之外，还有“entry_points”

配置的程序入口，与 Cmake 中的 install 类似，会让 ros2 run 找到程序入口。

```
1 from setuptools import find_packages, setup
2
3 package_name = 'pkg_py'
4
5 setup(
6     name=package_name,
7     version='0.0.0',
8     packages=find_packages(exclude=['test']),
9     data_files=[
10         ('share/ament_index/resource_index/packages',
11          ['resource/' + package_name]),
12         ('share/' + package_name, ['package.xml']),
13     ],
14     install_requires=['setuptools'],
15     zip_safe=True,
16     maintainer='ros',
17     maintainer_email='yzhcat@qq.com',
18     description='TODO: Package description',
19     license='TODO: License declaration',
20     tests_require=['pytest'],
21     entry_points={
22         'console_scripts': [
23             'hello_ros = pkg_py.hello_ros:main'
24         ],
25     },
26 )
```

话题通信程序实现

可以先运行默认生成的 hello 程序。

```
source install/local_setup.bash
ros2 run pkg_c hello_ros
ros2 run pkg_py hello_ros
```

终端会显示 hello world pkg_c package 和 Hi from pkg_py.

下面完成两个简单的程序，创建 ros 节点，完成节点间数据的收发。



进入功能包创建文件

topic_helloworld_pub.py、topic_helloworld_sub.py

```
touch topic_helloworld_pub.py
touch topic_helloworld_sub.py
```

```

1#!/usr/bin/env python3
2# -*- coding: utf-8 -*-
3
4"""
5@作者: 古月原(guyuehome.com)
6@说明: ROS2话题示例-发布"Hello World"话题
7"""
8
9import rclpy # ROS2 Python接口库
10from rclpy.node import Node # ROS2 节点类
11from std_msgs.msg import String # 字符串消息类型
12
13"""
14创建一个发布者节点
15"""
16class PublisherNode(Node):
17
18    def __init__(self, name):
19        super().__init__(name) # ROS2节点父类初始化
20        self.pub = self.create_publisher(String, "chatter", 10) # 创建发布者对象 (消息类型、话题名、队列长度)
21        self.timer = self.create_timer(0.5, self.timer_callback) # 创建一个定时器 (单位为秒的周期, 定时执行的回调函数)
22
23    def timer_callback(self):
24        msg = String() # 创建定时器周期执行的回调函数
25        msg.data = "Hello World" # 创建一个String类型的消息对象
26        self.pub.publish(msg) # 填充消息对象中的消息数据
27        self.get_logger().info('Publishing: "%s"' % msg.data) # 发布话题消息
28
29def main(args=None):
30    rclpy.init(args=args) # ROS2节点主入口main函数
31    node = PublisherNode("topic_helloworld_pub") # ROS2 Python接口初始化
32    rclpy.spin(node) # 创建ROS2节点对象并进行初始化
33    node.destroy_node() # 循环等待ROS2退出
34    rclpy.shutdown() # 销毁节点对象
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
```



```
cd ~/demo_ws
colcon build --packages-select pkg_py
```

运行命令

```
source install/local_setup.bash
ros2 run pkg_py topic_helloworld_pub
```

再开一个终端

```
source install/local_setup.bash
ros2 run pkg_py topic_helloworld_sub
```

```

ros@u22: ~/dev_ws
[INFO] [1753946575.826149734] [topic_helloworld_sub]: I heard: 'Hello World'
[INFO] [1753946576.212523376] [topic_helloworld_sub]: I heard: 'Hello World'
[INFO] [1753946576.827120587] [topic_helloworld_sub]: I heard: 'Hello World'
[INFO] [1753946577.326657653] [topic_helloworld_sub]: I heard: 'Hello World'
[INFO] [1753946577.830639369] [topic_helloworld_sub]: I heard: 'Hello World'
[INFO] [1753946578.325158486] [topic_helloworld_sub]: I heard: 'Hello World'
[INFO] [1753946578.825453782] [topic_helloworld_sub]: I heard: 'Hello World'
[INFO] [1753946579.325363986] [topic_helloworld_sub]: I heard: 'Hello World'
[INFO] [1753946579.827319939] [topic_helloworld_sub]: I heard: 'Hello World'
[INFO] [1753946580.326618739] [topic_helloworld_sub]: I heard: 'Hello World'
[INFO] [1753946580.826990768] [topic_helloworld_sub]: I heard: 'Hello World'
[INFO] [1753946581.326906636] [topic_helloworld_sub]: I heard: 'Hello World'
[INFO] [1753946581.825315807] [topic_helloworld_sub]: I heard: 'Hello World'
[INFO] [1753946582.326926611] [topic_helloworld_sub]: I heard: 'Hello World'
[INFO] [1753946582.825451985] [topic_helloworld_sub]: I heard: 'Hello World'
[INFO] [1753946583.325144929] [topic_helloworld_sub]: I heard: 'Hello World'
[INFO] [1753946583.825664989] [topic_helloworld_sub]: I heard: 'Hello World'
[INFO] [1753946584.325444524] [topic_helloworld_sub]: I heard: 'Hello World'
[INFO] [1753946584.826099522] [topic_helloworld_sub]: I heard: 'Hello World'
[INFO] [1753946585.327374143] [topic_helloworld_sub]: I heard: 'Hello World'

ros@u22: ~/dev_ws
[INFO] [1753946575.825729293] [topic_helloworld_pub]: Publishing: 'Hello World'
[INFO] [1753946576.325814166] [topic_helloworld_pub]: Publishing: 'Hello World'
[INFO] [1753946576.826860599] [topic_helloworld_pub]: Publishing: 'Hello World'
[INFO] [1753946577.326246078] [topic_helloworld_pub]: Publishing: 'Hello World'
[INFO] [1753946577.824790273] [topic_helloworld_pub]: Publishing: 'Hello World'
[INFO] [1753946578.324837857] [topic_helloworld_pub]: Publishing: 'Hello World'
[INFO] [1753946578.824992754] [topic_helloworld_pub]: Publishing: 'Hello World'
[INFO] [1753946579.325849259] [topic_helloworld_pub]: Publishing: 'Hello World'
[INFO] [1753946579.826730282] [topic_helloworld_pub]: Publishing: 'Hello World'
[INFO] [1753946580.326994215] [topic_helloworld_pub]: Publishing: 'Hello World'
[INFO] [1753946580.826189321] [topic_helloworld_pub]: Publishing: 'Hello World'
[INFO] [1753946581.325797884] [topic_helloworld_pub]: Publishing: 'Hello World'
[INFO] [1753946581.824653971] [topic_helloworld_pub]: Publishing: 'Hello World'
[INFO] [1753946582.326418333] [topic_helloworld_pub]: Publishing: 'Hello World'
[INFO] [1753946582.825880673] [topic_helloworld_pub]: Publishing: 'Hello World'
[INFO] [1753946583.325157769] [topic_helloworld_pub]: Publishing: 'Hello World'
[INFO] [1753946583.825133386] [topic_helloworld_pub]: Publishing: 'Hello World'
[INFO] [1753946584.325842229] [topic_helloworld_pub]: Publishing: 'Hello World'
[INFO] [1753946584.825377787] [topic_helloworld_pub]: Publishing: 'Hello World'
[INFO] [1753946585.326884826] [topic_helloworld_pub]: Publishing: 'Hello World'

```



topic_helloworld_sub 可以收到 topic_helloworld_pub 发来的数据了。可以用前面使用的 ros2 topic 和 ros2 node 命令查看这两个节点的信息和话题信息。

参考链接

- <https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Creating-A-Workspace/Creating-A-Workspace.html>
- <https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Creating-Your-First-ROS2-Package.html>
- <https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Publisher-And-Subscriber.html>

ROS2 学习资料

服务通信:

- [https://book.guyuehome.com/ROS2/2.核心概念/2.5 服务/](https://book.guyuehome.com/ROS2/2.核心概念/2.5服务/)
- <https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Service-And-Client.html>

自定义消息

<https://book.guyuehome.com/ROS2/2. 核心概念/2.6 通信接口/>

<https://docs.ros.org/en/humble/Concepts/Basic/About-Interfaces.html>

<https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Single-Package-Define-And-Use-Interface.html>

动作通信

<https://docs.ros.org/en/humble/Concepts/Basic/About-Actions.html>

<https://docs.ros.org/en/humble/Tutorials/Intermediate/Writing-an-Action-Server-Client/Py.html>

<https://book.guyuehome.com/ROS2/2. 核心概念/2.7 动作/>

ROS2 Documentation: <https://docs.ros.org/en/humble/>

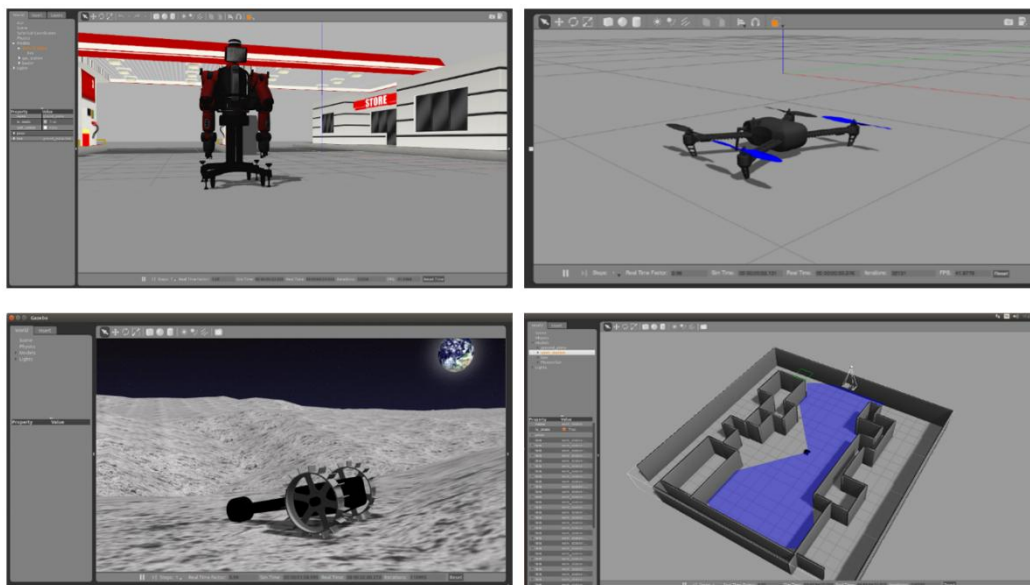
ROS2 21 讲: <https://book.guyuehome.com>

ROS 2 文档: <https://fishros.org/doc/ros2/humble/>

四、 Gazebo 仿真环境

基础介绍

Gazebo 是 ROS 系统中最为常用的三维物理仿真平台，支持动力学引擎，可以实现高质量的图形渲染，不仅可以模拟机器人及周边环境，还可以加入摩擦力、弹性系数等物理属性。

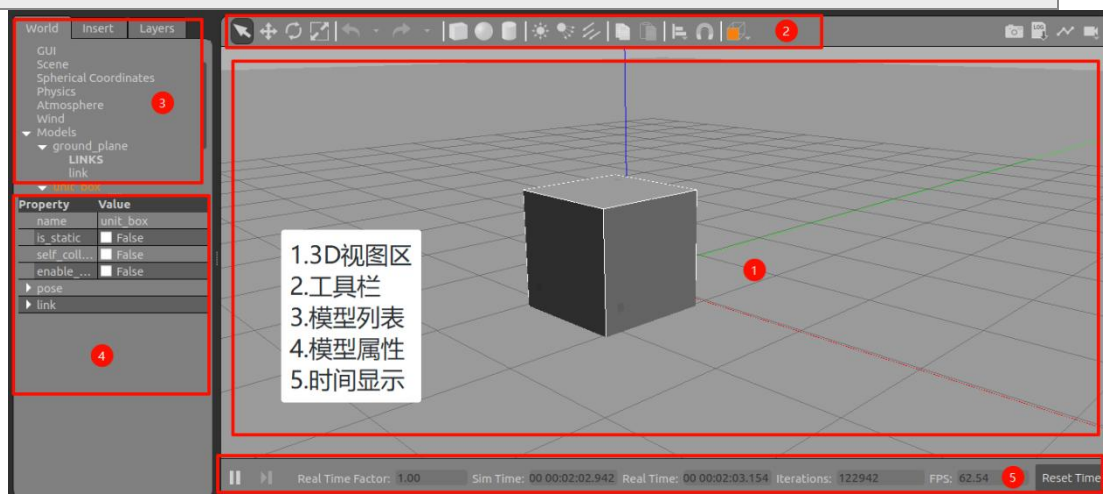


安装

```
sudo apt install ros-humble-gazebo-*
```

运行

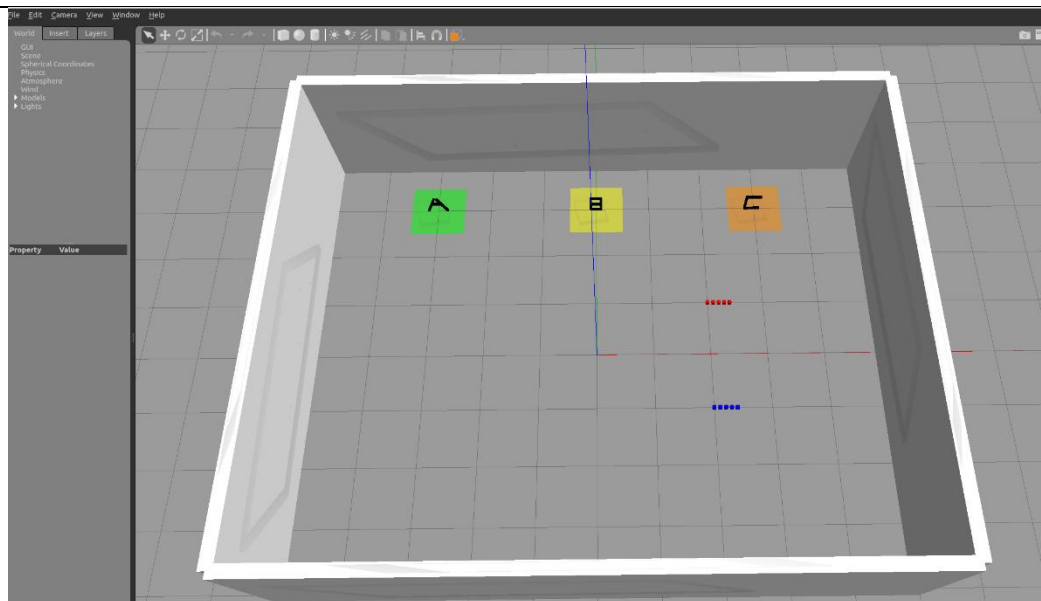
gazebo



加载仿真场景：

Gazebo 命令后面加上仿真场景文件 gazebo <word-path>

```
gazebo dev_ws/src/yzbot/mybot_description/worlds/box.world
```



在 ros 中需要加载 'libgazebo_ros_init.so', 和 'libgazebo_ros_factory.so', 两个插件, 所以 launch 中通常会这样写。

```
gazebo_world.launch.py M X
mybot > launch > gazebo_world.launch.py > generate_launch_description
17 def generate_launch_description():
20     urdf_model_path = os.path.join(model_pkg_share, f'urdf/{urdf_name}.urdf')
27     world_file_path = os.path.join(model_pkg_share, f'worlds/{world_name}.world')
28
29     # Start Gazebo server
30     start_gazebo_cmd = ExecuteProcess(
31         cmd=[
32             'gazebo', '--verbose',
33             '-s',
34             'libgazebo_ros_init.so',
35             '-s',
36             'libgazebo_ros_factory.so',
37             world_file_path
38         ],
39         output='screen')
```

ROS 中的模型文件叫做 [URDF](#), 一种使用 [XML](#) 语法的描述文件; 全称是统一机器人描述格式, 不仅可以清晰描述机器人自身的模型, 还可以描述机器人的外部环境。但是复杂的机器人模型一般会使用 **XACRO** 文件, XACRO 文件加入了更多编程化的实现方法, 可以让模型创建更友好。比如: 宏定义, 文件包含, 可编程接口。XACRO 建模过程就像写代码一样, 功能更为丰富了。

将模型加载到仿真场景中使用 `ros2 run gazebo_ros spawn_entity.py <arg>`. 完整步骤:

先用 xacro 编译 XACRO 文件生成 URDF 文件, 使用 robot_state_publisher 把模型文件发布到 ros 话题, 并提供关节和 tf 数据, 最后 spawn_entity.py 从 robot_description 获取机器人 URDF 模型并载入到仿真场景中。


```
gazebo_world.launch.py M X
mybot > launch > gazebo_world.launch.py > generate_launch_description
17 def generate_launch_description():
18     # 因为 urdf 文件中有一句 $(find mybot) 需要用 xacro 进行编译一下才行
19     xacro_file = urdf_model_path
20     doc = xacro.parse(open(xacro_file))
21     xacro.process_doc(doc)
22     # params = {'robot_description': doc.toxml()}
23     params = {'robot_description': remove_comments(doc.toxml())} # type: ignore
24
25     # 启动了 robot_state_publisher 节点后, 该节点会发布 robot_description 话题, 话题内容是模型文件 urdf 的内容
26     # 并且会订阅 /joint_states 话题, 获取关节的数据, 然后发布 tf 和 tf_static 话题.
27     node_robot_state_publisher = Node(
28         package='robot_state_publisher',
29         executable='robot_state_publisher',
30         parameters=[{'use_sim_time': True}, params, {'publish_frequency': 15.0}],
31         output='screen'
32     )
33
34     # Launch the robot, 通过 robot_description 话题进行模型内容获取从而在 gazebo 中生成模型
35     spawn_x_val = '0.0'
36     spawn_y_val = '0.0'
37     spawn_z_val = '0.0'
38     spawn_yaw_val = '0.0'
39     spawn_entity_cmd = Node(
40         package='gazebo_ros',
41         executable='spawn_entity.py',
42         arguments=['-entity', robot_name_in_model,
43                  '-topic', 'robot_description',
44                  '-x', spawn_x_val, '-y', spawn_y_val, '-z', spawn_z_val, '-Y', spawn_yaw_val], output='screen')
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
```

启动脚本

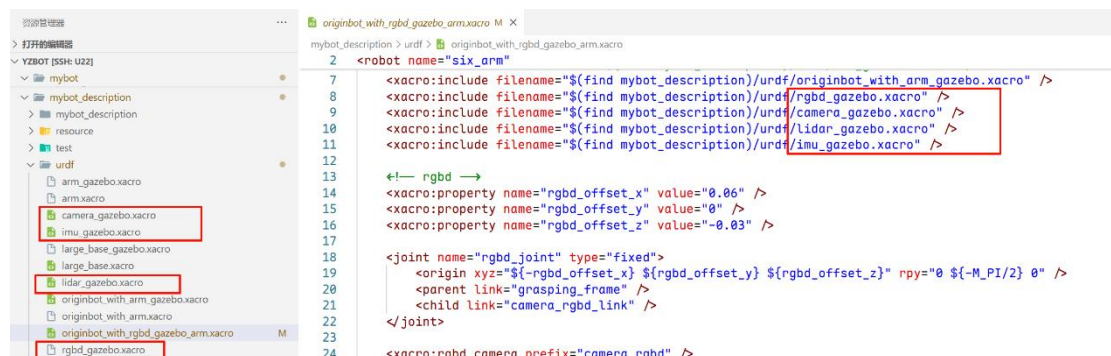
```
ros2 launch mybot gazebo_world.launch.py
```

启动仿真后, 再次使用 `ros2 topic list -v`, 可以看到输出了很多话题, 如图像话题 `/camera/image_raw`, 雷达话题 `/scan`, 速度控制话题 `/cmd_vel`; 可以使用前面学习的 `ros2` 命令行工具或者 `rqt` 查看或发布数据。

```
ros@u22:~$ ros2 topic list -v
Published topics:
* /accel/filtered [geometry_msgs/msg/AccelWithCovarianceStamped] 1 publisher
* /arm_controller/controller_state [control_msgs/msg/JointTrajectoryControllerState] 1 publisher
* /arm_controller/state [control_msgs/msg/JointTrajectoryControllerState] 1 publisher
* /arm_controller/transition_event [lifecycle_msgs/msg/TransitionEvent] 1 publisher
* /camera/camera_info [sensor_msgs/msg/CameraInfo] 1 publisher
* /camera/image_raw [sensor_msgs/msg/Image] 1 publisher
* /camera_rgb/camera_rgb/camera_info [sensor_msgs/msg/CameraInfo] 1 publisher
* /camera_rgb/camera_rgb/depth/image_raw [sensor_msgs/msg/Image] 1 publisher
* /camera_rgb/camera_rgb/image_raw [sensor_msgs/msg/Image] 1 publisher
* /camera_rgb/camera_rgb/points [sensor_msgs/msg/PointCloud2] 1 publisher
* /clock [roscpp_msgs/msg/Clock] 1 publisher
* /diagnostics [diagnostic_msgs/msg/DiagnosticArray] 1 publisher
* /dynamic_joint_states [control_msgs/msg/DynamicJointState] 1 publisher
* /gripper_controller/controller_state [control_msgs/msg/JointTrajectoryControllerState] 1 publisher
* /gripper_controller/state [control_msgs/msg/JointTrajectoryControllerState] 1 publisher
* /gripper_controller/transition_event [lifecycle_msgs/msg/TransitionEvent] 1 publisher
* /imu [sensor_msgs/msg/Imu] 1 publisher
* /joint_state_broadcaster/transition_event [lifecycle_msgs/msg/TransitionEvent] 1 publisher
* /joint_states [sensor_msgs/msg/JointState] 1 publisher
* /odom [nav_msgs/msg/Odometry] 1 publisher
* /odometry/filtered [nav_msgs/msg/Odometry] 1 publisher
* /parameter_events [rcl_interfaces/msg/ParameterEvent] 14 publishers
* /performance_metrics [gazebo_msgs/msg/PerformanceMetrics] 1 publisher
* /robot_description [std_msgs/msg/String] 1 publisher
* /rosout [rcl_interfaces/msg/Log] 15 publishers
* /scan [sensor_msgs/msg/LaserScan] 1 publisher
* /tf [tf2_msgs/msg/TFMessage] 3 publishers
* /tf_static [tf2_msgs/msg/TFMessage] 1 publisher

Subscribed topics:
* /arm_controller/joint_trajectory [trajectory_msgs/msg/JointTrajectory] 1 subscriber
* /clock [roscpp_msgs/msg/Clock] 14 subscribers
* /cmd_vel [geometry_msgs/msg/Twist] 1 subscriber
* /gripper_controller/joint_trajectory [trajectory_msgs/msg/JointTrajectory] 1 subscriber
* /imu [sensor_msgs/msg/Imu] 1 subscriber
* /joint_states [sensor_msgs/msg/JointState] 1 subscriber
* /odom [nav_msgs/msg/Odometry] 1 subscriber
* /parameter_events [rcl_interfaces/msg/ParameterEvent] 15 subscribers
* /set_pose [geometry_msgs/msg/PoseWithCovarianceStamped] 1 subscriber
* /tf [tf2_msgs/msg/TFMessage] 1 subscriber
* /tf_static [tf2_msgs/msg/TFMessage] 1 subscriber
```

相机雷达等传感器数据也是由 gazebo 传感器插件提供的，模型上仿真了深度相机，IMU,rgb 相机，雷达；仿真参数也可以按需要修改。



参考资料

Quick:

<https://book.guyuehome.com/ROS2/3.常用工具/3.3 URDF/>

<https://book.guyuehome.com/ROS2/3.常用工具/3.4 Gazebo/>

URDF:

<https://docs.ros.org/en/humble/Tutorials/Intermediate/URDF/URDF-Main.html>

<http://sdformat.org/spec?ver=1.5&elem=link>

<https://wiki.ros.org/cn/urdf/Tutorials>

Gazebo:

<https://classic.gazebosim.org/tutorials>

<https://classic.gazebosim.org/tutorials?cat=build world>

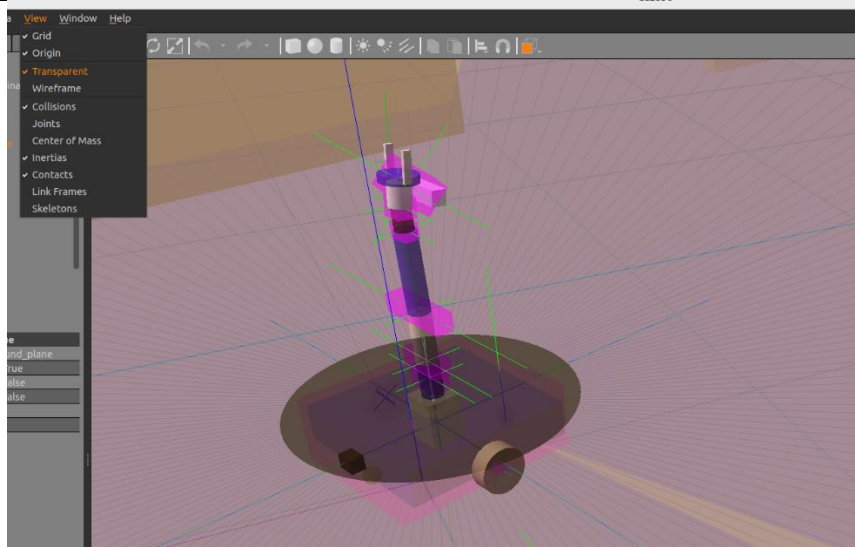
<https://classic.gazebosim.org/tutorials?cat=build robot>

<https://classic.gazebosim.org/tutorials?cat=sensors>

常见问题

控制小车模型移动时模型会倒，模型碰到墙或轮子压到方块后模型会倒，模型到处飞？

在 gazebo 的 View 菜单显示更多内容。



修改模型文件参数，如：

- 检查碰撞体积是否正常（黄色），应该刚好或小于模型显示的尺寸。
- 检查惯性矩阵是否有异常（粉色），不应该有远大于模型的异常方块。
- 降低重心，减小机械臂各关节的质量和惯性矩阵，增加底盘的质量。
- 减小场景中方块的质量，和摩擦力
- 底盘增加支撑轮

```
room.world X
mybot_description > worlds > room.world
2 <sdf version='1.4'>
3 <world name='default'>
3682 <model name='red_cube_1'>
3683 <pose>-1.5 1.0 0.75 0 0 0</pose>
3684 <link name='link'>
3685 <inertial><mass>0.005</mass></inertial>
3686 <collision name='collision'><geometry><box><size>0.03 0.03 0.
03</size></box></geometry></collision>
3687 <visual name='visual'>
3688 <geometry><box><size>0.03 0.03 0.03</size></box></geometry>
3689 <material><ambient>1 0 0 1</ambient><diffuse>1 0 0 1</
diffuse></material>
3690 </visual>
3691 </link>
3692 </model>
- 3693
```

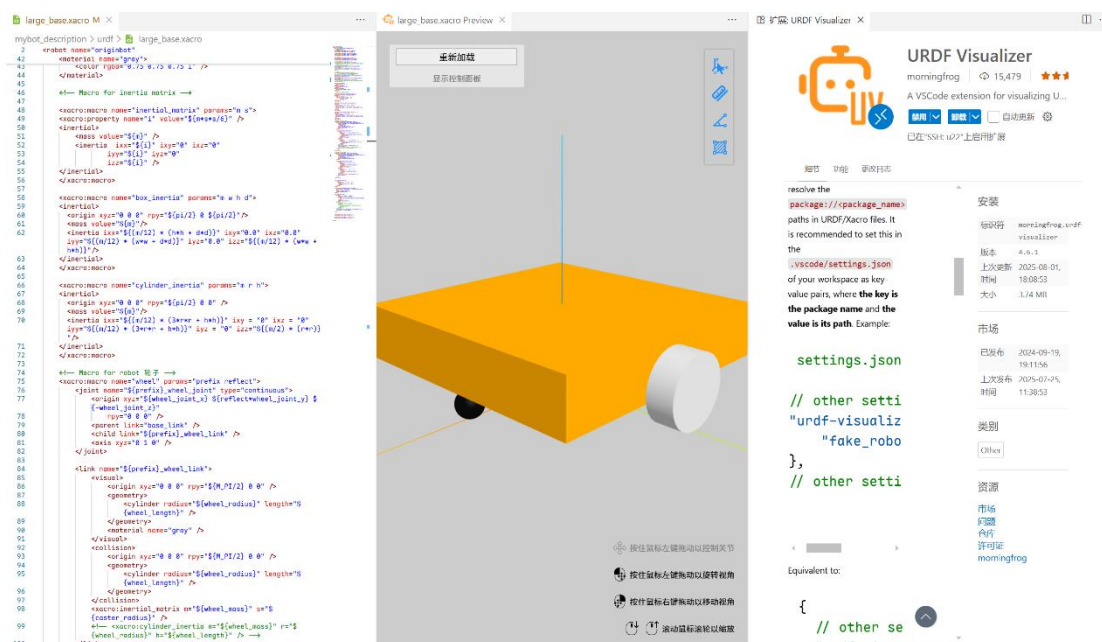
```

<contact>
  <ode>
    <soft_cfm>0.001</soft_cfm>
    <soft_erp>0.2</soft_erp>
    <max_vel>0.1</max_vel>
    <min_depth>0.001</min_depth>
  </ode>
</contact>
<bounce />
<friction>
  <ode>
    <mu>0.01</mu>
    <mu2>0.01</mu2>
  </ode>
  <torsional>
    <ode />
  </torsional>
</friction>

```

<!-- 增加软接触，避免弹飞 -->
 <!-- 限制最大恢复速度 -->
 <!-- 增加最小穿透深度 -->
 <!-- 极低摩擦力，容易滑开 -->

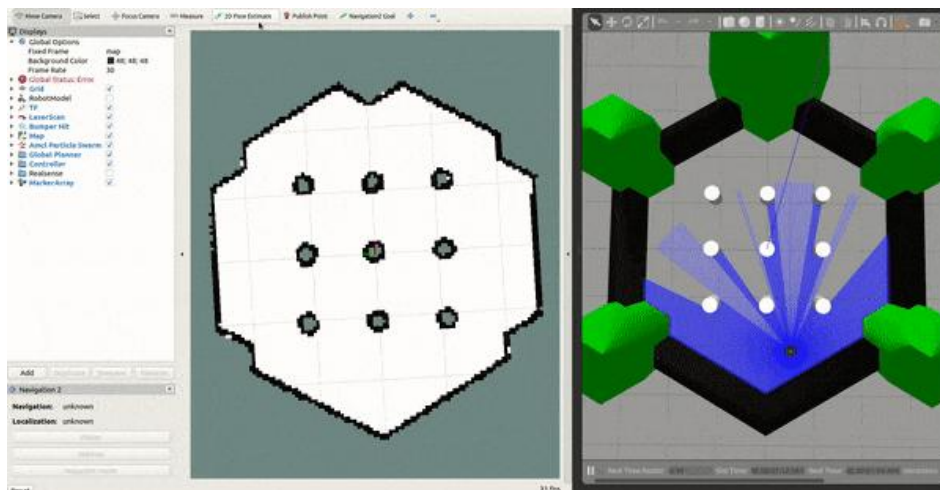
可以使用 vscode 插件，比较直观的查看模型变更。



五、 Nav2 导航系统

基础介绍

Nav2 是 ROS1 导航框架的后继者，它采用了与自动驾驶车辆相同的技术，并针对移动机器人及地面机器人进行了降级、优化和重构。该项目使移动机器人能够在复杂环境中导航，完成用户定义的应用任务，几乎适用于任何类型的机器人运动学。它不仅支持从点 A 移动到点 B，还能处理中间姿态，并实现其他类型的任务，如物体跟随、全覆盖导航等。它提供感知、规划、控制、定位、可视化等多种功能，用于构建高度可靠的自主系统。



Nav2: <https://docs.nav2.org/>

Nav2 中文文档:

http://dev.nav2.fishros.com/doc/getting_started/index.html

导航快速体验

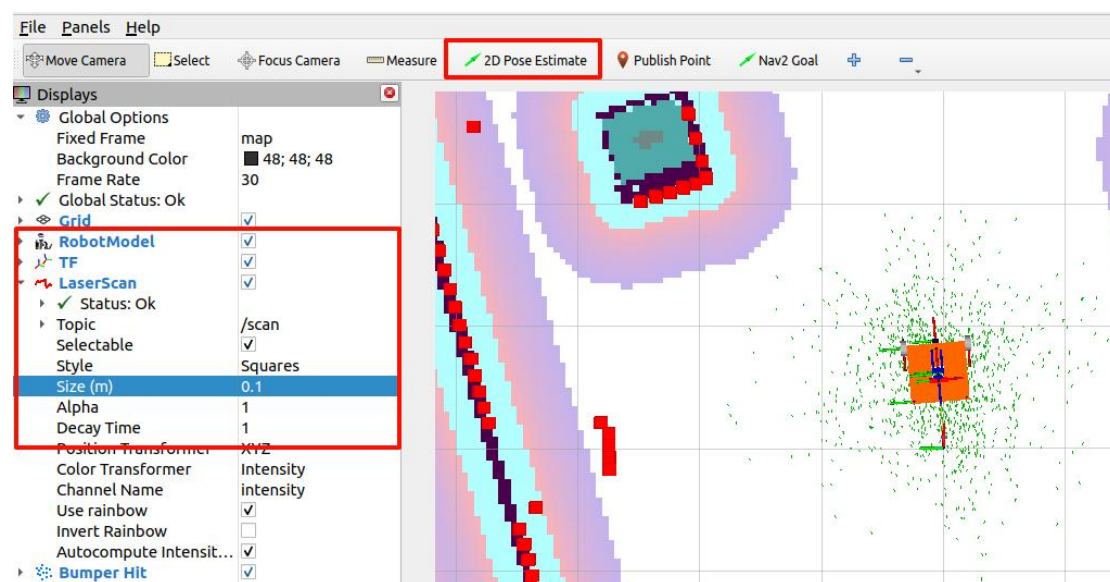
#启动仿真

```
ros2 launch mybot gazebo_world.launch.py
```

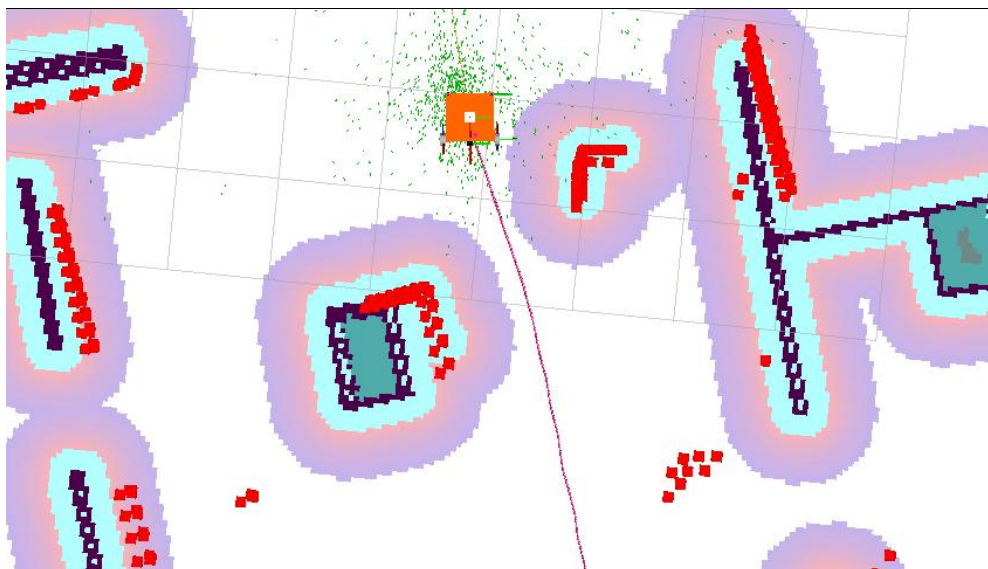
#启动 nav2 导航

```
ros2 launch bot_navigation nav_bringup_gazebo.launch.py
```

使用 2D Pose Estimate 工具设置初始位置。可调整雷达数据显示效果，根据雷达数据多次调整，直到雷达扫描的轮廓和地图大致重合



使用 Nav2 Goal 工具在地图上拖动，发布目标点和机器人朝向，之后地图上会显示运动轨迹，机器人会避开雷达扫描到的障碍物，到达目标点



Nav2 导航框架包含地图服务器、AMCL 定位、路径规划服务器和全局代价地图、控制器服务器和局部代价地图和导航行为树等多个模块组成。

在导航功能的启动文件中启动了 nav2_bringup bringup_launch.py, 会根据配置文件的内容启动所需的组件。

```

nav_bringup_gazebo.launch.py M x originbot_nav2.yaml M
bot_navigation > launch > nav_bringup_gazebo.launch.py > generate_launch_description
30 def generate_launch_description():
31     navigation2_dir = get_package_share_directory('bot_navigation')
32     nav2_bringup_dir = get_package_share_directory('nav2_bringup')
33     map_yaml_file = "map.yaml"
34
35     use_sim_time = LaunchConfiguration('use_sim_time', default='true')
36     map_yaml_path = LaunchConfiguration('map', default=os.path.join(navigation2_dir, f'maps/{map_yaml_file}'))
37     nav2_param_path = LaunchConfiguration('params_file', default=os.path.join(navigation2_dir, 'param', 'originbot_nav2.yaml'))
38     slam = LaunchConfiguration('slam', default='False')
39     rviz_config_dir = os.path.join(nav2_bringup_dir, 'rviz', 'nav2_default_view.rviz')
40
41     return LaunchDescription([
42         DeclareLaunchArgument('use_sim_time', default_value=use_sim_time, description='Use simulation (Gazebo) clock if true'),
43         DeclareLaunchArgument('params_file', default_value=nav2_param_path, description='Full path to param file to load'),
44         DeclareLaunchArgument("map", default_value=map_yaml_path, description='Full path to map file to load'),
45
46         IncludeLaunchDescription(
47             PythonLaunchDescriptionSource([nav2_bringup_dir, '/launch', '/bringup_launch.py']),
48             launch_arguments={
49                 'map': map_yaml_path,
50                 'use_sim_time': use_sim_time,
51                 'params_file': nav2_param_path,
52                 'slam': slam,}.items(),

```

这里向地图服务器传入了地图路径, originbot_nav2.yaml 中配置了 AMCL, 规划器、控制器、代价地图等参数配置。

bot_navigation > param > originbot_nav2.yaml

```
1 > amcl: ...
8 >   base_frame_id: "base_footprint" ...
28 >   robot_model_type: "nav2_amcl::DifferentialMotionModel" ...
39 >   scan_topic: scan
40
41 bt_navigator:
42   ros__parameters:
43     global_frame: map
44     robot_base_frame: base_footprint
45     odom_topic: /odometry/filtered
46 >   # odom_topic: /odom ...
47
48
49
68
69 controller_server:
70   ros__parameters:
71     controller_frequency: 20.0
```

通常要先配置数据源话题，如雷达 scan、里程计 odom 等；和机器人参数如基座标、机器人尺寸、差速模型等。其他可以保持默认后续调整。可参考官方文档调整 <https://docs.nav2.org/configuration/index.html>

发布一个航点

输入 ros2 action list 可以看到 /navigate_to_pose 这个动作接口，类型是 nav2_msgs/action/NavigateToPose 作用导航到指定位置；可以使用 ros2 interface 命令查看信息结构，或者 rqt 工具查看。

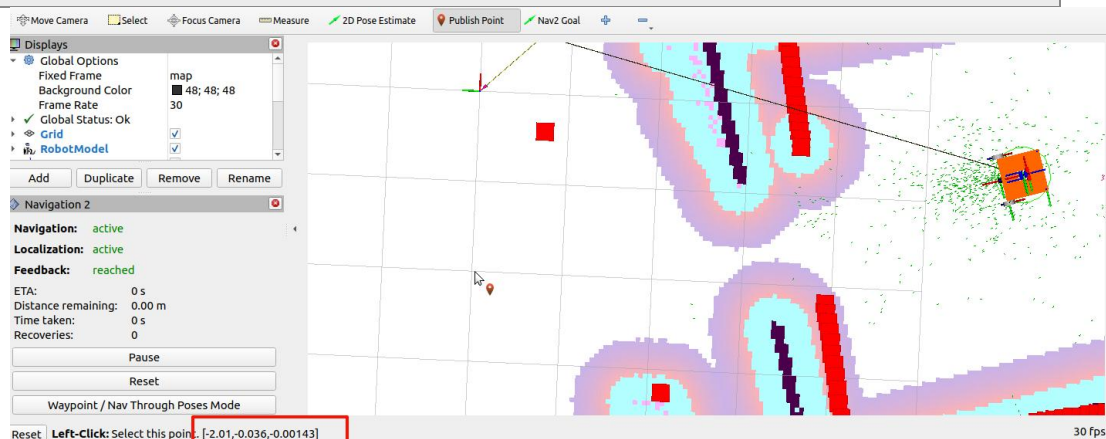
ros2 interface show nav2_msgs/action/NavigateToPose

```
ros@22:~$ ros2 action list
/arn_controller/follow_joint_trajectory
/assisted_teleop
/backstop
/compute_path_through_poses
/compute_path_to_pose
/drive_on_heading
/follow_path
/follow_waypoints
/gripper_controller/follow_joint_trajectory
/navigate_through_poses
/navigate_to_pose
/smooth_path
/spin
/wait
ros@22:~$ ros2 interface show nav2_msgs/action/NavigateToPose
#goal definition
geometry_msgs/PoseStamped pose
std_msgs/Header header
  builtin_interfaces/Time stamp
  int32 sec
  uint32 nanosec
  string frame_id
  Pose pose
    Point position
      float64 x
      float64 y
      float64 z
    Quaternion orientation
      float64 x
      float64 y
      float64 z
      float64 w
  string behavior_tree
  #result definition
std_msgs/Empty result
#feedback definition
geometry_msgs/PoseStamped current_pose
std_msgs/Header header
  builtin_interfaces/Time stamp
  int32 sec
  uint32 nanosec
  string frame_id
  Pose pose
    Point position
      float64 x
      float64 y
      float64 z
    Quaternion orientation
      float64 x
      float64 y
      float64 z
      float64 w
  builtin_interfaces/Duration navigation_time
  int32 sec
  uint32 nanosec
  builtin_interfaces/Duration estimated_time_remaining
  int32 sec
  uint32 nanosec
  int16 number_of_recoveries
  float64 distance_remaining
```

File Plugins Running Perspectives Help			
Action Type Browser			
Message: nav2_msgs			
- // NavigateToPose			
Tree			
- Action Goal	nav2_msgs/NavigateToPose/Goal	nav2_msgs/NavigateToPose/Goal	Path
- pose	geometry_msgs/PoseStamped	nav2_msgs/NavigateToPose/Goal/pose	
- header	std_msgs/Header	nav2_msgs/NavigateToPose/Goal/pose/header	
- stamp	builtin_interfaces/Time	nav2_msgs/NavigateToPose/Goal/pose/header/stamp	
- frame_id	string	nav2_msgs/NavigateToPose/Goal/pose/header/frame_id	
- pose	geometry_msgs/Pose	nav2_msgs/NavigateToPose/Goal/pose/pose	
- position	geometry_msgs/Point	nav2_msgs/NavigateToPose/Goal/pose/position	
- x	double	nav2_msgs/NavigateToPose/Goal/pose/position/x	
- y	double	nav2_msgs/NavigateToPose/Goal/pose/position/y	
- z	double	nav2_msgs/NavigateToPose/Goal/pose/position/z	
- orientation	geometry_msgs/Quaternion	nav2_msgs/NavigateToPose/Goal/pose/orientation	
- x	double	nav2_msgs/NavigateToPose/Goal/pose/orientation/x	
- y	double	nav2_msgs/NavigateToPose/Goal/pose/orientation/y	
- z	double	nav2_msgs/NavigateToPose/Goal/pose/orientation/z	
- w	double	nav2_msgs/NavigateToPose/Goal/pose/orientation/w	
- behavior_tree	string	nav2_msgs/NavigateToPose/Goal/behavior_tree	
- Action Result	nav2_msgs/NavigateToPose/Result	nav2_msgs/NavigateToPose/Result	
- result	std_msgs/Empty	nav2_msgs/NavigateToPose/Result/result	
- Action Feedback	nav2_msgs/NavigateToPose/Feedback	nav2_msgs/NavigateToPose/Feedback	
- current_pose	geometry_msgs/PoseStamped	nav2_msgs/NavigateToPose/Feedback/current_pose	
- navigation_time	builtin_interfaces/Duration	nav2_msgs/NavigateToPose/Feedback/navigation_time	
- estimated_time_remaining	builtin_interfaces/Duration	nav2_msgs/NavigateToPose/Feedback/estimated_time_remaining	
- number_of_recoveries	int16	nav2_msgs/NavigateToPose/Feedback/number_of_recoveries	
- distance_remaining	float64	nav2_msgs/NavigateToPose/Feedback/distance_remaining	

可以在终端使用 ros2 action send_goal 发布动作请求。

```
ros2 action send_goal /navigate_to_pose nav2_msgs/action/NavigateToPose "{pose: {header: {frame_id: map}, pose: {position: {x: -2, y: 0}}}}" --feedback
```



BasicNavigator 提供了一个简单的接口实现发布导航的功能，

https://docs.nav2.org/commander_api/index.html

```
from geometry_msgs.msg import PoseStamped
from nav2_simple_commander.robot_navigator import BasicNavigator, TaskResult
import rclpy
from rclpy.duration import Duration

rclpy.init()
nav = BasicNavigator()
# 等待导航启动完成
nav.waitUntilNav2Active()
# 设置目标点坐标
goal_pose = PoseStamped()
goal_pose.header.frame_id = 'map'
goal_pose.header.stamp = nav.get_clock().now().to_msg()
goal_pose.pose.position.x = 1.0
goal_pose.pose.position.y = 1.0
goal_pose.pose.orientation.w = 1.0
# 发送目标接收反馈结果
nav.goToPose(goal_pose)
while not nav.isTaskComplete():
    feedback = nav.getFeedback()
    nav.get_logger().info('导航...')
# 超时自动取消
if Duration.from_msg(feedback.navigation_time) > Duration(seconds=600): # type: ignore
    nav.cancelTask()
# 最终结果判断
result = nav.getResult()
if result == TaskResult.SUCCEEDED:
    nav.get_logger().info('导航结果: 成功')
elif result == TaskResult.CANCELED:
    nav.get_logger().warn('导航结果: 被取消')
elif result == TaskResult.FAILED:
    nav.get_logger().error('导航结果: 失败')
```



```
else:  
    nav.get_logger().error('导航结果: 返回状态无效')
```

在基础可以实现这样的任务函数。

```
1 def run_full_mission(self, target, back):  
2     """执行完整任务序列"""  
3     self.get_logger().info("Starting full mission sequence")  
4  
5     # 第一步: 导航到目标位置  
6     target_pose = self.create_pose(target[0], target[1], target[2])  
7     nav_success = self.navigate_to(target_pose)  
8  
9     if not nav_success:  
10        self.get_logger().error("Failed to reach target. Aborting mission.")  
11        return  
12  
13    # 第二步: 执行抓取  
14    self.get_logger().info("Executing grasp action (simulated)")  
15    self.simulate_grasp() # 模拟抓取  
16  
17    # 第三步: 导航回仓库  
18    warehouse_pose = self.create_pose(back[0], back[1], back[2])  
19    nav_success = self.navigate_to(warehouse_pose)  
20  
21    if nav_success:  
22        self.get_logger().info("Mission completed successfully!")  
23    else:  
24        self.get_logger().error("Failed to return to warehouse")
```

参考资料

Nav2: <https://docs.nav2.org/>

configuration : <https://docs.nav2.org/configuration/index.html>

Nav2 中文文档:

http://dev.nav2.fishros.com/doc/getting_started/index.html

BasicNavigator: https://docs.nav2.org/commander_api/index.html

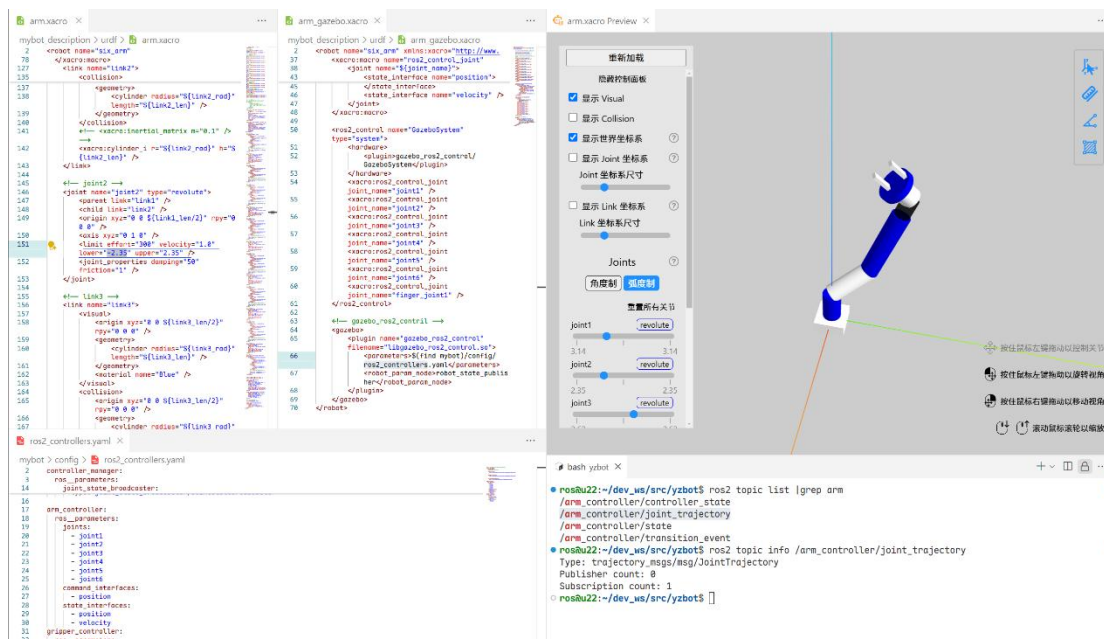
Nav2 Behavior Trees: https://docs.nav2.org/behavior_trees/index.html

六、机械臂控制和 MoveIt2

MoveIt2 是 ROS2 中的一个集成开发平台, 一个专注于机器人操作 (尤其是机械臂) 的开源运动规划框架。由多种用于操纵机械臂的功能包组成, 包括: 运动规划、操作、控制接口、逆向运动学、3D 感知和碰撞检测等。MoveIt 简化了复杂的机器人运动规划和应用开发工作。无论是学术研究还是工业应用, MoveIt2 为开发者提供了一个高层次的功能封装, 使得开发者可以更专注于机器

人应用逻辑本身，而不必深入底层算法细节。

一般机械臂的关节都会提供位置控制、力控制或速度控制接口给我们使用。



如 Gazebo 仿真的这个六轴机械臂提供了 `/joint_trajectory` 话题和 `/follow_joint_trajectory` 动作接口，可以控制六个关节的角度。假如我们可以改变每一个关节的角度，那么就可以让机械臂的末端到达各种姿态。

但随之而来的问题就来了，如何让机械臂末端以我们想要的速度、想要的轨迹到达我们想要的位置，基于 ROS2 的机械臂运动规划、感知、运动学、控制和导航的 Moveit2 就被设计出来解决这一类问题。

机械臂控制快速体验

```
ros2 launch mybot gazebo_world.launch.py #启动仿真
```

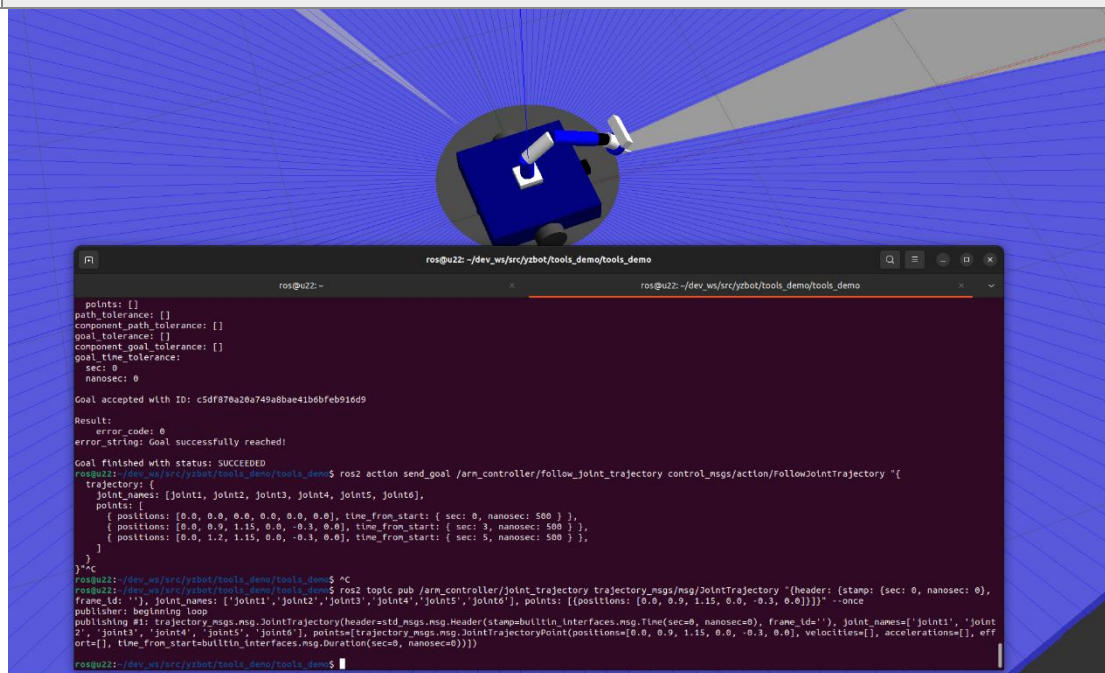
直接控制关节 (Topic)：

```
ros2 topic pub /arm_controller/joint_trajectory trajectory_msgs/msg/JointTrajectory
"{header: {stamp: {sec: 0, nanosec: 0}, frame_id: ''}, joint_names:
['joint1', 'joint2', 'joint3', 'joint4', 'joint5', 'joint6'], points: [{positions: [0.0, 0.9,
1.15, 0.0, -0.3, 0.0]}]}" --once
```

直接控制关节 (Action)：

```
ros2 action send_goal /arm_controller/follow_joint_trajectory
control_msgs/action/FollowJointTrajectory "{
  trajectory: {
    joint_names: [joint1, joint2, joint3, joint4, joint5, joint6],
    points: [
      { positions: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0], time_from_start: { sec: 0, nanosec: 500 } },
```

```
{ positions: [0.0, 0.9, 1.15, 0.0, -0.3, 0.0], time_from_start: { sec: 3, nanosec: 500 } },
{ positions: [0.0, 1.2, 1.15, 0.0, -0.3, 0.0], time_from_start: { sec: 5, nanosec: 500 } },
]
}
}"
```



实际应用中，关节控制通常用在主动控制，如遥控，或者机械臂只要特定几个姿态时的场景；我们很难直接控制关节让机械臂的末端到达指定的位置，如指定物体的抓取，这时就用到了逆运动了。

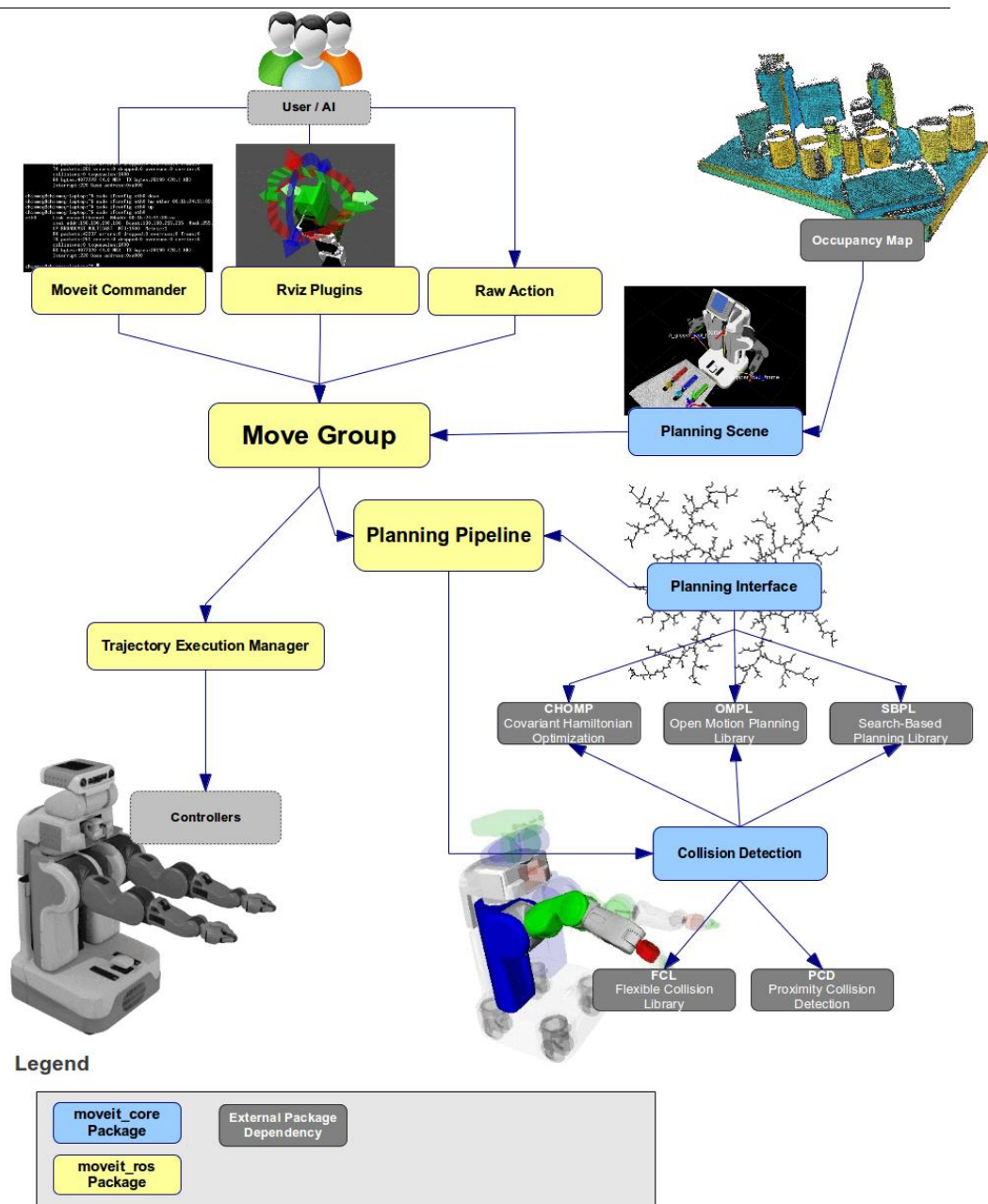
关节正解：[关节角度→末端位姿]，现有的关节角度推导末端执行器位姿

关节逆解：[末端位姿→关节角度]，期望位姿推导关节角度

如果已知目标物体的位置，可以通过以下流程实现控制机械臂末端移动到目标物体位置。

已知目标位姿》计算末端位姿》[当前关节角度+末端位姿]》关节逆解》目标关节角度》电机控制

实际的流程要更复杂，规划器、轨迹优化、关节限制、障碍、碰撞等；

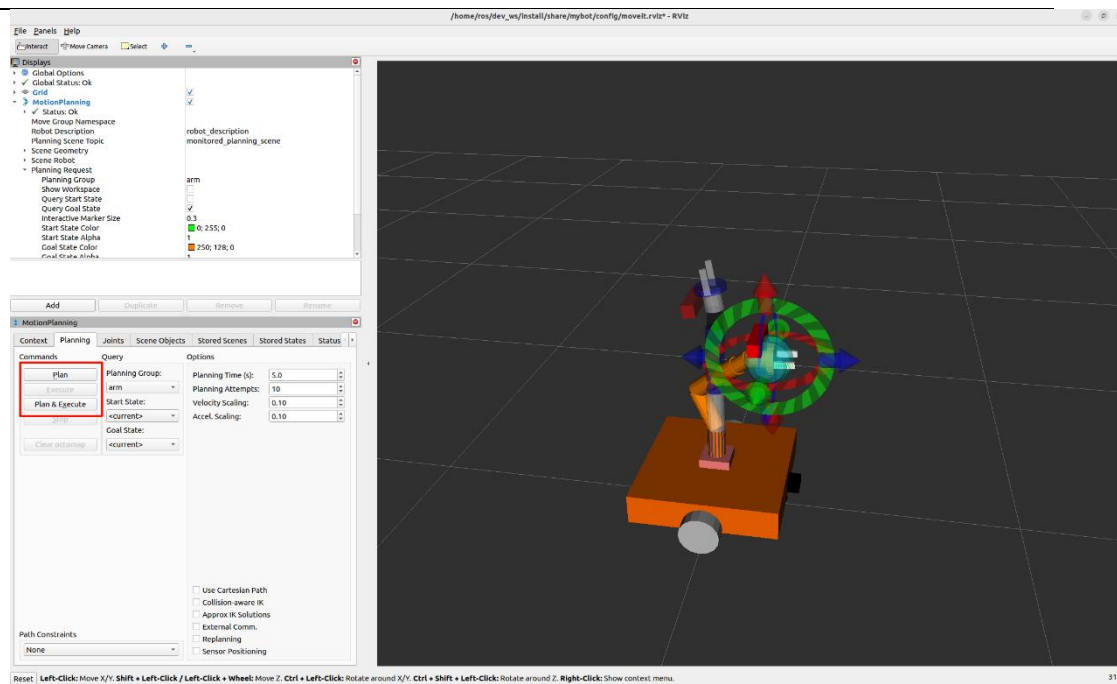


但是 Moveit 提供了比较友好的接口，可以快速使用这一流程。

```
ros2 launch mybot my_moveit_rviz.launch.py #启动 moveit(机械臂规划、控制)
```

可以看到机械臂当前的状态，和 moveit 控制面板

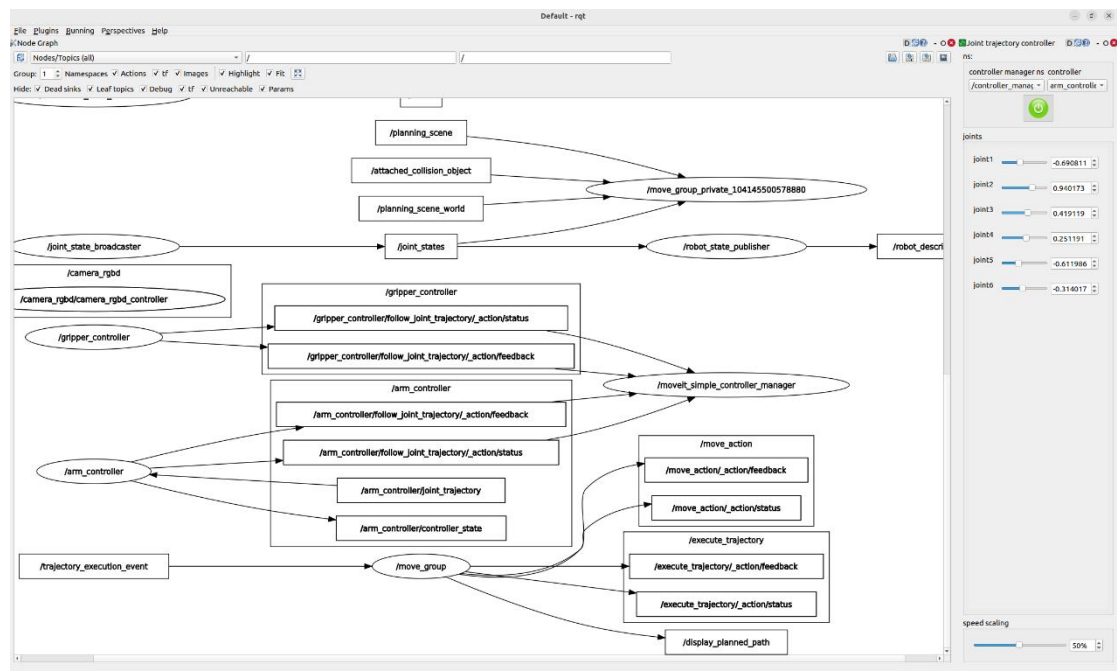
鼠标调整机械臂末端位置，点击面板 Plan & Execute 执行动作，机械臂会运动到目标位置。



参考链接：

https://moveit.picknik.ai/humble/doc/tutorials/quickstart_in_rviz
/quickstart_in_rviz_tutorial.html

发布目标点



核心 MoveIt! 服务

1. 运动规划服务：

o /compute_ik

类型: moveit_msgs/srv/GetPositionIK

作用: 计算逆运动学 (给定末端位姿求关节角度)

o /compute_fk

类型: moveit_msgs/srv/GetPositionFK

作用: 计算正运动学 (给定关节角度求末端位姿)

o /compute_cartesian_path

类型: moveit_msgs/srv/GetCartesianPath

作用: 计算笛卡尔路径 (直线运动规划)

核心 MoveIt! 动作

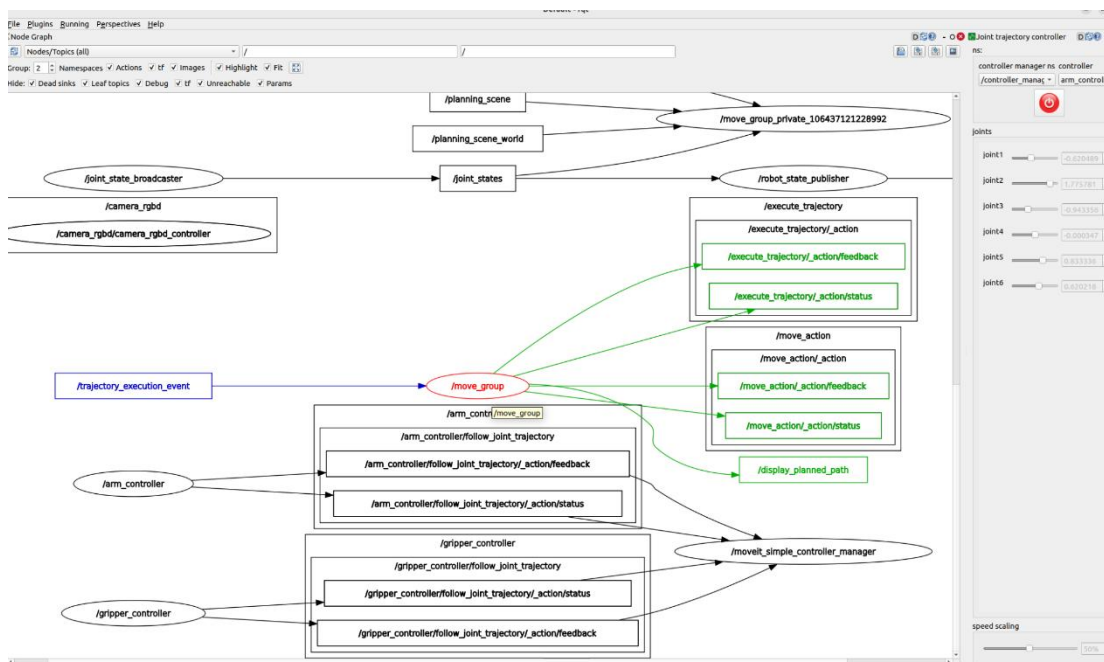
动作接口比服务更适用于长时间运行的任务:

1. /move_action

类型: moveit_msgs/action/MoveGroup

作用: 主运动规划接口, 发送目标位姿后自动完成: 路径规划、轨迹优化、碰撞检测、执行控制。

MoveIt 启动后给出了很多接口, 要控制机械臂末端到指定位置只需要调用 /move_action 动作接口就可以实现。



但是直接调用 /move_action 动作接口比较复杂, 使用 move_group 可以更方

便的实现。

```
cd ~/dev_ws
ros2 pkg create \
  --build-type ament_cmake \
  --dependencies moveit_ros_planning_interface rclcpp \
  --node-name hello_moveit hello_moveit
```

编辑/home/ros/dev_ws/src/yzbot/hello_moveit/src/hello_moveit.cpp

```
#include <memory>
#include <rclcpp/rclcpp.hpp>
#include <moveit/move_group_interface/move_group_interface.h>

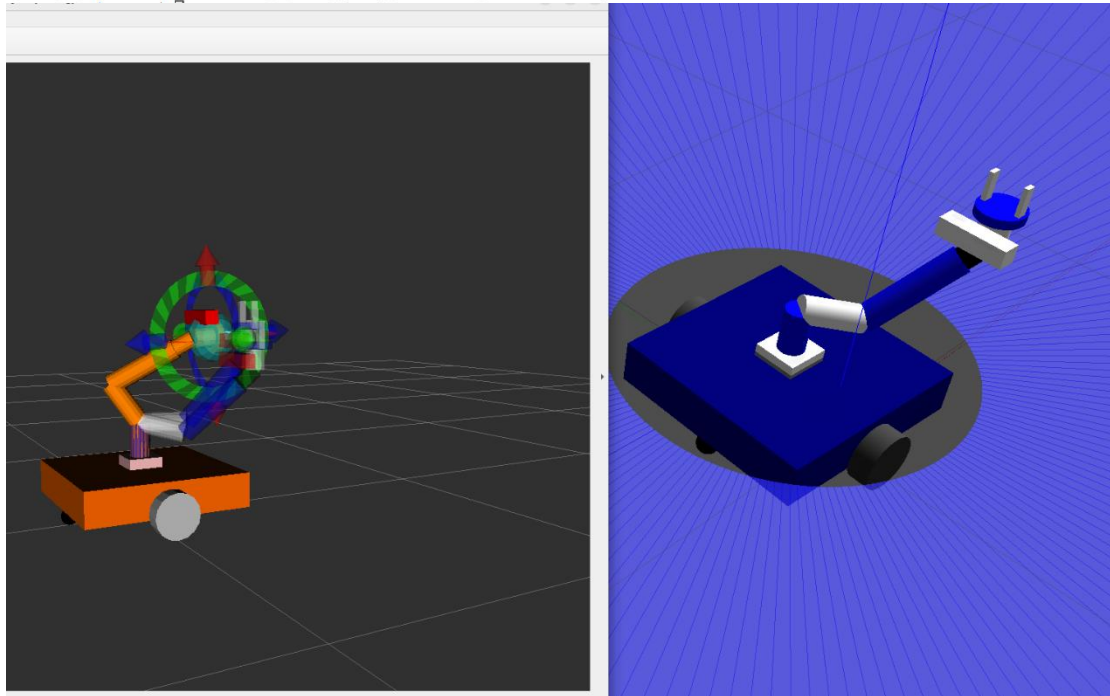
int main(int argc, char *argv[])
{
    // Initialize ROS and create the Node
    rclcpp::init(argc, argv);
    auto const node = std::make_shared<rclcpp::Node>(
        "hello_moveit",
        rclcpp::NodeOptions().automatically_declare_parameters_from_overrides(true));
    // Create a ROS logger
    auto const logger = rclcpp::get_logger("hello_moveit");
    // Create the MoveIt MoveGroup Interface
    using moveit::planning_interface::MoveGroupInterface;
    auto move_group_interface = MoveGroupInterface(node, "arm");
    // Set a target Pose
    geometry_msgs::msg::Pose target_pose;
    target_pose.orientation.w = 1.0;
    target_pose.position.x = 0.28;
    target_pose.position.y = -0.2;
    target_pose.position.z = 0.5;
    move_group_interface.setPoseTarget(target_pose);
    // Create a plan to that target pose
    auto const [success, plan] = [&move_group_interface]
    {
        moveit::planning_interface::MoveGroupInterface::Plan msg;
        auto const ok = static_cast<bool>(move_group_interface.plan(msg));
        return std::make_pair(ok, msg);
    }();
    // Execute the plan
    if (success)
    {
        move_group_interface.execute(plan);
    }
    else
    {
        RCLCPP_ERROR(logger, "Planing failed!");
    }
}
```

```
}  
// Shutdown ROS  
rclcpp::shutdown();  
return 0;  
}
```

编译文件&运行

```
colcon build --packages-select hello_moveit  
source install/setup.bash  
ros2 run hello_moveit hello_moveit
```

```
[INFO] [1754291054.184717628] [move_group_interface]: Ready to take commands for planning group arm.  
[INFO] [1754291054.184817619] [move_group_interface]: MoveGroup action client/server ready  
[INFO] [1754291054.208571838] [move_group_interface]: Planning request accepted  
[INFO] [1754291054.310766061] [move_group_interface]: Planning request complete!  
[INFO] [1754291054.311892627] [move_group_interface]: time taken to generate plan: 0.0246064 seconds  
[INFO] [1754291054.313318400] [move_group_interface]: Execute request accepted  
[INFO] [1754291072.706193909] [move_group_interface]: Execute request success!
```



参考资料

https://moveit.picknik.ai/humble/doc/tutorials/quickstart_in_rviz/quickstart_in_rviz_tutorial.html

https://moveit.picknik.ai/humble/doc/tutorials/your_first_project/your_first_project.html

<https://moveit.picknik.ai/humble/doc/examples/examples.html>

<https://github.com/moveit/moveit2>

七、 计算机视觉技术

机器视觉，就是用计算机来模拟人的视觉功能，但这并不仅仅是人眼的简单延伸，更重要的是像人脑一样，可以从客观事物的图像中提取信息，进行处理并加以理解，最终用于实际检测、控制等场景。



机器视觉凭借速度、精度和可重复性等优势，非常适合对结构化场景进行定量测量。一般来讲，典型的机器视觉系统可以分为如图所示的三个部分：图像采集、图像分析和控制输出。

选择合适的相机是机器视觉系统设计的重要环节，通常相机可以获取的信息除了 RGB 图像，还可以获取如深度、能量等信息，不仅直接决定了采集图像的质量和速度，同时也与整个系统的运行模式相关。

图像处理系统接收到相机传来的数字图像之后，通过各种软件算法进行图像特征提取、特征分析，最后进行判断。这是各种视觉算法研究最为集中的部分，如基于 opencv 实现颜色过滤、轮廓检测，到基于深度学习的目标检测算法。从传统的模式识别算法，到当前热门的各种机器学习方法，都是为了更好的让机器理解环境。

视觉识别的结果最终要和机器人的某些行为绑定，也就是第三个部分——输出，包含 I/O 接口、运动控制、可视化显示等。当图像处理系统完成图像分析后，将识别的结果输出。比如视觉识别到了抓取目标的位置，通过 IO 口控制夹爪完成抓取和放置、将识别的信息可视化等。

Opencv 视觉识别

OpenCV 提供一套开源而且标准的机器视觉处理库，主要使用 C/C++ 语言编写，

执行效率较高。OpenCV 实现了图像处理和计算机视觉方面很多通用算法，在开发视觉应用的时候，就不需要重新去造轮子，而是基于这些基础库，专注自己应用的优化。OpenCV 目前支持的编程语言也非常多，无论你熟悉哪一种，都可以调用 OpenCV 快速开始视觉开发，比如 C ++, [Python](#), [Java](#)、[MATLAB](#) 等语言，而且还支持 Windows, Linux, Android 和 Mac OS 等操作系统。

创建功能包

尝试编写一个节点，实现图像获取、颜色识别、轮廓检测，输出检测信息方便后续控制使用。

```
# 新建功能包 cv_demo
ros2 pkg create --build-type ament_python cv_demo --node-name hello_cv --license Apache-2.0
# 编译
cd ~/demo_ws
colcon build --symlink-install --packages-select cv_demo
# 运行
source install/setup.bash
ros2 run cv_demo hello_cv
```

参数 `--symlink-install` 可以把 `python`, `cfg` 等文件编译时通过链接方式到执行目录，不需重复执行编译。

图像数据

在 ROS 中图像消息通常是通过 Image 和 CompressedImage 消息进行定义的。

```
ros@u22:~/demo_ws$ ros2 interface show sensor_msgs/msg/Image
# This message contains an uncompressed image
# (0, 0) is at top-left corner of image

std_msgs/Header header # Header timestamp should be acquisition time of image
  builtin_interfaces/Time stamp
    int32 sec
    uint32 nanosec
  string frame_id
    # Header frame_id should be optical frame of camera
    # origin of frame should be optical center of camera
    # +x should point to the right in the image
    # +y should point down in the image
    # +z should point into to plane of the image
    # If the frame_id here and the frame_id of the CameraInfo
    # message associated with the image conflict
    # the behavior is undefined

uint32 height # image height, that is, number of rows
uint32 width  # image width, that is, number of columns

# The legal values for encoding are in file src/image_encodings.cpp
# If you want to standardize a new string format, join
# ros-users@lists.ros.org and send an email proposing a new encoding.

string encoding # Encoding of pixels -- channel meaning, ordering, size
               # taken from the list of strings in include/sensor_msgs/image_encodings.hpp

uint8 is_bigendian # is this data bigendian?
uint32 step        # Full row length in bytes
uint8[] data       # actual matrix data, size is (step * rows)
```



```

ros@u22:~/demo_ws$ ros2 interface show sensor_msgs/msg/CompressedImage
# This message contains a compressed image.

std_msgs/Header header # Header timestamp should be acquisition time of image
  builtin_interfaces/Time stamp
    int32 sec
    uint32 nanosec
  string frame_id
    # Header frame_id should be optical frame of camera
    # origin of frame should be optical center of camera
    # +x should point to the right in the image
    # +y should point down in the image
    # +z should point into to plane of the image

string format
  # Specifies the format of the data
  # Acceptable values:
  #   jpeg, png, tiff

uint8[] data # Compressed image buffer
  
```

数据结构中首先是消息头，包含序号、时间戳、坐标系这三项内容。

接下来是一幅图像的相关数据。比如纵向和横向的分辨率，两者做一个乘法之后的结果，就表示这幅图像中有多少像素点，每一个像素点的描述方式，在这个 encoding 中定义，最为常用的是 RGB 格式，也就是一个像素点的颜色，由红、绿、蓝三原色组成。

下边是图像数据的大小端模式，step 表示一行图像数据的字节数量，方便之后在数据中寻址使用。

data 是一个二维数组，依次存储每个像素点的 RGB 信息，RGB 分别是 3 个 8 位的数据表示，大家可以算一下，如果是一幅 1920*1080 分辨率的高清图片，这个数组将会保存 3*1920*1080 个字节的数据，也就是 6MB 左右的大小，假设摄像头每秒采集 20 幅图像，还要乘以 20，上百兆每秒的数据量着实不小，会占用大量的带宽资源。

➤ **Header**: 消息头，包含消息序号，时间戳和绑定坐标系；

➤ **height**: 图像的纵向分辨率；

➤ **width**: 图像的横向分辨率；

➤ **encoding**: 图像的编码格式，包含 RGB、YUV 等常用格式，不涉及图像压缩编码；

➤ **is_bigendian**: 图像数据的大小端存储模式；

➤ **step**: 一行图像数据的字节数量，作为数据的步长参数；

➤ **data**: 存储图像数据的数组，大小为 step*height 字节；

```

ros@u22:~/demo_ws$ ros2 interface show sensor_msgs/msg/Image
# This message contains an uncompressed image
# (0,0) is at top-left corner of image

std_msgs/Header header # Header timestamp should be acquisition time of image
  builtin_interfaces/Time stamp
    int32 sec
    uint32 nanosec
  string frame_id
    # Header frame_id should be optical frame of camera
    # origin of frame should be optical center of camera
    # +x should point to the right in the image
    # +y should point down in the image
    # +z should point into to plane of the image
    # If the frame_id here and the frame_id of the cameraInfo
    # message associated with the image conflict
    # the behavior is undefined

uint32 height
uint32 width
  # Image height, that is, number of rows
  # Image width, that is, number of columns

# The legal values for encoding are in file src/image_encodings.cpp
# If you want to standardize a new string format, join
# ros-users@lists.ros.org and send an email proposing a new encoding.

string encoding
  # Encoding of pixels -- channel meaning, ordering, size
  # taken from the list of strings in include/sensor_msgs/image_encodings.hpp

uint8 is_bigendian
uint32 step
uint8[] data
  # Is this data bigendian?
  # Full row length in bytes
  # actual matrix data, size is (step * rows)
  
```

sensor_msgs/Image

1920*1080分辨率的摄像头产生一帧图像的数据大小是：3*1920*1080=6220800字节≈6MB

所以在日常的数据传输中，尤其是通过无线网络传输时，所有涉及到多设备无线传输时更为常用的方式是使用压缩后的图像。

```
ros@u22:~/demo_ws$ ros2 interface show sensor_msgs/msg/CompressedImage
# This message contains a compressed image.

std_msgs/Header header # Header timestamp should be acquisition time of image
  builtin_interfaces/Time stamp
    int32 sec
    uint32 nanosec
  string frame_id
    # Header frame_id should be optical frame of camera
    # origin of frame should be optical center of camera
    # +x should point to the right in the image
    # +y should point down in the image
    # +z should point into to plane of the image

string format
    # Specifies the format of the data
    # Acceptable values:
    #   jpeg, png, tiff

uint8[] data
    # Compressed image buffer
```

sensor_msgs/CompressedImage

- **format**: 图像的压缩编码格式 (jpeg、png、bmp)
- **data**: 存储图像数据数组

识别程序编写

[启动仿真环境](#)后，可以看到仿真的相机通过/camera/image_raw 发布图像数据，消息为 Image, 编码格式 rgb8。

```
ros@u22:~/demo_ws$ ros2 topic list |grep image
/camera/image_raw
/camera_rgbd/camera_rgbd/depth/image_raw
/camera_rgbd/camera_rgbd/image_raw
ros@u22:~/demo_ws$ ros2 topic info /camera/image_raw
Type: sensor_msgs/msg/Image
Publisher count: 1
Subscription count: 0
```

```
ros@u22:~/demo_ws$ ros2 topic echo /camera/image_raw
header:
  stamp:
    sec: 2441
    nanosec: 58000000
  frame_id: camera_link
height: 720
width: 1280
encoding: rgb8
is_bigendian: 0
step: 3840
data:
- 52
- 52
```

参照之前的话题通信程序案例，可以实现图像订阅，格式为 Image，话题为 /camera/image_raw。

```
self.sub = self.create_subscription(\
    String, "chatter", self.listener_callback, 10)
```

```
self.sub = self.create_subscription(
    Image, 'image_raw', self.listener_callback, 10)
```

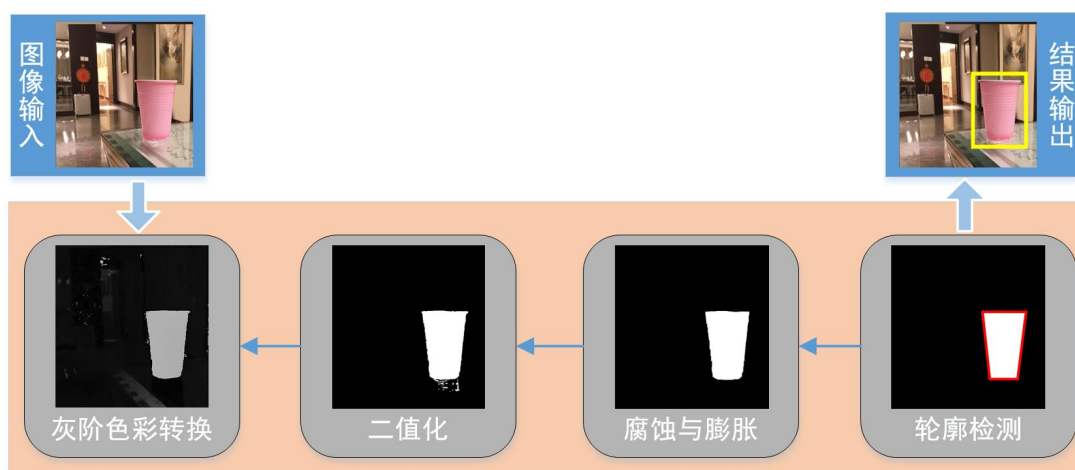
之后在回调函数 `self.listener_callback` 中通过 `opencv` 实现图像处理的逻辑；OpenCV 中针对图像数据有专门的定义，和 ROS 中的图像消息不同，为了可以兼容 OpenCV，需要使用 `CvBridge` 的功能包，它的任务很明确，就是将 ROS 中的图像消息转换为 OpenCV 中的图像数据结构，或者反过来，将 OpenCV 中的图像数据，转变成 ROS 中的图像消息。

`CvBridge` 就像是一座桥，打通了 ROS 与 OpenCV 之间的数据通道

修改 `hello_cv.py`

```
1 import rclpy                                # ROS2 Python接口库
2 from rclpy.node import Node                 # ROS2 节点类
3 from sensor_msgs.msg import Image           # 图像消息类型
4 from cv_bridge import CvBridge              # ROS与OpenCV图像转换类
5 import cv2                                  # OpenCV图像处理库
6 import numpy as np                          # Python数值计算库
7
8
9
10 创建一个订阅者节点
11
12 class ImageSubscriber(Node):
13     def __init__(self, name):
14         super().__init__(name)              # ROS2节点父类初始化
15         self.sub = self.create_subscription(
16             Image, '/camera/image_raw', self.listener_callback, 10) # 创建订阅者对象(消息类型、话题名、订阅者回调函数、队列长度)
17         self.cv_bridge = CvBridge()          # 创建一个图像转换对象，用于OpenCV图像与ROS的图像消息的互相转换
18
19     def listener_callback(self, data):
20         image = self.cv_bridge.imgmsg_to_cv2(data, 'bgr8') # 将ROS的图像消息转化成OpenCV图像
21         self.object_detect(image)            # 检测
22
23     def object_detect(self, image):
```

之后在 `object_detect` 中通过目标物体的特征实现检测逻辑；最直接的特征就是颜色，此外还有像素面积、轮廓检测、四边形检测、霍夫圆检测等



通常实现对特定颜色物体的检测需要把色彩模型从 RGB 转为 HSV。平时常见的图片一般都是 RGB 模型。但是这种方式不太符合人的直观感受，比如随便给你一个颜色，很少有人可以说出这个颜色对应的三原色比例。所以产生了第二种更

加符合人描述和解释的方式，叫做 HSV 模型。也就是每个像素点使用色调、饱和度、亮度进行描述。

HSV 模型由 Smith 在 1978 年创建，也称为倒锥形模型。

其中 H 表示色调，用模型的圆形边界描述，是一个 0 度到 360 度的角度值，在这个锥体模型上，以红色为起点，值为 0 度，逆时针方向计算，度数逐渐增大，直到 360 度又回到了红色。其中绿色，是 120 度，蓝色是 240 度，对应到三原色的话，刚好还是红绿蓝均分了色调 360 度的取值范围。

S 是饱和度，用模型的水平轴描述，表示颜色接近光谱色的程度，取值范围是 0%到 100%，值越大，颜色约饱和，也就是看上去越艳丽。

V 表示亮度，用模型的垂直轴描述，也就是颜色的明亮程度，取值范围也是 0%到 100%，如果是 0%的话，就是最暗的黑色，如果是 100%的，就是最亮的白色。

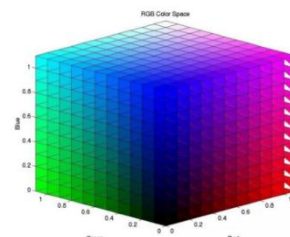
彩色模型

➤ RGB (红, 绿, 蓝)

广泛应用于彩色监视器和彩色视频摄像机，平时常见的图片一般都是RGB模型；

➤ HSV (色调, 饱和度, 亮度)

更符合人描述和解释颜色的方式，对人来说是自然且非常直观的。



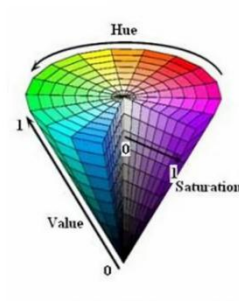
HSV模型

➤ A. R. Smith在1978年创建的一种颜色空间, 也称六角锥体模型(Hexcone Model);

➤ 色调 (H: hue) : 用角度度量, 取值范围为0°~360°, 从红色开始按逆时针方向计算, 红色为0°, 绿色为120°, 蓝色为240°;

➤ 饱和度 (S: saturation) : 取值范围为0%~100%, 值越大, 颜色越艳丽;

➤ 亮度 (V: value) : 取值范围为0%(黑色)~100%(白色), 值越大, 颜色越明亮。

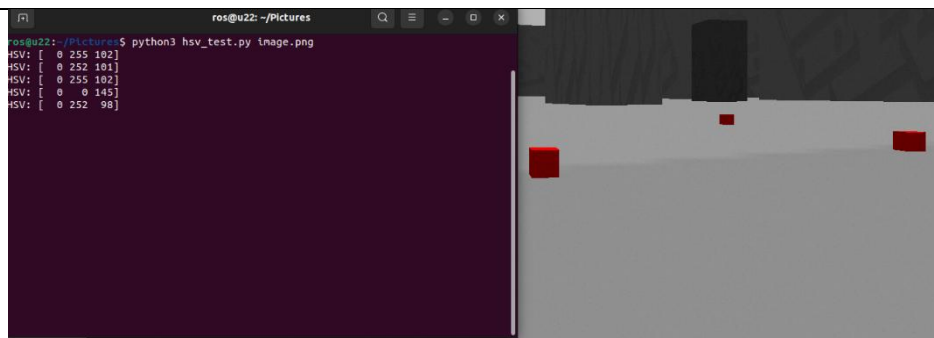


通过 opencv 可以很方便获取图片像素的 HSV, H[0-180], S[0-255], V[0-255]

```
img = cv2.imread(image_path)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
print("HSV:", hsv[y, x])
```

https://gitee.com/yzhcat/mcode_snippet/raw/master/hsv_test/hsv_test.py

	红	橙	黄	绿	青	蓝	紫	黑	灰	白
Hmin	0	156	11	26	35	78	100	125	0	0
Hmax	10	180	25	34	77	99	124	155	180	180
Smin	43	43	43	43	43	43	43	43	0	0
Smax	255	255	255	255	255	255	255	255	43	30
Vmin	46	46	46	46	46	46	46	46	46	221
Vmax	255	255	255	255	255	255	255	46	220	255



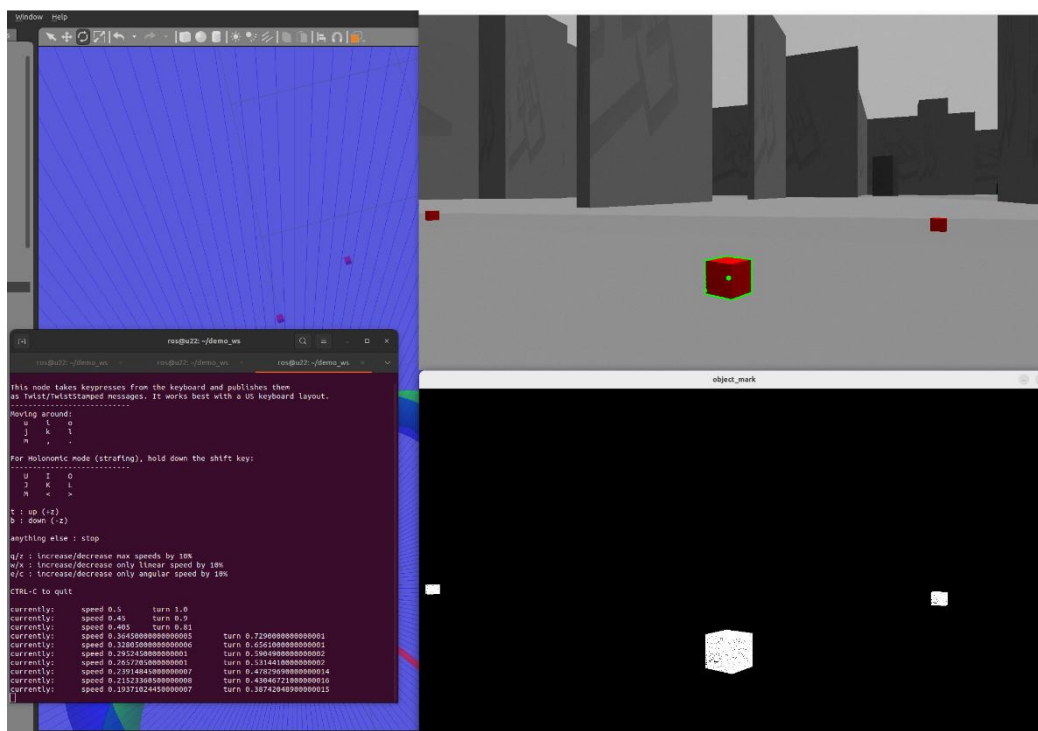
hello_cv.py 实现 object_detect 函数。

```
def object_detect(self, image):
    lower_red = np.array([0, 248, 91])    # 红色的Hsv阈值下限
    upper_red = np.array([0, 255, 255])   # 红色的Hsv阈值上限
    hsv_img = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)    # 图像从BGR颜色模型转换为Hsv模型
    mask_red = cv2.inRange(hsv_img, lower_red, upper_red) # 图像二值化
    contours, hierarchy = cv2.findContours(
        mask_red, cv2.RETR_LIST, cv2.CHAIN_APPROX_NONE) # 图像中轮廓检测

    for cnt in contours:                    # 去除一些轮廓面积太小的噪声
        if cnt.shape[0] < 150:
            continue

        (x, y, w, h) = cv2.boundingRect(cnt)    # 得到轮廓的左上角xy像素坐标及轮廓范围的宽和高
        cv2.drawContours(image, [cnt], -1, (0, 255, 0), 2) # 将轮廓勾勒出来
        cv2.circle(image, (int(x+w/2), int(y+h/2)), 5,
            (0, 255, 0), -1)                    # 将图像中心点画出来
    cv2.imshow("object_mark", mask_red)        # 使用OpenCV显示处理后的图像效果
    cv2.imshow("object", image)                # 使用OpenCV显示处理后的图像效果
    cv2.waitKey(10)
```

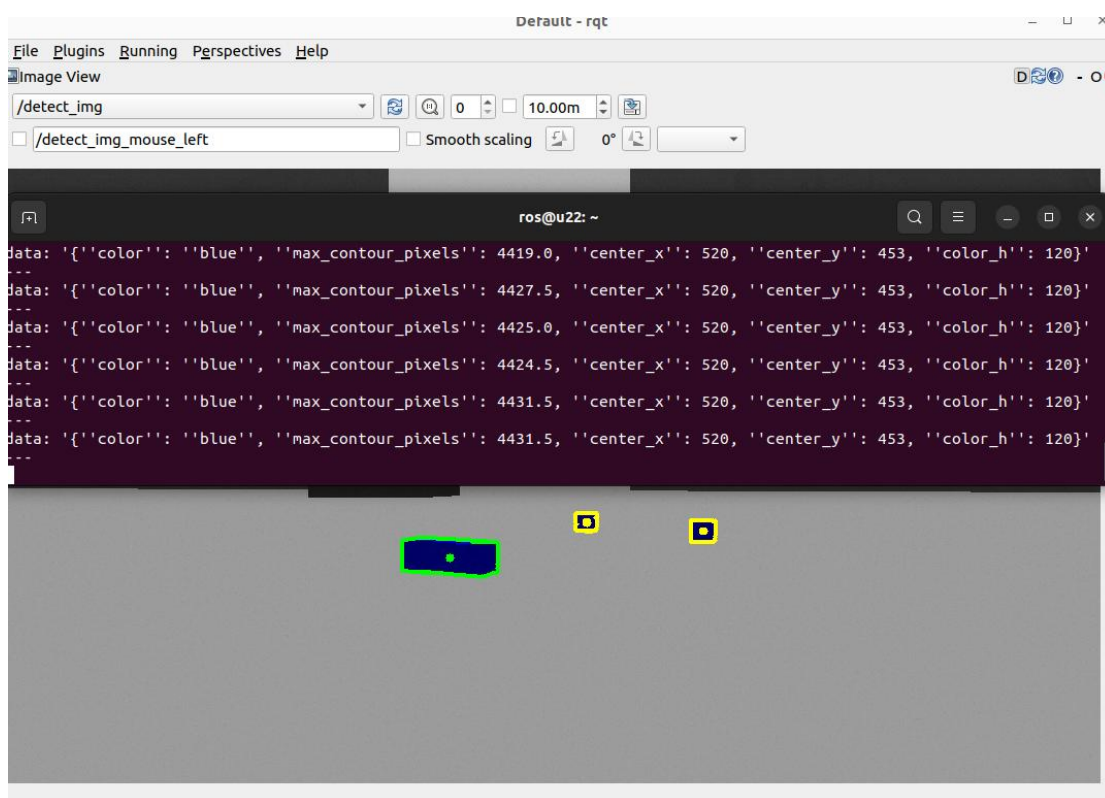
object_detect 中定义刚刚得到的红色颜色阈值，之后通过阈值使用 cv2.inRange 对图片二值化，cv2.findContours 轮廓检测并过滤小的轮廓，之后计算每个轮廓的中心点。



之后只需要把数据通过话题输出就可以了；再创建两个发布者，一个发布识别后的图像，消息格式 Image；一个发布识别信息，消息格式可以是字符串拼接或者自定义消息。

```
class ImageSubscriber(Node):
    def __init__(self, name):
        super().__init__(name)  # ROS2节点父类初始化
        self.sub = self.create_subscription(  # 创建一个图像转换对象，用于OpenCV图像与ROS的图像消息的互相转换
            Image, 'detect_img', self.listener_callback, 10)
        self.cv_bridge = CvBridge()
        self.color_lower = np.array([0, 248, 91])  # HSV阈值下限
        self.color_upper = np.array([120, 255, 255])  # HSV阈值上限
        self.pub_info = self.create_publisher(String, "detect_info", 10)  # 创建发布者对象（消息类型、话题名、队列长度）
        self.pub_img = self.create_publisher(Image, "detect_img", 10)  # 创建发布者对象（消息类型、话题名、队列长度）

    def listener_callback(self, data):
        image = self.cv_bridge.imgmsg_to_cv2(data, 'bgr8')  # 将ROS的图像消息转化成OpenCV图像
        result_img, detect_info = self.object_detect(image)  # 检测物体并获取结果
        #发布处理后的图像
        ros_img = self.cv_bridge.cv2_to_imgmsg(result_img, "bgr8")
        self.pub_img.publish(ros_img)
        #发布检测信息
        info_msg = String()
        info_msg.data = str(detect_info)
        self.pub_info.publish(info_msg)
```



八、语言大模型的应用

语言大模型（LLM）在人机交互中的应用场景广泛且具有变革性，智能客服与虚拟助手、车载与物联网交互等。

在技术层面通常的方案有 NLP 解析、MCP 架构、Function Calling 等。NLP 解析 适用于简单指令转换，快但泛化能力弱。如输入："左边第二个盒子拿给我" → NLP 模型 → 解析为 {"direction": "left", "index": 2, "object":

"box"} → 映射为机器人指令 `robot.move(dir=left,index=2,obj=box)`; 从自然语言提取关键信息再映射为类机器人指令，控制机器人完成任务。

llama.cpp

llama.cpp 是一个轻量级、高性能的 C/C++ 库，用于在常规硬件上高效运行大型语言模型（LLMs）。它最初是 Meta 的 LLaMA 模型的纯 C/C++ 实现，但已经发展成为一个全面的支持数十种流行 LLMs 的框架，具有显著的效率。

可以将 llama.cpp 视为本地运行 AI 模型的瑞士军刀——它允许你在自己的硬件上运行强大的语言模型，而无需云服务或庞大的基础设施。

关键特性

纯 C/C++ 实现，依赖最小

llama.cpp 的核心优势在于其精简的实现，不依赖于像 PyTorch 或 TensorFlow 这样的大型 AI 框架。这种“裸金属”方法意味着：

更小的占用空间 - 可在资源有限的设备上运行

更广泛的兼容性 - 跨操作系统和硬件工作

更易于集成 - 可嵌入各种应用程序

安装

llama.cpp 通过源码构建安装

```
git clone https://github.com/ggerganov/llama.cpp
cd llama.cpp
cmake -B build
cmake --build build --config Release
```

下载编译完成后，程序生产在 `~/llama.cpp/build/bin/` 目录。为了方便使用，可以把路径加到 `$PATH` 环境变量。

修改 `.bashrc` 文件，`gedit ~/.bashrc`，把下面代码添加到文件。

```
export PATH=~/llama.cpp/build/bin/:$PATH
```



下载模型文件

llama.cpp 使用 GGUF（GGML 通用格式）作为其主要模型格式。llama.cpp

支持广泛的模型架构，不仅仅是 LLaMA 模型：

- LLaMA 系列 (LLaMA、LLaMA-2、Llama-3 等)
- Falcon 模型
- Mistral 模型
- MPT 模型
- Mixtral 模型
- Phi 模型
- Qwen 模型
- BERT 模型
- Command-R 模型
- GPT-2 模型
- Baichuan 模型
- DBRX 模型
- 以及许多其他模型

可以在多个来源找到预转换的 GGUF 模型：

1. **Hugging Face:** 许多仓库托管了流行模型的 GGUF 版本
2. **TheBloke:** 提供了大量转换为 GGUF 的模型，具有各种量化级别
3. **官方模型发布:** 一些组织提供官方的 GGUF 版本

GGUF 模型获取方式参考：

<https://horizonrobotics.feishu.cn/docx/LQU9dYy.jcoXJ9hxJdUYc2l4InEf#IwuFdbqqbohF52xnS9IcyRTKnlg>

运行测试

以 Qwen 模型为例

<https://modelscope.cn/models/Qwen/Qwen2.5-Coder-0.5B-Instruct-GGUF/files>

下载模型，并把模型文件放到指定目录，如 `~/llama.cpp/`

```
wget
https://modelscope.cn/models/Qwen/Qwen2.5-Coder-0.5B-Instruct-GGUF/resolve/master/qwen2.5-coder-0.5b-instruct-q4_k_m.gguf
```

```
mv qwen2.5-coder-0.5b-instruct-q4_k_m.gguf ~/llama.cpp/
```

使用 llama-cli 加载模型：

```
llama-cli -m ~/llama.cpp/qwen2.5-coder-0.5b-instruct-q4_k_m.gguf -n 512 -c 2048 -sys "你是编程小助手" -co -cnv --threads 8
```

在终端即可与模型对话

```
- To return control without starting a new line, end your input with '/'.
- If you want to submit another line, end your input with '\'.

system
你是编程小助手

> c++中退出while循环的方法有几种
在C++中，退出'while'循环的方法有很多，具体取决于你想要在循环中退出的条件是什么。以下是几种常见的方法：

### 1. 使用'break'语句
'break'语句用于退出'while'循环。

```cpp
while (true) {
 // 你的循环代码
 if (someCondition) {
 break;
 }
}
```

### 2. 使用'continue'语句
'continue'语句用于跳过当前循环的剩余部分，并继续执行下一个循环。

```cpp
while (true) {
 // 你的循环代码
 if (someCondition) {
 continue;
 }
}
```

程序调用

使用服务端启动模型

```
llama-server -m ~/llama.cpp/qwen2.5-coder-0.5b-instruct-q4_k_m.gguf -c 2048 --threads 8
--port 8081
```

使用 requests 向服务端请求对话

```
16 print("输入 'quit' 或 'exit' 退出程序")
17 print("-" * 40)
18 while True:
19 try:
20 # 获取用户输入
21 user_input = input("\n> ").strip()
22
23 if user_input.lower() in ["quit", "exit"]: ...
24
25 if not user_input: ...
26
27 # 发送消息并获取回复
28 print("正在思考...")
29 response = self.send_message(user_input)
30 print(f"\n助手: {response}")
31
32 except KeyboardInterrupt: ...
33 except Exception as e: ...
34
35 def send_message(self, user_input):
36 # 构建消息历史
37 messages = [{"role": "system", "content": self.system_prompt}]
38 messages.extend(self.chat_history)
39 messages.append({"role": "user", "content": user_input})
40
41 data = { ...
42
43 try:
44 response = requests.post(
45 f"{self.base_url}/v1/chat/completions".
```

参考代码：

[https://gitee.com/yzhcat/mcode\\_snippet/raw/master/llama.cpp\\_demo/  
chat\\_interactive.py](https://gitee.com/yzhcat/mcode_snippet/raw/master/llama.cpp_demo/chat_interactive.py)

## 运行程序

```
python3 chat_interactive.py
```

```
ros@u22:~/dev_ws/src/yzbot/tools_demo/tools_demo$ python3 chat_interactive.py
=== Llama 交互式聊天程序 ===
输入 'quit' 或 'exit' 退出程序

> hello
正在思考...

助手: Hello! How can I assist you today?

> 1m=?cm
正在思考...

助手: 1 meter is equal to 100 centimeters.

> exit
再见!
```

接入到 ros

```
3 import rclpy
4 from rclpy.node import Node
5 import json
6
7
8 class LlamaChat(Node):
9 """LlamaChat 类,
10 需要启动 llama-server -m ~/llama.cpp/qwen2.5-coder-0.5b-instruct-q4_k
11 """
12
13 def __init__(self, base_url="http://localhost:8081"):
14 super().__init__("LlamaChat")
15 self.base_url = base_url
16 self.chat_history = []
17 self.system_prompt = '你是编程小助手。'
18
19 # 创建话题发布者
20 self.command_publisher = self.create_publisher(
21 String, "llama_command", 10)
22
```

使用 rclpy.node.Node 创建发布者，把服务端反馈的文本发送到 ros 话题。

```
45
46 # 询问处理选项
47 while True:
48 option = input("\n选择处理方式 [N/R]: ").strip()
49 if option.upper() in ["N", "R"]:
50 self.process_response(response, option)
51 break
52 else:
53 print("请输入有效选项: N, R")
54
55 def process_response(self, response, option):
56 """处理回复的不同选项"""
57 if option.upper() == "N":
58 print("不处理, 继续对话...")
59 elif option.upper() == "R":
60 print("发布到话题...")
61 # 创建String消息并发布
62 msg = String()
63 msg.data = response
64 self.command_publisher.publish(msg)
65 print(f"已发布消息: {response}")
66
```

```
ros2 run chat_interactive_ros.py
```

参考代码:

[https://gitee.com/yzhcat/mcode\\_snippet/raw/master/llama.cpp\\_demo/chat\\_interactive\\_ros.py](https://gitee.com/yzhcat/mcode_snippet/raw/master/llama.cpp_demo/chat_interactive_ros.py)



```

=== Llama 交互式聊天程序 ===
输入 'quit' 或 'exit' 退出程序
每次回复后可选择处理方式:
N - 不处理
R - 发布到话题

> hello
正在思考...

助手: Hello! How can I assist you today?

选择处理方式 [N/R]: n
不处理, 继续对话...

> 1m=2cm
正在思考...

助手: 1 meter is equal to approximately 3.6284 centimeters.

选择处理方式 [N/R]: r
发布到话题...
已发布消息: 1 meter is equal to approximately 3.6284 centimeters.

> exit
再见!
^CKeyboard interrupt received, shutting down.

```

```

ros@u22:~$ ros2 topic echo /llama_command
data: 1 meter is equal to approximately 3.6284 centimeters.

```

调整提示词, 之后按照模型回复的结果, 依次执行导航到 color 位置, 拾取并导航 to 区域放置物块, 循环 num 次。

```

13
14
15 def __init__(self, base_url="http://localhost:8081"):
16 super().__init__("LlamaChat")
17 self.base_url = base_url
18 self.chat_history = []
19 self.system_prompt = '''
20 You are a robot pick-and-place assistant.
21 Return only pure JSON array, no extra text.
22 Example input: "我要两个蓝色方块到A区, 三个红色去b区"
23 Example output: [{"color": "blue", "num": 2, "to": "A"}, {"color": "red", "num": 3, "to": "b"}]
24
25 Example input: "三个红色去C区"
26 Example output: [{"color": "red", "num": 3, "to": "c"}]
27
28 Now extract from user: {{USER_INPUT}}
29 '''
30
31 # 创建话题发布者 (替换服务客户端)
32 self.command_publisher = self.create_publisher(
33
34 print("=== Llama 交互式聊天程序 ===")
35 print("输入 'quit' 或 'exit' 退出程序")
36 print("每次回复后可选择处理方式:")
37 print("N - 不处理")
38 print("R - 发布到话题")
39 print("_" * 40)
40 while True:
41 try:

```

```

data: []

data: []

data: [{"color": "blue", "num": 2, "to": "A"}, {"color": "red", "num": 3, "to": "b"}]

data: [{"color": "blue", "num": 3, "to": "a"}]

data: [{"color": "red", "num": 2, "to": "c"}, {"color": "blue", "num": 3, "to": "b"}]

data: [{"color": "red", "num": 1, "to": "a"}]

data: [{"color": "red", "num": 1, "to": "a"}]

data: [{"color": "blue", "num": 2, "to": "A"}, {"color": "red", "num": 3, "to": "b"}]

data: [{"color": "red", "num": 2, "to": "b"}, {"color": "blue", "num": 3, "to": "c"}]

ros@u22:~$

```

```

ros@u22:~/dev_ws/src/yzbot/tools_demo/tools_demo
N - 不处理
R - 发布到话题

> 两个蓝色块
正在思考...

助手: [{"color": "blue", "num": 2, "to": "B"}]

选择处理方式 [N/R]: n
不处理, 继续对话...

> 2 to c; 3 to b
正在思考...

助手: [{"color": "blue", "num": 2, "to": "B"}, {"color": "red", "num": 3, "to": "c"}]

选择处理方式 [N/R]: n
不处理, 继续对话...

> 2 red to b; blue to c need 3
正在思考...

助手: [{"color": "red", "num": 2, "to": "b"}, {"color": "blue", "num": 3, "to": "c"}]

选择处理方式 [N/R]: r
发布到话题...
已发布消息: [{"color": "red", "num": 2, "to": "b"}, {"color": "blue", "num": 3, "to": "c"}]

```

## 参考资料

llama.cpp 源码: <https://github.com/ggml-org/llama.cpp>

AI 文档: <https://zread.ai/ggml-org/llama.cpp>

## 九、可视化工具使用

### RQt

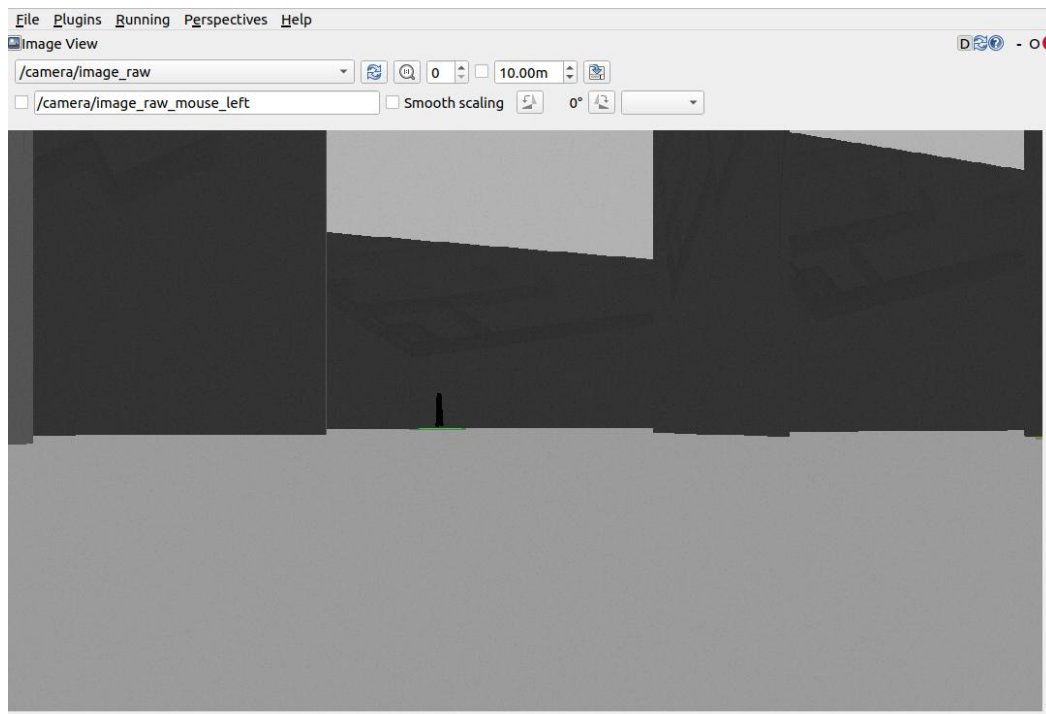
RQt 是一个图形用户界面框架, 实现了各种工具和接口的插件形式。如 ros 图像话题的显示, 话题、服务、动作通信的调用, TF tree, 节点图等。

使用命令安装 rqt 和 rqt 插件

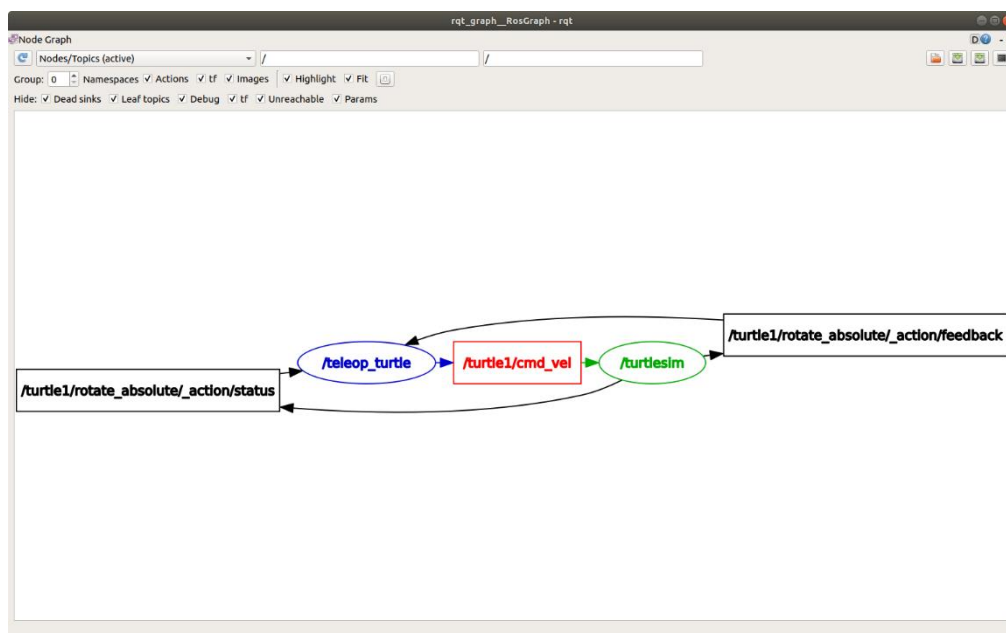
```
sudo apt install ros-humble-rqt*
```

比较常用的插件

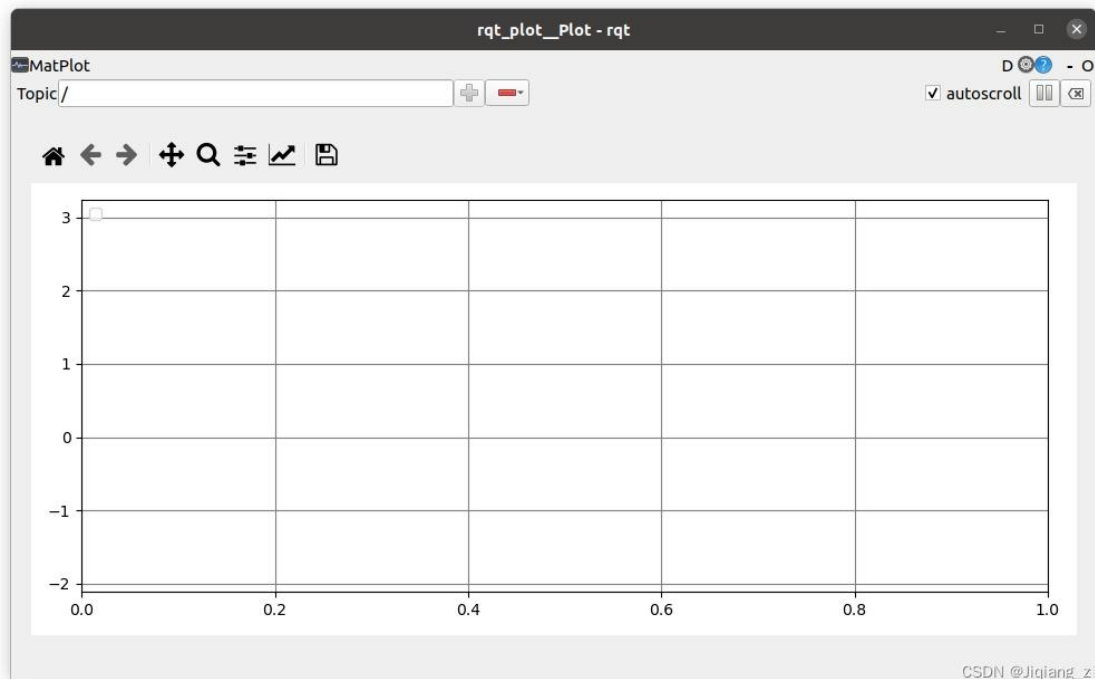
rqt\_image\_view: 可视化 ROS 图像



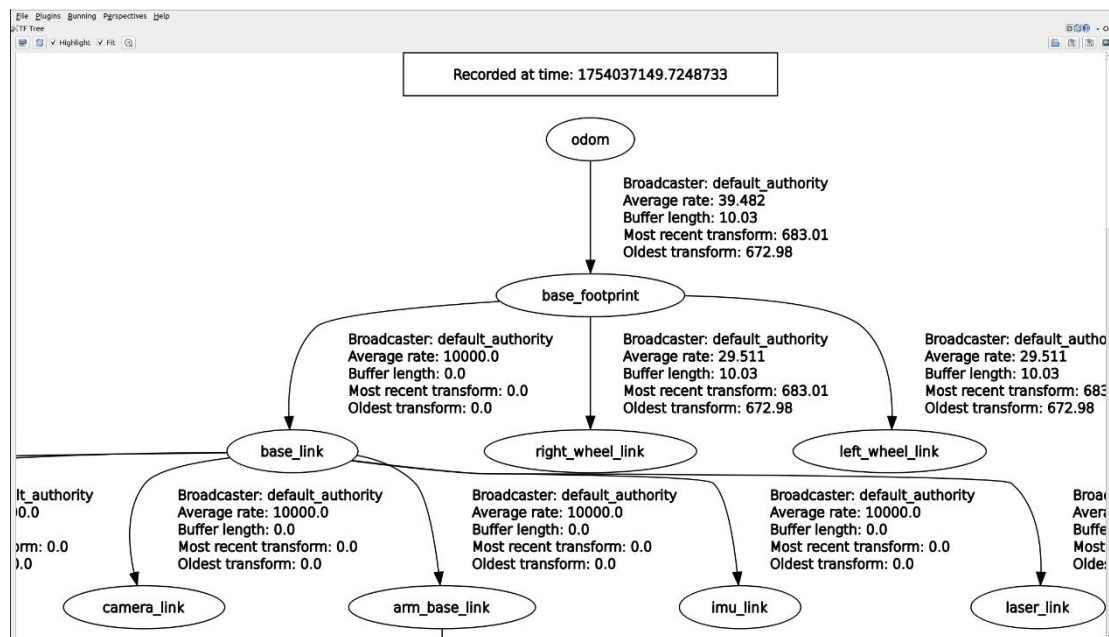
rqt\_graph: 可视化显示计算图



rqt\_plot: 图形绘制插件, 可以以 2D 绘图的方式绘制发布在 topic 上的数据



rqt\_tf\_tree: 显示坐标变换树图



<https://docs.ros.org/en/humble/Concepts/Intermediate/About-RQt.html>

[ml](#)

## RViz

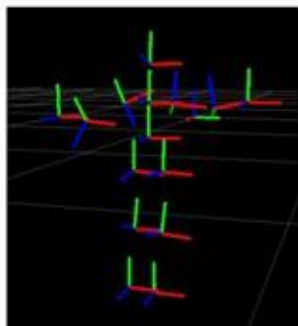
RViz 是 ROS 框架的 3D 可视化工具。

机器人开发过程中，各种各样的功能，如果我们只是从数据层面去做分析，

很难快速理解数据的效果，比如给你一堆 0 到 255 的数字，问这幅图像描述的内容是什么？你肯定一脸懵。但如果我们把这些数字通过颜色渲染出来，岂不就一目了然么？



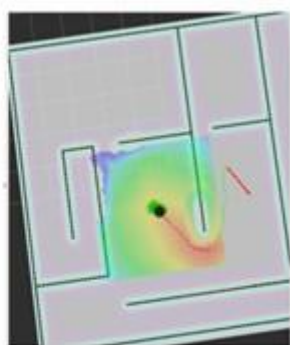
机器人模型



坐标



运动规划

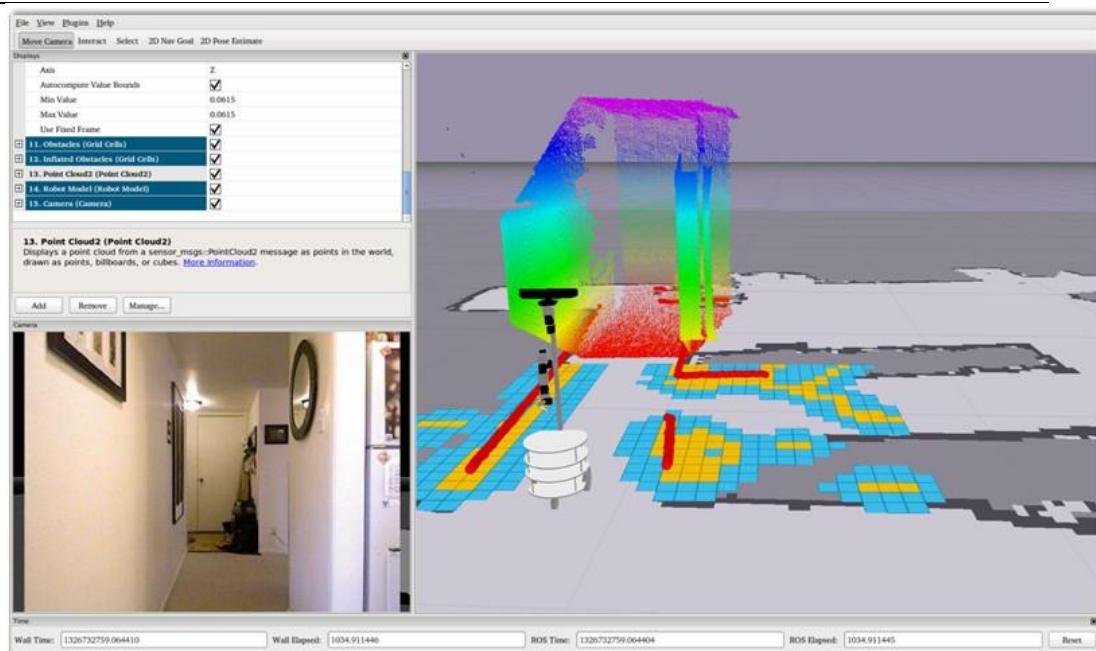


导航

类似的场景还有很多，比如机器人模型，我们需要知道自己设计的模型长啥样，还有模型内部众多坐标系在运动过程中都在哪些位置。

再比如机械臂运动规划和移动机器人自主导航，我们希望能看到机器人周边的环境、规划的路径，当然还有传感器的信息，摄像头、三维相机、激光雷达等等，数据是用来做计算的，可视化的效果才是给人看的。

所以，数据可视化可以大大提高开发效率，Rviz 就是这样一款机器人开发过程中的数据可视化软件，机器人模型、传感器信息、环境信息等等，全都可以在这里搞定。



Rviz 的核心框架是基于 **Qt 可视化工具** 打造的一个开放式平台，官方出厂就自带了很多机器人常用的可视化显示插件，只要我们按照 ROS 中的消息发布对应的话题，就可以看到图形化的效果了。如果我们对显示的效果不满意，或者想添加某些新的显示项，也可以在 Rviz 这个平台中，开发更多可视化效果，方便打造我们自己的上位机。

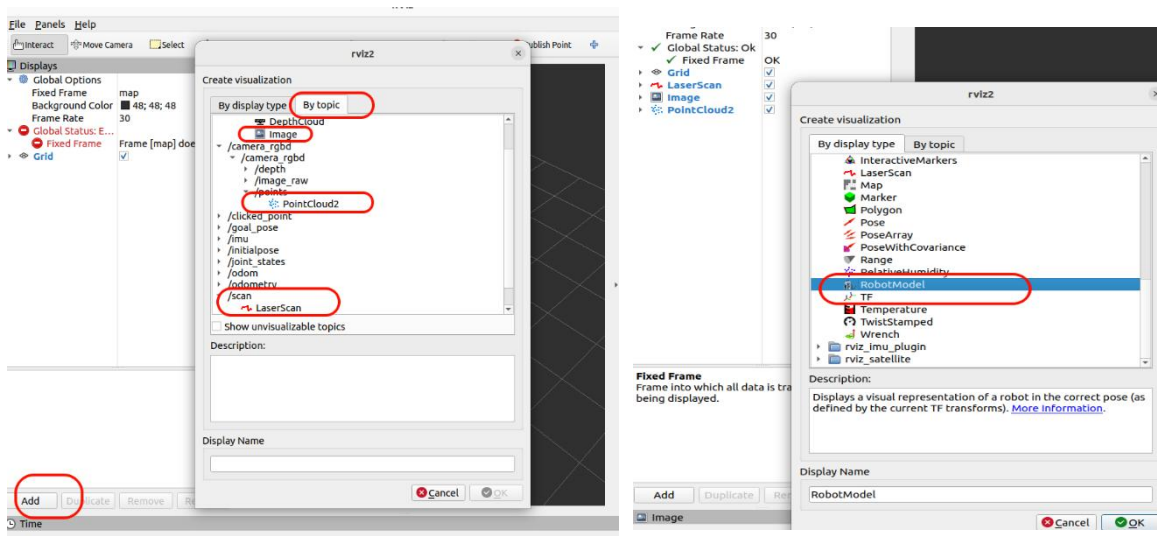
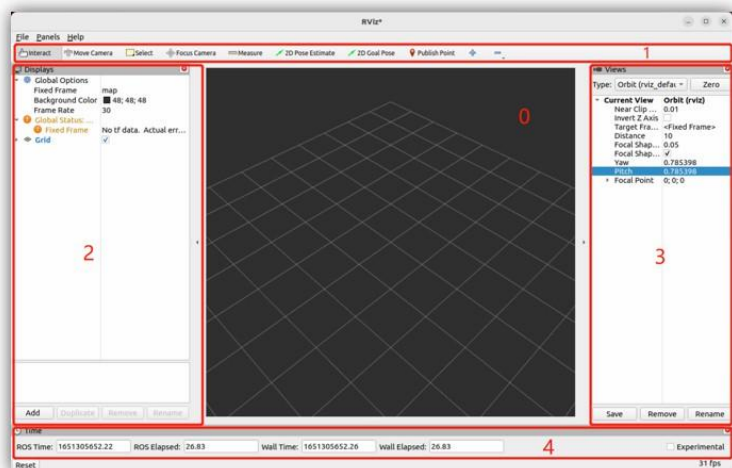
### 运行方法

启动一个终端，使用如下命令即可启动：

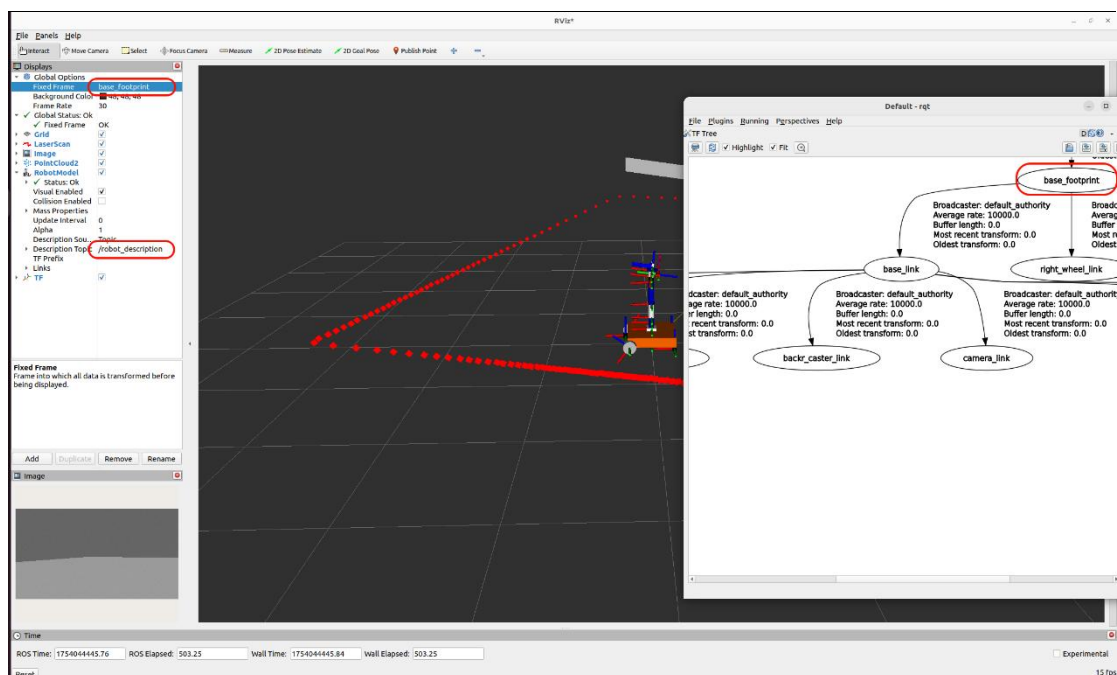
```
rviz2
```



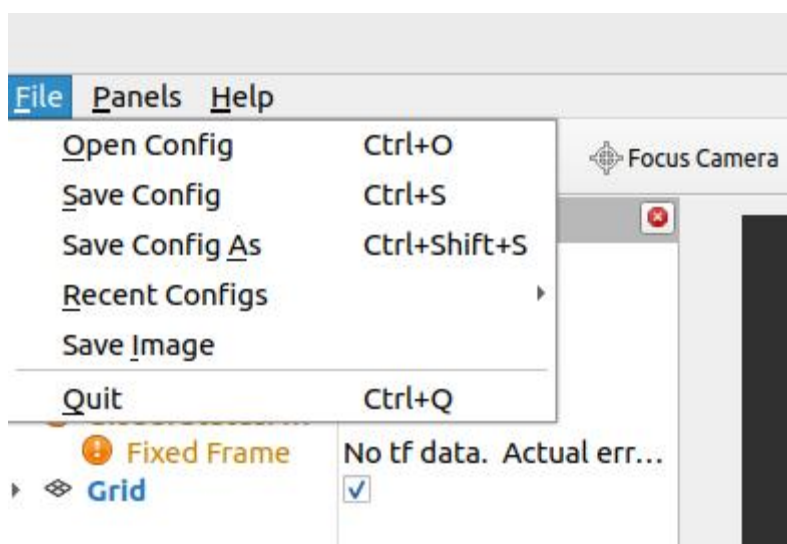
- **0 : 3D视图区**
- **1 : 工具栏**
- **2 : 显示项列表**
- **3 : 视角设置区**
- **4 : 时间显示区**



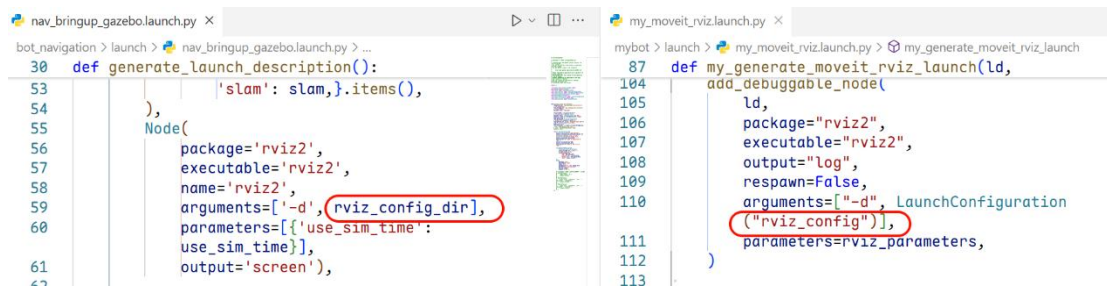
点击下方 Add 按钮，By topic 可以添加已经存在的数据源，如图像，雷达，点云等数据，By display type 可以添加其他类型数据，如 TF 树，模型。



设置固定坐标系和模型可以看到数据已经显示了。



布局的配置文件可以导入、导出；启动导航和 moveit 时，rviz 加载了对应的文件。



<https://book.guyuehome.com/ROS2/3.常用工具/3.5 Rviz/>

<https://docs.ros.org/en/humble/Tutorials/Intermediate/RViz/RViz-M>

## Foxglove

Foxglove Studio 是一个跨平台机器人数据可视化和分析的开源工具。支持实时 ROS 1 / ROS 2 数据流、离线 MCAP / .bag / .db3 / PX4 ULog 回放。

维度	RQt	RViz	Foxglove Studio
UI 技术	Qt-Widgets	OGRE 3D/Qt	Web/Electron + WebGL
运行环境	需本地 ROS	需本地 ROS	零 ROS 本地依赖
扩展机制	Python 插件	C++/Python Display/Plugin	任意 JS/TS Extension
数据格式	ROS 消息	ROS 消息	ROS、MCAP、Protobuf、JSON、自定义
典型用途	时序曲线、参数调参	三维机器人状态	实时/离线混合调试、跨平台仪表盘

使用 WebSocket 协议与机器人系统通信，能够处理实时的传感器数据和控制指令。可以在浏览器 (<https://app.foxglove.dev>) 或桌面端 (Win / macOS / Linux) 零依赖运行。

## Rosbridge

[Robot Web Tools](#) 为 ROS 构建基于 Web 的机器人应用程序的开源库和工具。为非 ROS 程序提供了 API 实现到 ROS 的功能如 Node.js, JavaScript。

rosbridge 为 ROS 提供了一个 JSON 接口，允许任何客户端发送 JSON 来发布或订阅 ROS 主题、调用 ROS 服务等。rosbridge 支持多种传输层，包括 WebSockets 和 TCP。



**Foxglove Studio**  
Visualization and debugging tool for live or recorded data



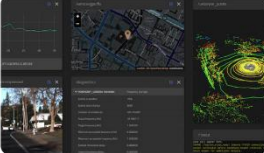
**transAct**  
An open-source robot fleet management dashboard



**ROS Control Center**  
Control center for ros robots



**ROSweb**  
Supervisory system for ROS

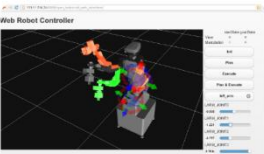


**ROSboard**  
turn your robot into a web server to visualize ROS topics

**Nodes**



**rosbridge**  
websocket interface to ROS systems



**visualization rwt**  
suite of nodes for web based robot visualization

**Image Topics using WebRTC**

WebRTC peer that can be configured to stream a image topic. The node hosts a webserver that can be used to create and configure a WebRTC peer. For more information, see the [ROS wiki](#).

Used as part of the [Robot Web Tools](#) effort.

**WebRTC ROS**  
Streaming of ROS Image Topics using WebRTC

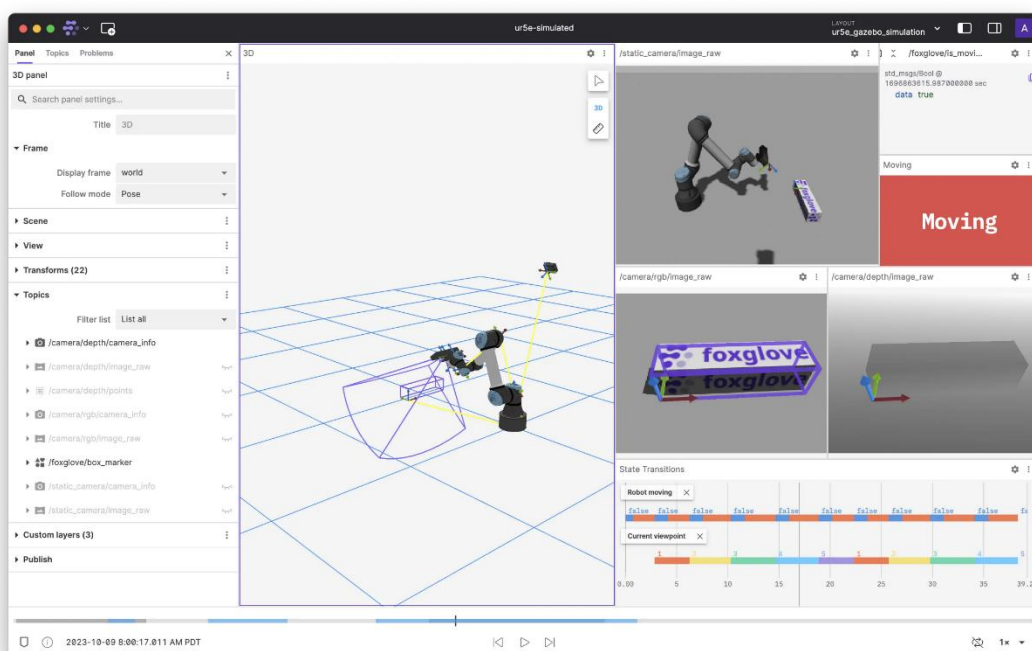
rosbridge\_suite 是一个 ROS 元包，包含所有 rosbridge 包。

可以使用 `sudo apt install ros-$ROS_DISTRO-rosbridge-suite` 命令安装。

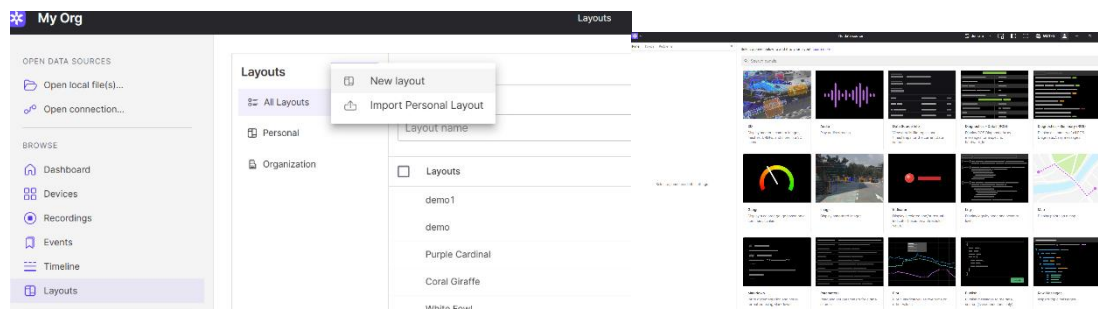
当启动 `ros2 launch rosbridge_server rosbridge_websocket_launch.xml` 命令后，所有的 ros 数据都会通过 WebSockets 转发。它把 ROS 话题/服务/参数转译成 JSON → WebSocket → 任何客户端。

rosbridge 客户端是使用其 JSON API 与 rosbridge 通信的程序。Rosbridge 客户包括 JavaScript API、Java API、Rust API，通过 WebSockets 与 rosbridge 通信。

Foxglove Studio 把 rosbridge 当作数据源之一，基于 WebSockets 协议实现 3D、Map、Image、Plot、Log、Raw Message、Parameters 等多种数据的可视化显示。相比于 rqt,rviz 可以更灵活的自定义面板，定制可视化效果。

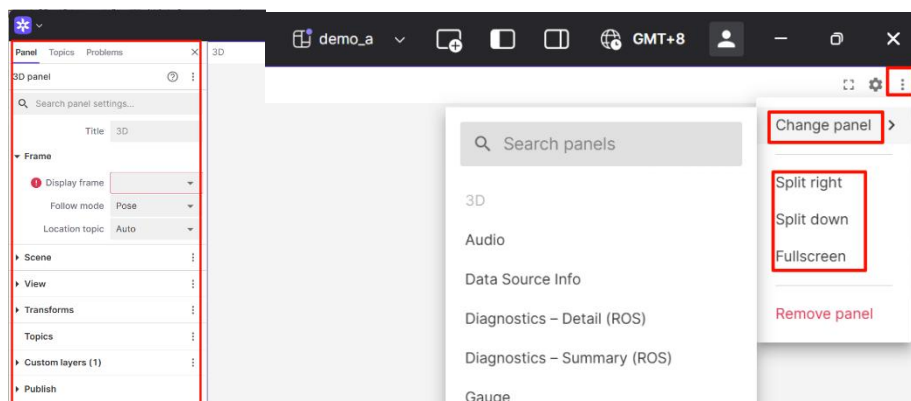


## 自定义布局

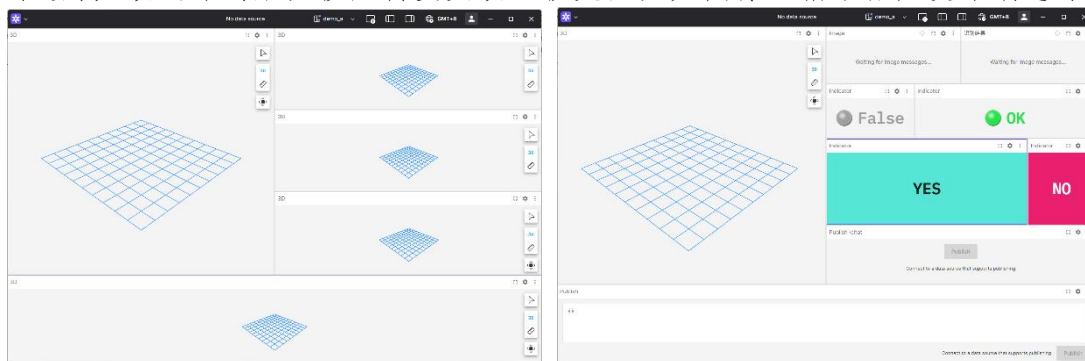


创建一个空的布局后，会显示默认支持的面板类型如 3D 面板。

点击面板，左侧会显示面板的配置选项。面板右上角可以更改或分割面板。



创建好大致的布局后，按照需要更改面板类型，如图像、指示器，发布消息等。



## 与 ros 通信

面板创建之后如何与 ROS 交互？

只需要启动 rosbridge 转发程序，在 foxglove 输入数据设备的 IP 和端口，然后正常启动 ros 程序就可以了。

```
ros2 launch rosbridge_server rosbridge_websocket_launch.xml
```

```
ros@u22:~$ ros2 launch rosbridge_server rosbridge_websocket_launch.xml
[INFO] [launch]: All log files can be found below /home/ros/.ros/log/2025-08-15-19-39-19-558982-u22-42836
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [rosbridge_websocket-1]: process started with pid [42837]
[INFO] [rosapi_node-2]: process started with pid [42839]
[rosbridge_websocket-1] [WARN] [1755157160.568996927] [rosbridge_websocket]: The 'default_call_service_timeout' parameter is currently set to 0.0, which means service calls will block indefinitely if no response is received. Please note that in the Jazzy and later releases, the default value for this parameter will be updated to 5.0 seconds.
[rosbridge_websocket-1] [WARN] [1755157160.569818113] [rosbridge_websocket]: The 'call_services_in_new_thread' parameter is currently set to False, which means service calls will block the main thread. Please note that in the Jazzy and later releases, the default value for this parameter will be updated to True.
[rosbridge_websocket-1] [WARN] [1755157160.570187805] [rosbridge_websocket]: The 'send_action_goals_in_new_thread' parameter is currently set to False, which means sending action goals will block the main thread. Please note that in the Jazzy and later releases, the default value for this parameter will be updated to True.
[rosbridge_websocket-1] [INFO] [1755157160.57618216] [rosbridge_websocket]: Rosbridge WebSocket server started on port 9090

ros@u22:~$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
 inet 192.168.110.182 netmask 255.255.255.0 broadcast 192.168.110.255
 inet6 fe80::d32c:b185:4d3b:b663 prefixlen 64 scopeid 0x20<link>
 ether 08:0e:29:7b:91:38 txqueuelen 1000 (Ethernet)
 RX packets 472655 bytes 43922032 (439.2 MB)
 RX errors 0 dropped 0 overruns 0 frame 0
 TX packets 225845 bytes 62861079 (62.8 MB)
 TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

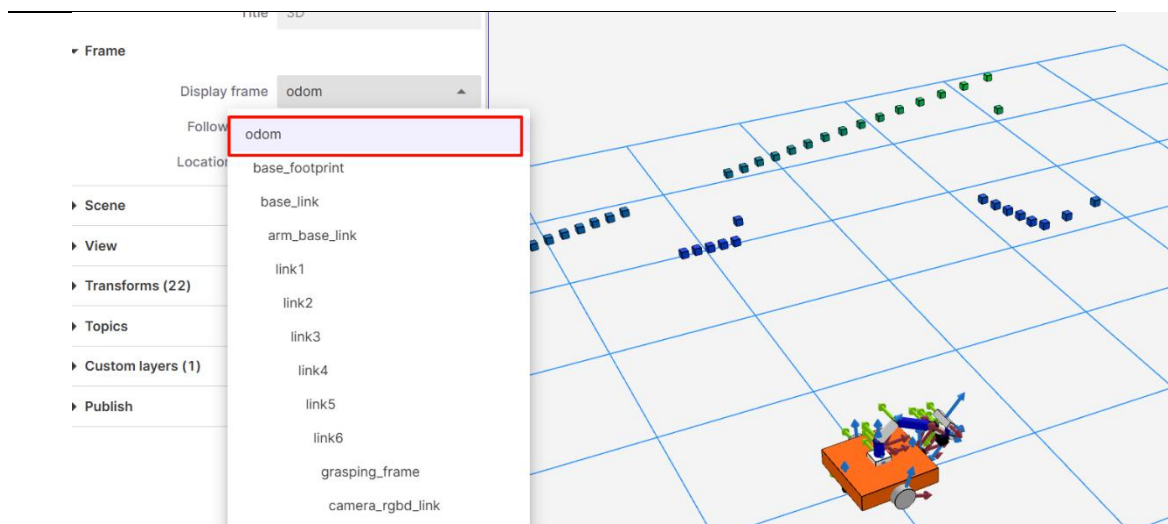
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
 inet 127.0.0.1 netmask 255.0.0.0
```

启动之后默认端口 9090，当前的 IP 为 192.168.110.182。在左侧 “Open Data sources” → “Open connection” → 选 Rosbridge，URL 填写 ws://192.168.110.182:9090，点击 “Open”，之后正常启动 ros 程序。

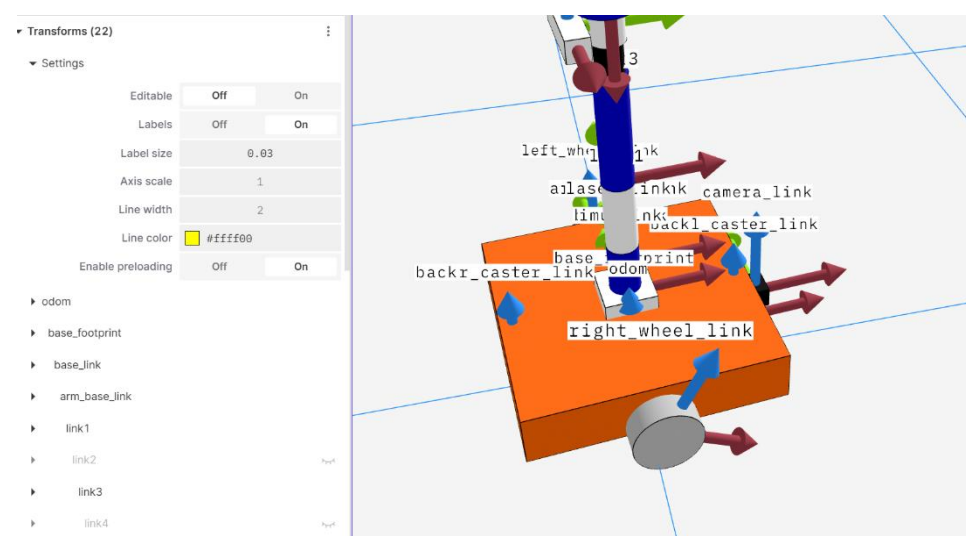


The screenshot shows the ROS2 Rviz2 interface with the 'Image' panel selected. The 'Topic' dropdown menu is open, showing a list of available topics. The selected topic is '/camera/image\_raw'. The interface also shows the 'General' tab and various settings like 'Flip vertical', 'Rotation', 'Color mode', and 'Value min'.

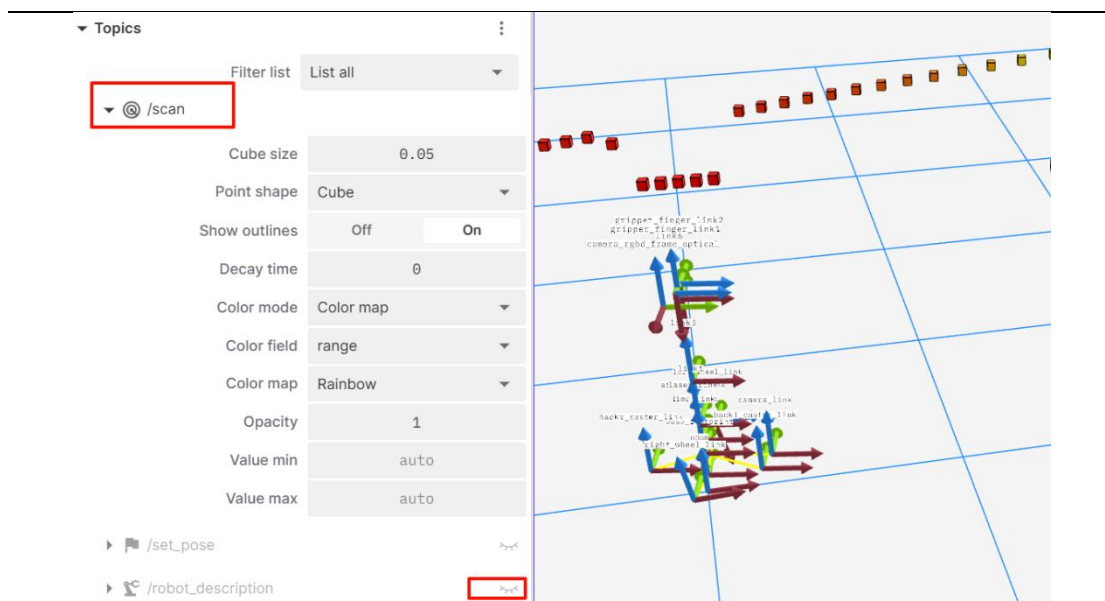
在 3D 面板选择显示的坐标系，通常为 map、odom 或者 base。



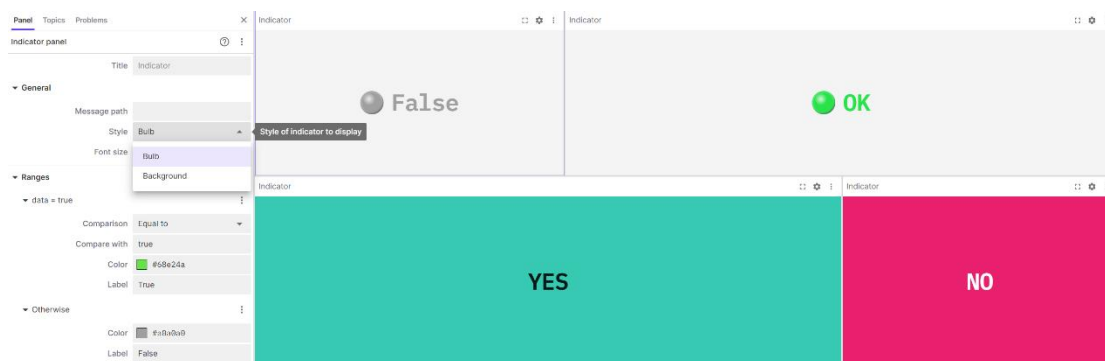
在 transforms 选项可以设置各个坐标系显示效果，或者隐藏。



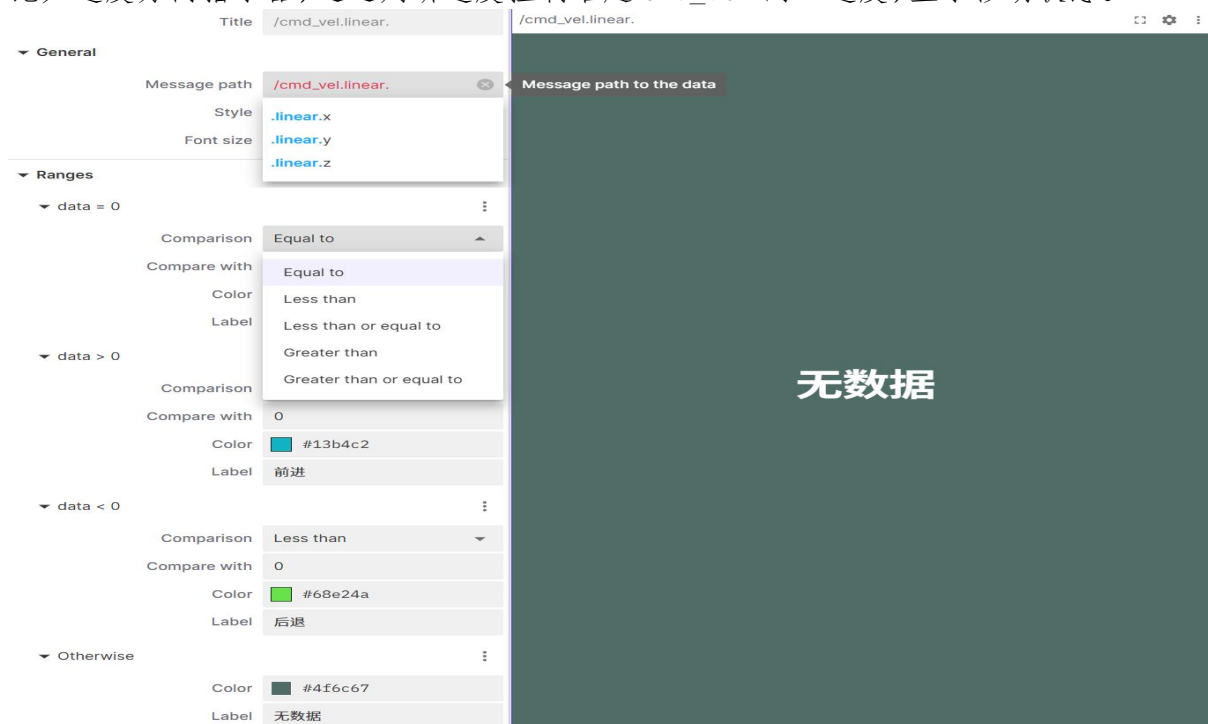
在 Topics 选项中可以自定义显示的话题，如雷达数据，模型显示。每个话题可以自定义显示效果，如雷达的点云大小，颜色设置。



指示器面板有两种样式；可以通过简单的规则判断，实现自定义内容显示。



比如速度方向指示器,通过判断速度控制话题 `cmd vel` 的 `x` 速度,显示移动状态。



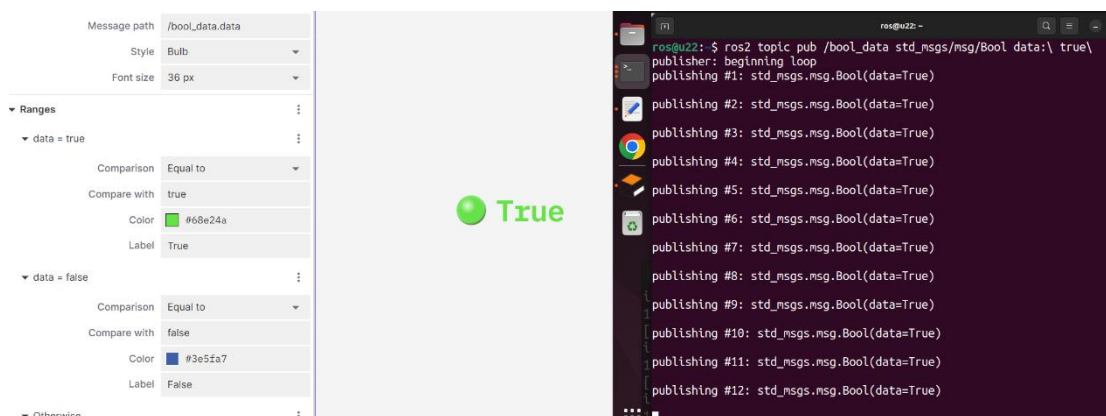
```
ros@u22:~$ ros2 interface show std_msgs/msg/Int16
This was originally provided as an example message.
It is deprecated as of Foxy
It is recommended to create your own semantically meaningful message.
However if you would like to continue using this please use the equivalent in example_msgs.

int16 data
ros@u22:~$ ros2 interface show std_msgs/msg/Int
std_msgs/msg/Int16 std_msgs/msg/Int32 std_msgs/msg/Int64 std_msgs/msg/Int8
std_msgs/msg/Int16MultiArray std_msgs/msg/Int32MultiArray std_msgs/msg/Int64MultiArray std_msgs/msg/Int8MultiArray
ros@u22:~$ ros2 interface show std_msgs/msg/Bool
This was originally provided as an example message.
It is deprecated as of Foxy
It is recommended to create your own semantically meaningful message.
However if you would like to continue using this please use the equivalent in example_msgs.

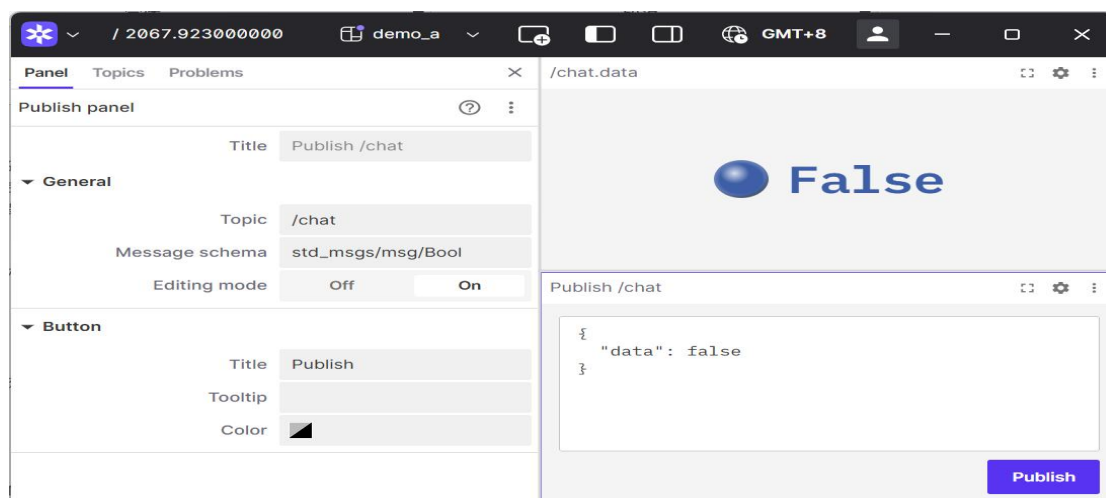
bool data
ros@u22:~$ ros2 interface show std_msgs/msg/String
This was originally provided as an example message.
It is deprecated as of Foxy
It is recommended to create your own semantically meaningful message.
However if you would like to continue using this please use the equivalent in example_msgs.

string data
```

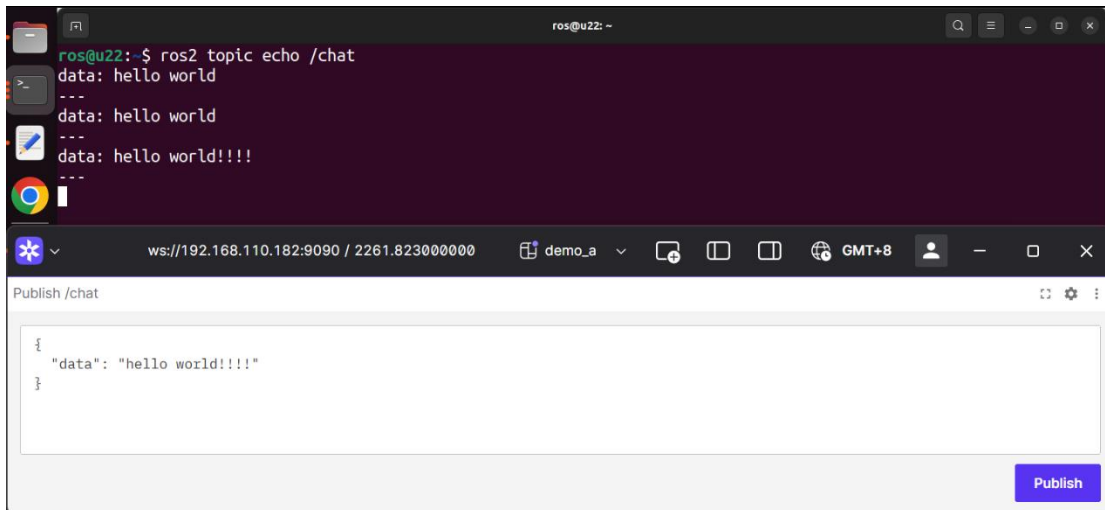
通常的状态显示可以使用简单类型，如 int,bool,string，这些 ros 消息格式内容比较简单，只有一个 data 数据。



发布面板可以向指定话题发布指定格式的数据。如向 chat 话题发布 bool 数据。此时仪表面板也能正常收到消息切换状态。



面板改成 String 使用 ROS 可以订阅话题中的数据。



The screenshot shows a web browser window with two main panels. The top panel is a terminal window titled 'ros@u22: ~' with a dark background. It shows the command 'ros2 topic echo /chat' being executed, followed by three lines of output: 'data: hello world', 'data: hello world', and 'data: hello world!!!!'. The bottom panel is a JSON editor titled 'Publish /chat'. It contains a JSON object: 

```
{
 "data": "hello world!!!!"
}
```

. A 'Publish' button is located at the bottom right of the JSON editor.

## 参考资料

rosbridge\_suite: [https://wiki.ros.org/rosbridge\\_suite](https://wiki.ros.org/rosbridge_suite)

Robot Web Tools: <https://robotwebtools.github.io/>

foxglove: <https://foxglove.dev/examples>

Foxglove Documentation: <https://docs.foxglove.dev/docs>